

node

当然可以。下面给你一份 **Node.js 入门教学路线**，并先从第一课开始讲。

Node.js 是什么？

Node.js 是一个可以让 JavaScript 运行在服务器端的环境。

以前 JavaScript 主要运行在浏览器里，比如：

```
alert("Hello")
```

而有了 Node.js，你可以用 JavaScript 做后端开发，比如：

- 搭建 Web 服务器
 - 操作文件
 - 连接数据库
 - 写接口 API
 - 做爬虫
 - 写命令行工具
 - 开发实时聊天系统
-

一、学习 Node.js 前需要会什么？

建议你先掌握：

1. JavaScript 基础

比如：

```
let name = "Tom"
const age = 18

function sayHello() {
  console.log("Hello")
}

sayHello()
```

需要了解：

- 变量
- 函数
- 对象

- 数组
 - 条件判断
 - 循环
 - Promise
 - async / await
-

二、安装 Node.js

进入官网：

```
https://nodejs.org
```

下载 **LTS 版本**。

安装完成后，在终端输入：

```
node -v
```

如果看到类似：

```
v20.11.0
```

说明安装成功。

再输入：

```
npm -v
```

如果出现版本号，说明 npm 也安装好了。

三、第一个 Node.js 程序

新建一个文件：

```
app.js
```

写入：

```
console.log("Hello Node.js")
```

然后在终端运行：

```
node app.js
```

输出：

```
Hello Node.js
```

这就是你的第一个 Node.js 程序。

四、Node.js 和浏览器 JavaScript 的区别

浏览器中的 JavaScript 可以操作页面

比如：

```
document.querySelector("h1")
```

但是 Node.js 没有 `document`，因为它不是运行在浏览器里。

Node.js 可以操作服务器资源

比如读取文件：

```
const fs = require("fs")

const content = fs.readFileSync("./test.txt", "utf-8")

console.log(content)
```

浏览器普通 JavaScript 不能直接这样操作本地文件。

五、Node.js 核心模块

Node.js 自带很多模块，常用的有：

模块	作用
fs	文件操作
path	路径处理
http	创建服务器
url	解析 URL
crypto	加密

模块	作用
events	事件处理

六、创建一个简单服务器

新建文件：

```
server.js
```

写入：

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "text/plain; charset=utf-8")
  res.end("你好，这是我的第一个 Node.js 服务器")
})

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

运行：

```
node server.js
```

然后打开浏览器访问：

```
http://localhost:3000
```

你会看到：

```
你好，这是我的第一个 Node.js 服务器
```

七、npm 是什么？

npm 是 Node.js 的包管理工具。

可以用它安装第三方库。

比如安装 Express：

```
npm install express
```

Express 是一个常用的 Node.js Web 框架。

八、使用 Express 创建服务器

先初始化项目：

```
mkdir node-demo
cd node-demo
npm init -y
```

安装 Express：

```
npm install express
```

新建：

```
app.js
```

写入：

```
const express = require("express")

const app = express()

app.get("/", (req, res) => {
  res.send("Hello Express")
})

app.get("/user", (req, res) => {
  res.json({
    name: "张三",
    age: 18
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

运行：

```
node app.js
```

访问：

```
http://localhost:3000
```

或者：

```
http://localhost:3000/user
```

九、推荐学习顺序

建议按这个顺序学：

1. Node.js 基础运行环境
2. CommonJS 模块系统
3. fs 文件操作
4. path 路径处理
5. http 创建服务器
6. npm 包管理
7. Express 框架
8. 路由
9. 中间件
10. 请求参数处理
11. 数据库 MySQL / MongoDB
12. 登录注册
13. JWT 鉴权
14. 文件上传
15. 项目部署

十、你现在可以练习一下

创建一个 `index.js` 文件：

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "text/html; charset=utf-8")

  if (req.url === "/") {
    res.end("<h1>首页</h1>")
  } else if (req.url === "/about") {
    res.end("<h1>关于我们</h1>")
  } else {
    res.end("<h1>404 页面不存在</h1>")
  }
})

server.listen(3000, () => {
  console.log("服务器运行在: http://localhost:3000")
})
```

运行：

```
node index.js
```

然后访问：

```
http://localhost:3000  
http://localhost:3000/about  
http://localhost:3000/abc
```

如果你愿意，我可以接着从 **Node.js 第一课：模块系统 require 和 module.exports** 开始一步一步教你。

从Node.js 第一课：模块系统 require 和 module.exports 开始。

Node.js 里，一个 `.js` 文件就可以看成一个“模块”。模块的作用是：把代码拆开，方便复用和管理。

1. 什么是模块？

比如你现在有一个项目：

```
node-demo  
├─ app.js  
└─ math.js
```

`math.js` 负责放数学方法，`app.js` 负责使用这些方法。

2. 导出模块：module.exports

先创建 `math.js`：

```
function add(a, b) {  
  return a + b  
}  
  
module.exports = add
```

这里的意思是：

```
module.exports = add
```

把 `add` 函数导出去，让别的文件可以使用。

3. 引入模块：require

再创建 `app.js`：

```
const add = require("./math")

const result = add(3, 5)

console.log(result)
```

运行：

```
node app.js
```

输出：

```
8
```

注意：

```
require("./math")
```

表示引入当前目录下的 `math.js` 文件。

`.js` 后缀可以省略。

4. 导出多个内容

如果一个文件里有多个方法，可以导出一个对象。

`math.js`：

```
function add(a, b) {
  return a + b
}

function sub(a, b) {
  return a - b
}

function mul(a, b) {
  return a * b
}

module.exports = {
  add,
  sub,
```



```
    mul  
  }  
}
```

app.js :

```
const math = require("./math")  
  
console.log(math.add(10, 3))  
console.log(math.sub(10, 3))  
console.log(math.mul(10, 3))
```

运行结果：

```
13  
7  
30
```

也可以这样写：

```
const { add, sub, mul } = require("./math")  
  
console.log(add(10, 3))  
console.log(sub(10, 3))  
console.log(mul(10, 3))
```

这叫 **对象解构**。

5. exports 的简写方式

Node.js 里还可以这样写：

```
exports.add = function (a, b) {  
  return a + b  
}  
  
exports.sub = function (a, b) {  
  return a - b  
}
```

使用时：

```
const math = require("./math")  
  
console.log(math.add(1, 2))  
console.log(math.sub(5, 3))
```

但是注意：初学者建议优先使用 `module.exports` ，更清晰。

6. module.exports 和 exports 的区别

一开始，Node.js 内部大概是这样：

```
const exports = module.exports
```

也就是说：

```
exports.add = ...
```

本质上是在给 `module.exports` 添加属性。

这个可以：

```
exports.add = function () {}
```

但是这个不推荐：

```
exports = {  
  add() {}  
}
```

因为这样只是让 `exports` 指向了一个新对象，并没有真正修改 `module.exports`。

正确写法是：

```
module.exports = {  
  add() {}  
}
```

记住一句话：

真正被导出的是 `module.exports`。

7. require 的路径规则

当前目录

```
require("./math")
```

表示当前目录下的 `math.js`。

上一级目录

```
require("../math")
```

表示上一级目录下的 `math.js`。

引入 Node.js 内置模块

```
const fs = require("fs")
```

比如 `fs` 是 Node.js 自带的文件模块。

引入第三方模块

```
const express = require("express")
```

这种一般需要先安装：

```
npm install express
```

8. require 会缓存模块

比如：

`count.js`：

```
console.log("count.js 被加载了")

let count = 0

count++

module.exports = count
```

`app.js`：

```
const a = require("./count")
const b = require("./count")
const c = require("./count")

console.log(a)
console.log(b)
console.log(c)
```

运行结果：

```
count.js 被加载了
1
```

```
1
1
```

说明 `count.js` 只执行了一次。

Node.js 会缓存已经加载过的模块。

9. 一个更真实的例子

项目结构：

```
node-demo
├─ app.js
└─ userService.js
```

`userService.js`：

```
const users = [
  { id: 1, name: "张三" },
  { id: 2, name: "李四" }
]

function getAllUsers() {
  return users
}

function getUserById(id) {
  return users.find(user => user.id === id)
}

module.exports = {
  getAllUsers,
  getUserById
}
```

`app.js`：

```
const { getAllUsers, getUserById } = require("./userService")

console.log(getAllUsers())

console.log(getUserById(2))
```

运行：

```
node app.js
```

输出：

```
[
  { id: 1, name: '张三' },
  { id: 2, name: '李四' }
]
{ id: 2, name: '李四' }
```

这就是实际项目里常见的写法：
一个文件负责提供功能，另一个文件负责调用功能。

本课重点

你需要记住这几个点：

1. 一个 `.js` 文件就是一个模块
 2. `module.exports` 用来导出内容
 3. `require()` 用来引入模块
 4. `require("./xxx")` 引入自己写的文件
 5. `require("fs")` 引入 Node.js 内置模块
 6. `require("express")` 引入第三方模块
 7. 真正导出的是 `module.exports`
-

练习题

创建三个文件：

```
node-demo
├─ app.js
├─ calculator.js
└─ user.js
```

`calculator.js` 要导出：

- `add(a, b)`
- `minus(a, b)`
- `multiply(a, b)`
- `divide(a, b)`

`user.js` 要导出一个对象：

```
{
  name: "张三",
  age: 20,
  city: "北京"
}
```

app.js 中引入这两个模块，并打印：

加法结果
减法结果
乘法结果
除法结果
用户信息

参考答案如下。

calculator.js：

```
function add(a, b) {  
  return a + b  
}  
  
function minus(a, b) {  
  return a - b  
}  
  
function multiply(a, b) {  
  return a * b  
}  
  
function divide(a, b) {  
  return a / b  
}  
  
module.exports = {  
  add,  
  minus,  
  multiply,  
  divide  
}
```

user.js：

```
module.exports = {  
  name: "张三",  
  age: 20,  
  city: "北京"  
}
```

app.js：

```
const calculator = require("./calculator")  
const user = require("./user")  
  
console.log("加法结果: ", calculator.add(10, 2))  
console.log("减法结果: ", calculator.minus(10, 2))
```

```
console.log("乘法结果: ", calculator.multiply(10, 2))
console.log("除法结果: ", calculator.divide(10, 2))
console.log("用户信息: ", user)
```

运行：

```
node app.js
```

输出：

```
加法结果:  12
减法结果:  8
乘法结果:  20
除法结果:  5
用户信息:  { name: '张三', age: 20, city: '北京' }
```

下一课可以学：**Node.js 第二课：fs 文件系统模块**。

Node.js 第二课：fs 文件系统模块

fs 是 Node.js 内置的文件系统模块，全称是：

```
file system
```

它可以用来操作文件和文件夹，比如：

- 读取文件
- 写入文件
- 追加内容
- 删除文件
- 创建文件夹
- 判断文件是否存在
- 获取文件信息
- 重命名文件

一、引入 fs 模块

Node.js 内置模块不需要安装，直接引入即可：

```
const fs = require("fs")
```

例如：

```
const fs = require("fs")

console.log(fs)
```

二、准备测试文件

新建一个项目目录：

```
node-fs-demo
├─ app.js
└─ hello.txt
```

在 `hello.txt` 中写入：

```
Hello Node.js
```

三、读取文件 `readFile`

1. 异步读取文件

`app.js`：

```
const fs = require("fs")

fs.readFile("./hello.txt", "utf-8", (err, data) => {
  if (err) {
    console.log("读取失败", err)
    return
  }

  console.log("文件内容: ")
  console.log(data)
})
```

运行：

```
node app.js
```

输出：

```
文件内容:
Hello Node.js
```

解释：

```
fs.readFile("./hello.txt", "utf-8", callback)
```

参数说明：

参数	说明
"./hello.txt"	文件路径
"utf-8"	编码格式
callback	读取完成后的回调函数

如果不写 `"utf-8"`，读取出来的是 Buffer：

```
fs.readFile("./hello.txt", (err, data) => {  
  console.log(data)  
})
```

可能输出：

```
<Buffer 48 65 6c 6c 6f 20 4e 6f 64 65 2e 6a 73>
```

四、同步读取文件 `readFileSync`

```
const fs = require("fs")  
  
const data = fs.readFileSync("./hello.txt", "utf-8")  
  
console.log(data)
```

同步读取更简单，但会阻塞程序。

五、异步和同步的区别

异步写法

```
const fs = require("fs")  
  
console.log("开始读取")  
  
fs.readFile("./hello.txt", "utf-8", (err, data) => {  
  console.log("读取完成")  
  console.log(data)  
})  
  
console.log("程序继续执行")
```

输出顺序通常是：

```
开始读取
程序继续执行
读取完成
Hello Node.js
```

同步写法

```
const fs = require("fs")

console.log("开始读取")

const data = fs.readFileSync("./hello.txt", "utf-8")

console.log("读取完成")
console.log(data)

console.log("程序继续执行")
```

输出：

```
开始读取
读取完成
Hello Node.js
程序继续执行
```

六、写入文件 writeFile

`writeFile` 会覆盖原来的内容。

```
const fs = require("fs")

fs.writeFile("./hello.txt", "新的内容", "utf-8", err => {
  if (err) {
    console.log("写入失败", err)
    return
  }

  console.log("写入成功")
})
```

运行后，`hello.txt` 的内容会变成：

```
新的内容
```

七、同步写入 writeFileSync

```
const fs = require("fs")

fs.writeFileSync("./hello.txt", "同步写入的内容", "utf-8")

console.log("写入完成")
```

八、追加内容 appendFile

如果不想覆盖原内容，而是继续在后面添加内容，可以使用 `appendFile`。

```
const fs = require("fs")

fs.appendFile("./hello.txt", "\n追加的一行内容", "utf-8", err => {
  if (err) {
    console.log("追加失败", err)
    return
  }

  console.log("追加成功")
})
```

`\n` 表示换行。

九、同步追加 appendFileSync

```
const fs = require("fs")

fs.appendFileSync("./hello.txt", "\n同步追加内容", "utf-8")

console.log("追加完成")
```

十、判断文件是否存在 existsSync

```
const fs = require("fs")

const exists = fs.existsSync("./hello.txt")

console.log(exists)
```

如果文件存在，输出：

```
true
```

如果不存在，输出：

```
false
```

常见用法：

```
const fs = require("fs")

if (fs.existsSync("./hello.txt")) {
  console.log("文件存在")
} else {
  console.log("文件不存在")
}
```

十一、获取文件信息 stat

```
const fs = require("fs")

fs.stat("./hello.txt", (err, stats) => {
  if (err) {
    console.log("获取失败", err)
    return
  }

  console.log(stats)
  console.log("是否是文件: ", stats.isFile())
  console.log("是否是文件夹: ", stats.isDirectory())
  console.log("文件大小: ", stats.size)
})
```

常用方法：

方法/属性	说明
stats.isFile()	是否是文件
stats.isDirectory()	是否是文件夹
stats.size	文件大小，单位是字节
stats.birthtime	创建时间
stats.mtime	修改时间

十二、创建文件夹 mkdir

```
const fs = require("fs")

fs.mkdir("./uploads", err => {
  if (err) {
    console.log("创建失败", err)
    return
  }

  console.log("文件夹创建成功")
})
```

如果文件夹已经存在，会报错。

可以先判断：

```
const fs = require("fs")

if (!fs.existsSync("./uploads")) {
  fs.mkdirSync("./uploads")
  console.log("创建成功")
} else {
  console.log("文件夹已经存在")
}
```

十三、递归创建多级文件夹

比如创建：

```
a/b/c
```

可以这样写：

```
const fs = require("fs")

fs.mkdir("./a/b/c", { recursive: true }, err => {
  if (err) {
    console.log("创建失败", err)
    return
  }

  console.log("多级文件夹创建成功")
})
```

同步写法：

```
const fs = require("fs")

fs.mkdirSync("./a/b/c", { recursive: true })
```

```
console.log("创建完成")
```

十四、读取文件夹 readdir

假设目录结构：

```
node-fs-demo
├─ app.js
├─ hello.txt
└─ uploads
```

读取当前目录：

```
const fs = require("fs")

fs.readdir("./", (err, files) => {
  if (err) {
    console.log("读取目录失败", err)
    return
  }

  console.log(files)
})
```

输出类似：

```
[ 'app.js', 'hello.txt', 'uploads' ]
```

十五、删除文件 unlink

```
const fs = require("fs")

fs.unlink("./hello.txt", err => {
  if (err) {
    console.log("删除失败", err)
    return
  }

  console.log("删除成功")
})
```

同步写法：

```
const fs = require("fs")

fs.unlinkSync("./hello.txt")

console.log("删除完成")
```

十六、删除文件夹 rmdir / rm

如果删除空文件夹：

```
const fs = require("fs")

fs.rmdir("./uploads", err => {
  if (err) {
    console.log("删除失败", err)
    return
  }

  console.log("文件夹删除成功")
})
```

如果要删除非空文件夹，推荐使用 `rm`：

```
const fs = require("fs")

fs.rm("./uploads", { recursive: true, force: true }, err => {
  if (err) {
    console.log("删除失败", err)
    return
  }

  console.log("删除成功")
})
```

同步写法：

```
const fs = require("fs")

fs.rmSync("./uploads", {
  recursive: true,
  force: true
})

console.log("删除完成")
```

参数说明：

```
{
  recursive: true,
```

```
    force: true
  }
```

参数	说明
recursive: true	递归删除子文件和子目录
force: true	如果路径不存在，不报错

十七、重命名文件 rename

```
const fs = require("fs")

fs.rename("./hello.txt", "./new-hello.txt", err => {
  if (err) {
    console.log("重命名失败", err)
    return
  }

  console.log("重命名成功")
})
```

rename 也可以用来移动文件。

比如：

```
const fs = require("fs")

fs.rename("./new-hello.txt", "./uploads/new-hello.txt", err => {
  if (err) {
    console.log("移动失败", err)
    return
  }

  console.log("移动成功")
})
```

十八、Promise 写法：fs.promises

现代 Node.js 项目中，更推荐使用 Promise 或 async/await。

```
const fs = require("fs/promises")

async function main() {
  try {
    const data = await fs.readFile("./hello.txt", "utf-8")
    console.log(data)
  } catch (err) {
```



```
    console.log("读取失败", err)
  }
}

main()
```

写入文件 Promise 版本

```
const fs = require("fs/promises")

async function main() {
  try {
    await fs.writeFile("./hello.txt", "Promise 写入内容", "utf-8")
    console.log("写入成功")
  } catch (err) {
    console.log("写入失败", err)
  }
}

main()
```

追加文件 Promise 版本

```
const fs = require("fs/promises")

async function main() {
  try {
    await fs.appendFile("./hello.txt", "\nPromise 追加内容", "utf-8")
    console.log("追加成功")
  } catch (err) {
    console.log("追加失败", err)
  }
}

main()
```

十九、路径问题

在 Node.js 中，很多初学者会遇到路径问题。

比如：

```
fs.readFile("./hello.txt", "utf-8", callback)
```

这里的 `./hello.txt` 不一定是当前 JS 文件所在目录，而是你执行 `node` 命令时所在的目录。

例如：

```
node app.js
```

如果你在项目根目录执行，没问题。

但如果你从别的目录执行，就可能找不到文件。

二十、使用 `__dirname` 解决路径问题

`__dirname` 表示当前 JS 文件所在目录。

```
const fs = require("fs")
const path = require("path")

const filePath = path.join(__dirname, "hello.txt")

fs.readFile(filePath, "utf-8", (err, data) => {
  if (err) {
    console.log("读取失败", err)
    return
  }

  console.log(data)
})
```

推荐实际项目中这样写：

```
const path = require("path")

const filePath = path.join(__dirname, "hello.txt")
```

二十一、综合案例：日志记录器

创建项目：

```
node-fs-demo
├─ app.js
└─ logs
```

如果没有 `logs` 文件夹，我们自动创建。

`app.js`：

```
const fs = require("fs")
const path = require("path")

const logDir = path.join(__dirname, "logs")
const logFile = path.join(logDir, "app.log")

if (!fs.existsSync(logDir)) {
  fs.mkdirSync(logDir)
}

function writeLog(message) {
  const time = new Date().toLocaleString()
  const content = `[${time}] ${message}\n`

  fs.appendFile(logFile, content, "utf-8", err => {
    if (err) {
      console.log("日志写入失败", err)
      return
    }

    console.log("日志写入成功")
  })
}

writeLog("用户登录")
writeLog("用户查看商品")
writeLog("用户退出")
```

运行：

```
node app.js
```

会生成：

```
logs/app.log
```

内容类似：

```
[2026/5/27 10:30:00] 用户登录
[2026/5/27 10:30:00] 用户查看商品
[2026/5/27 10:30:00] 用户退出
```

本课重点

这一课你需要掌握：

1. `fs` 是 Node.js 的文件系统模块
2. `readFile` 读取文件
3. `writeFile` 写入文件，会覆盖原内容

4. `appendFile` 追加内容
5. `existsSync` 判断路径是否存在
6. `stat` 获取文件或目录信息
7. `mkdir` 创建文件夹
8. `readdir` 读取目录
9. `unlink` 删除文件
10. `rm` 删除文件夹
11. `rename` 重命名或移动文件
12. 推荐使用 `path.join(__dirname, 文件名)` 处理路径
13. 现代项目中推荐使用 `fs/promises` + `async/await`

课后练习

创建如下项目：

```
fs-practice
├── app.js
└── data
```

要求：

1. 如果 `data` 文件夹不存在，就创建它
2. 在 `data/user.txt` 中写入：

```
姓名：张三
年龄：20
城市：北京
```

3. 再追加一行：

```
职业：前端工程师
```

4. 读取并打印 `user.txt` 的内容
5. 获取并打印文件大小
6. 把 `user.txt` 重命名为 `profile.txt`

参考答案

```
const fs = require("fs")
const path = require("path")

const dataDir = path.join(__dirname, "data")
const userFile = path.join(dataDir, "user.txt")
const profileFile = path.join(dataDir, "profile.txt")
```

```
if (!fs.existsSync(dataDir)) {
  fs.mkdirSync(dataDir)
}

fs.writeFileSync(userFile, "姓名: 张三\n年龄: 20\n城市: 北京", "utf-8")

fs.appendFileSync(userFile, "\n职业: 前端工程师", "utf-8")

const content = fs.readFileSync(userFile, "utf-8")
console.log("文件内容: ")
console.log(content)

const stats = fs.statSync(userFile)
console.log("文件大小: ", stats.size, "字节")

fs.renameSync(userFile, profileFile)
console.log("重命名完成")
```

运行:

```
node app.js
```

下一课可以学习:

Node.js 第三课: path 路径模块。

Node.js 第三课: path 路径模块

path 是 Node.js 内置的路径处理模块。

它主要用来处理文件路径, 比如:

- 拼接路径
- 获取文件名
- 获取目录名
- 获取扩展名
- 解析路径
- 规范化路径
- 判断是否是绝对路径

一、为什么需要 path 模块?

很多初学者会这样写路径:

```
const filePath = __dirname + "/data/user.txt"
```

这种写法在 Linux/macOS 上通常没问题，因为它们的路径分隔符是 `/`。

但是 Windows 的路径分隔符通常是 `\`：

```
C:\Users\Tom\project\data\user.txt
```

所以为了兼容不同系统，推荐使用 `path` 模块。

二、引入 path 模块

```
const path = require("path")
```

`path` 是 Node.js 内置模块，不需要安装。

三、dirname 和 filename

在学习 `path` 之前，先认识两个常用变量。

1. __dirname

表示当前 JS 文件所在的文件夹路径。

```
console.log(__dirname)
```

假设文件位置是：

```
D:\node-demo\app.js
```

输出可能是：

```
D:\node-demo
```

2. __filename

表示当前 JS 文件的完整路径。

```
console.log(__filename)
```

输出可能是：

```
D:\node-demo\app.js
```

四、path.join 拼接路径

最常用的是 `path.join()` 。

```
const path = require("path")

const filePath = path.join(__dirname, "data", "user.txt")

console.log(filePath)
```

输出可能是：

Windows：

```
D:\node-demo\data\user.txt
```

macOS/Linux：

```
/Users/tom/node-demo/data/user.txt
```

推荐这样写：

```
const filePath = path.join(__dirname, "data", "user.txt")
```

不要这样写：

```
const filePath = __dirname + "/data/user.txt"
```

五、path.resolve 解析绝对路径

`path.resolve()` 会返回一个绝对路径。

```
const path = require("path")

const result = path.resolve("data", "user.txt")

console.log(result)
```

如果当前运行目录是：

```
D:\node-demo
```

输出：

```
D:\node-demo\data\user.txt
```

也可以这样：

```
const filePath = path.resolve(__dirname, "data", "user.txt")
console.log(filePath)
```

join 和 resolve 的简单区别

path.join

主要用于拼接路径：

```
path.join("a", "b", "c")
```

结果：

```
a/b/c
```

path.resolve

主要用于解析成绝对路径：

```
path.resolve("a", "b", "c")
```

结果类似：

```
当前工作目录/a/b/c
```

遇到 / 时区别明显

```
const path = require("path")

console.log(path.join("/a", "/b", "c"))
console.log(path.resolve("/a", "/b", "c"))
```


结果：

```
/a/b/c  
/b/c
```

原因：

- `join` 只是拼接并规范化
- `resolve` 遇到绝对路径，会从最后一个绝对路径开始解析

六、`path.basename` 获取文件名

```
const path = require("path")  
  
const filePath = "/users/tom/project/app.js"  
  
console.log(path.basename(filePath))
```

输出：

```
app.js
```

如果不想要扩展名：

```
console.log(path.basename(filePath, ".js"))
```

输出：

```
app
```

七、`path.dirname` 获取目录名

```
const path = require("path")  
  
const filePath = "/users/tom/project/app.js"  
  
console.log(path.dirname(filePath))
```

输出：

```
/users/tom/project
```

八、path.extname 获取扩展名

```
const path = require("path")

console.log(path.extname("index.html"))
console.log(path.extname("photo.png"))
console.log(path.extname("app.js"))
```

输出：

```
.html
.png
.js
```

注意：

```
console.log(path.extname("README"))
```

输出：

没有扩展名时，结果是空字符串。

九、path.parse 解析路径

```
const path = require("path")

const filePath = "/users/tom/project/app.js"

const result = path.parse(filePath)

console.log(result)
```

输出：

```
{
  root: '/',
  dir: '/users/tom/project',
  base: 'app.js',
  ext: '.js',
  name: 'app'
}
```

字段说明：

字段	说明
root	根路径
dir	目录
base	文件名 + 扩展名
ext	扩展名
name	文件名，不含扩展名

十、path.format 生成路径

`path.format()` 和 `path.parse()` 相反。

```
const path = require("path")

const filePath = path.format({
  dir: "/users/tom/project",
  name: "app",
  ext: ".js"
})

console.log(filePath)
```

输出：

```
/users/tom/project/app.js
```

十一、path.normalize 规范化路径

```
const path = require("path")

const result = path.normalize("/users/tom//project/../app.js")

console.log(result)
```

输出：

```
/users/tom/app.js
```

它会处理：

- 多余的 `/`
- `..`

十二、path.isAbsolute 判断是否是绝对路径

```
const path = require("path")

console.log(path.isAbsolute("/users/tom"))
console.log(path.isAbsolute("users/tom"))
```

macOS/Linux 输出：

```
true
false
```

Windows 示例：

```
console.log(path.isAbsolute("C:\\Users\\Tom"))
console.log(path.isAbsolute("data\\user.txt"))
```

输出：

```
true
false
```

十三、path.relative 获取相对路径

```
const path = require("path")

const from = "/users/tom/project/src"
const to = "/users/tom/project/public/img/logo.png"

const result = path.relative(from, to)

console.log(result)
```

输出：

```
../public/img/logo.png
```

意思是：从 src 目录到 logo.png 的相对路径。

十四、path.sep 路径分隔符

```
const path = require("path")

console.log(path.sep)
```

Windows 输出：

```
\
```

macOS/Linux 输出：

```
/
```

十五、path.delimiter 环境变量分隔符

```
const path = require("path")

console.log(path.delimiter)
```

Windows 输出：

```
;
```

macOS/Linux 输出：

```
:
```

例如环境变量 PATH：

Windows：

```
C:\Windows;C:\Program Files\nodejs
```

macOS/Linux：

```
/usr/bin:/bin:/usr/local/bin
```

十六、Windows 和 POSIX 路径

Node.js 的 `path` 会根据当前系统自动处理路径。

如果你想明确处理 Windows 路径或 POSIX 路径，可以用：

```
path.win32
path.posix
```

示例：

```
const path = require("path")

console.log(path.win32.basename("C:\\Users\\Tom\\app.js"))
console.log(path.posix.basename("/users/tom/app.js"))
```

输出：

```
app.js
app.js
```

十七、配合 fs 使用

这是实际项目中最常见的用法。

项目结构：

```
path-demo
├─ app.js
├─ data
│   └─ user.txt
```

app.js：

```
const fs = require("fs")
const path = require("path")

const filePath = path.join(__dirname, "data", "user.txt")

const content = fs.readFileSync(filePath, "utf-8")

console.log(content)
```

这样不管你在 Windows、macOS 还是 Linux 上运行，都更稳定。

十八、实际案例：扫描目录中的图片文件

项目结构：

```
path-demo
├─ app.js
└─ uploads
    ├─ a.jpg
    ├─ b.png
    ├─ c.gif
    └─ readme.txt
```

app.js :

```
const fs = require("fs")
const path = require("path")

const uploadDir = path.join(__dirname, "uploads")

const files = fs.readdirSync(uploadDir)

const imageFiles = files.filter(file => {
  const ext = path.extname(file).toLowerCase()
  return [".jpg", ".jpeg", ".png", ".gif", ".webp"].includes(ext)
})

console.log(imageFiles)
```

输出：

```
[ 'a.jpg', 'b.png', 'c.gif' ]
```

十九、实际案例：批量修改文件扩展名

项目结构：

```
path-demo
├─ app.js
└─ files
    ├─ a.txt
    ├─ b.txt
    └─ c.txt
```

把 .txt 改成 .md 。

app.js :

```
const fs = require("fs")
const path = require("path")

const dir = path.join(__dirname, "files")

const files = fs.readdirSync(dir)
```

```
files.forEach(file => {
  const oldPath = path.join(dir, file)

  const info = path.parse(file)

  if (info.ext === ".txt") {
    const newFileName = info.name + ".md"
    const newPath = path.join(dir, newFileName)

    fs.renameSync(oldPath, newPath)

    console.log(`${file} -> ${newFileName}`)
  }
})
```

运行：

```
node app.js
```

结果：

```
a.txt -> a.md
b.txt -> b.md
c.txt -> c.md
```

二十、常见错误

错误 1：手动拼接路径

不推荐：

```
const filePath = __dirname + "/data/user.txt"
```

推荐：

```
const filePath = path.join(__dirname, "data", "user.txt")
```

错误 2：以为 `./` 一定是当前文件目录

```
fs.readFileSync("./data/user.txt", "utf-8")
```

这里的 `./` 是你执行 `node` 命令时所在的目录，不一定是当前 JS 文件所在目录。

推荐：


```
const filePath = path.join(__dirname, "data", "user.txt")
fs.readFileSync(filePath, "utf-8")
```

错误 3: Windows 路径中的反斜杠没有转义

错误:

```
const filePath = "C:\Users\Tom\Desktop\a.txt"
```

正确:

```
const filePath = "C:\\Users\\Tom\\Desktop\\a.txt"
```

或者:

```
const filePath = path.join("C:", "Users", "Tom", "Desktop", "a.txt")
```

本课重点

这一课你需要掌握:

1. `path` 是 Node.js 内置路径模块
2. `path.join()` 最常用, 用来拼接路径
3. `path.resolve()` 用来生成绝对路径
4. `path.basename()` 获取文件名
5. `path.dirname()` 获取目录名
6. `path.extname()` 获取扩展名
7. `path.parse()` 把路径解析成对象
8. `path.format()` 把对象转换成路径
9. `path.normalize()` 规范化路径
10. `__dirname` 是当前 JS 文件所在目录
11. `__filename` 是当前 JS 文件完整路径
12. 实际开发中经常使用:

```
path.join(__dirname, "data", "user.txt")
```

课后练习

创建项目:

```
path-practice
├─ app.js
├─ public
│   └─ images
│       ├── logo.png
│       ├── banner.jpg
│       ├── avatar.webp
│       └─ readme.txt
```

要求：

1. 使用 `path.join()` 拼接 `images` 目录路径
2. 使用 `fs.readdirSync()` 读取文件列表
3. 筛选出图片文件
4. 打印每个图片文件的：
 - 文件名
 - 扩展名
 - 完整路径

参考答案

```
const fs = require("fs")
const path = require("path")

const imageDir = path.join(__dirname, "public", "images")

const files = fs.readdirSync(imageDir)

const imageExts = [".jpg", ".jpeg", ".png", ".gif", ".webp"]

files.forEach(file => {
  const ext = path.extname(file).toLowerCase()

  if (imageExts.includes(ext)) {
    const fullPath = path.join(imageDir, file)

    console.log("文件名: ", file)
    console.log("扩展名: ", ext)
    console.log("完整路径: ", fullPath)
    console.log("-----")
  }
})
```

下一课可以学习：

Node.js 第四课：http 创建服务器模块。

Node.js 第四课：http 创建服务器模块

http 是 Node.js 内置模块，用来创建 HTTP 服务器。

它可以让我们用 Node.js 搭建一个最基础的 Web 服务。

一、什么是 HTTP?

HTTP 是浏览器和服务器之间通信的协议。

比如你在浏览器访问：

```
http://localhost:3000
```

大致过程是：

```
浏览器发送请求 -> Node.js 服务器接收请求 -> 服务器返回响应 -> 浏览器显示内容
```

二、引入 http 模块

http 是 Node.js 内置模块，不需要安装：

```
const http = require("http")
```

三、创建第一个服务器

新建项目：

```
http-demo
└─ app.js
```

app.js：

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.end("Hello Node.js HTTP Server")
})

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

运行：

```
node app.js
```

打开浏览器访问：

```
http://localhost:3000
```

页面显示：

```
Hello Node.js HTTP Server
```

四、createServer 的两个核心对象

```
const server = http.createServer((req, res) => {  
  // req: request, 请求对象  
  // res: response, 响应对象  
})
```

1. req 请求对象

req 代表客户端发送来的请求。

常用属性：

属性	说明
req.url	请求地址
req.method	请求方法
req.headers	请求头

示例：

```
const http = require("http")  
  
const server = http.createServer((req, res) => {  
  console.log("请求地址: ", req.url)  
  console.log("请求方法: ", req.method)  
  console.log("请求头: ", req.headers)  
  
  res.end("OK")  
})  
  
server.listen(3000, () => {  
  console.log("服务器已启动: http://localhost:3000")  
})
```

访问：

```
http://localhost:3000/user
```

终端可能输出：

```
请求地址: /user
请求方法: GET
请求头: {
  host: 'localhost:3000',
  connection: 'keep-alive',
  ...
}
```

2. res 响应对象

`res` 用来给客户端返回数据。

常用方法：

方法	说明
<code>res.end()</code>	结束响应并返回内容
<code>res.write()</code>	分段写入响应
<code>res.setHeader()</code>	设置响应头
<code>res.statusCode</code>	设置状态码

示例：

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.statusCode = 200
  res.setHeader("Content-Type", "text/plain; charset=utf-8")
  res.end("你好, Node.js")
})

server.listen(3000)
```

五、解决中文乱码

如果直接返回中文：

```
res.end("你好")
```

浏览器可能乱码。

需要设置响应头：

```
res.setHeader("Content-Type", "text/plain; charset=utf-8")
res.end("你好")
```

完整代码：

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "text/plain; charset=utf-8")
  res.end("你好，这是 Node.js 服务器")
})

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

六、根据不同路径返回不同内容

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "text/html; charset=utf-8")

  if (req.url === "/") {
    res.end("<h1>首页</h1>")
  } else if (req.url === "/about") {
    res.end("<h1>关于我们</h1>")
  } else if (req.url === "/contact") {
    res.end("<h1>联系我们</h1>")
  } else {
    res.statusCode = 404
    res.end("<h1>404 页面不存在</h1>")
  }
})

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

访问：

```
http://localhost:3000
http://localhost:3000/about
http://localhost:3000/contact
http://localhost:3000/abc
```

七、返回 JSON 数据

后端接口常常返回 JSON。

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json; charset=utf-8")

  const user = {
    id: 1,
    name: "张三",
    age: 20
  }

  res.end(JSON.stringify(user))
})

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

访问：

```
http://localhost:3000
```

返回：

```
{
  "id": 1,
  "name": "张三",
  "age": 20
}
```

注意：

```
res.end()
```

只能返回字符串或 Buffer，所以对象需要用：

```
JSON.stringify(obj)
```

八、处理 URL 查询参数

例如访问：

```
http://localhost:3000/user?id=100&name=zhangsan
```

可以用 URL 解析。

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "application/json; charset=utf-8")

  const urlObj = new URL(req.url, "http://localhost:3000")

  const pathname = urlObj.pathname
  const id = urlObj.searchParams.get("id")
  const name = urlObj.searchParams.get("name")

  res.end(JSON.stringify({
    pathname,
    id,
    name
  })))
})

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

访问：

```
http://localhost:3000/user?id=100&name=zhangsan
```

返回：

```
{
  "pathname": "/user",
  "id": "100",
  "name": "zhangsan"
}
```

九、根据请求方法处理 GET / POST

```
const http = require("http")

const server = http.createServer((req, res) => {
  res.setHeader("Content-Type", "text/plain; charset=utf-8")

  if (req.method === "GET") {
    res.end("这是 GET 请求")
  } else if (req.method === "POST") {
    res.end("这是 POST 请求")
  }
})
```



```
    } else {
      res.end("其他请求")
    }
  })

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

十、接收 POST 请求体数据

POST 请求的数据不是一次性直接在 `req.body` 里。

在原生 `http` 模块中，需要通过事件接收：

```
const http = require("http")

const server = http.createServer((req, res) => {
  if (req.method === "POST" && req.url === "/login") {
    let body = ""

    req.on("data", chunk => {
      body += chunk
    })

    req.on("end", () => {
      res.setHeader("Content-Type", "application/json; charset=utf-8")

      console.log("接收到的数据: ", body)

      res.end(JSON.stringify({
        message: "登录成功",
        data: body
      }))
    })
  } else {
    res.setHeader("Content-Type", "text/plain; charset=utf-8")
    res.end("Hello")
  }
})

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

可以用 Postman、Apifox 或 curl 测试：

```
curl -X POST http://localhost:3000/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"123456"}'
```

返回：

```
{
  "message": "登录成功",
  "data": "{\"username\":\"admin\",\"password\":\"123456\"}"
}
```

如果想把字符串转成对象：

```
const data = JSON.parse(body)
```

完整写法：

```
req.on("end", () => {
  res.setHeader("Content-Type", "application/json; charset=utf-8")

  const data = JSON.parse(body)

  res.end(JSON.stringify({
    message: "登录成功",
    username: data.username
  })))
})
```

十一、返回 HTML 页面

项目结构：

```
http-demo
├─ app.js
└─ index.html
```

index.html：

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>Node HTTP</title>
</head>
<body>
  <h1>这是 HTML 页面</h1>
  <p>由 Node.js 返回</p>
</body>
</html>
```

app.js：

```
const http = require("http")
const fs = require("fs")
const path = require("path")

const server = http.createServer((req, res) => {
  const filePath = path.join(__dirname, "index.html")

  fs.readFile(filePath, "utf-8", (err, data) => {
    if (err) {
      res.statusCode = 500
      res.setHeader("Content-Type", "text/plain; charset=utf-8")
      res.end("服务器错误")
      return
    }

    res.setHeader("Content-Type", "text/html; charset=utf-8")
    res.end(data)
  })
})

server.listen(3000, () => {
  console.log("服务器已启动: http://localhost:3000")
})
```

十二、简易静态资源服务器

项目结构：

```
http-demo
├─ app.js
└─ public
   ├─ index.html
   ├─ style.css
   └─ app.js
```

public/index.html：

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>静态资源服务器</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>首页</h1>
  <script src="/app.js"></script>
</body>
</html>
```

public/style.css :

```
body {  
  background: #f5f5f5;  
  color: #333;  
}
```

public/app.js :

```
console.log("前端 JS 加载成功")
```

服务器 app.js :

```
const http = require("http")  
const fs = require("fs")  
const path = require("path")  
  
const publicDir = path.join(__dirname, "public")  
  
const mimeTypes = {  
  ".html": "text/html; charset=utf-8",  
  ".css": "text/css; charset=utf-8",  
  ".js": "application/javascript; charset=utf-8",  
  ".json": "application/json; charset=utf-8",  
  ".png": "image/png",  
  ".jpg": "image/jpeg",  
  ".jpeg": "image/jpeg",  
  ".gif": "image/gif",  
  ".webp": "image/webp"  
}  
  
const server = http.createServer((req, res) => {  
  let urlPath = req.url  
  
  if (urlPath === "/") {  
    urlPath = "/index.html"  
  }  
  
  const filePath = path.join(publicDir, urlPath)  
  
  const ext = path.extname(filePath)  
  
  const contentType = mimeTypes[ext] || "application/octet-stream"  
  
  fs.readFile(filePath, (err, data) => {  
    if (err) {  
      res.statusCode = 404  
      res.setHeader("Content-Type", "text/plain; charset=utf-8")  
      res.end("404 文件不存在")  
      return  
    }  
  
    res.setHeader("Content-Type", contentType)  
    res.end(data)  
  })  
})
```

```
    })  
  })  
  
  server.listen(3000, () => {  
    console.log("静态服务器启动: http://localhost:3000")  
  })  
}
```

访问：

```
http://localhost:3000
```

十三、注意静态服务器的路径安全

上面的简易版本还不够安全。

如果用户访问：

```
http://localhost:3000/../../app.js
```

可能造成目录穿越风险。

更安全的写法应该检查路径是否在 `public` 目录下：

```
const requestedPath = path.normalize(urlPath)  
const filePath = path.join(publicDir, requestedPath)  
  
if (!filePath.startsWith(publicDir)) {  
  res.statusCode = 403  
  res.end("Forbidden")  
  return  
}
```

完整片段：

```
let urlPath = decodeURIComponent(req.url.split("?")[0])  
  
if (urlPath === "/") {  
  urlPath = "/index.html"  
}  
  
const filePath = path.join(publicDir, path.normalize(urlPath))  
  
if (!filePath.startsWith(publicDir)) {  
  res.statusCode = 403  
  res.setHeader("Content-Type", "text/plain; charset=utf-8")  
  res.end("禁止访问")  
  return  
}
```

十四、封装响应函数

写接口时，经常封装一个统一返回 JSON 的方法：

```
function sendJSON(res, data, statusCode = 200) {  
  res.statusCode = statusCode  
  res.setHeader("Content-Type", "application/json; charset=utf-8")  
  res.end(JSON.stringify(data))  
}
```

使用：

```
const http = require("http")  
  
function sendJSON(res, data, statusCode = 200) {  
  res.statusCode = statusCode  
  res.setHeader("Content-Type", "application/json; charset=utf-8")  
  res.end(JSON.stringify(data))  
}  
  
const server = http.createServer((req, res) => {  
  if (req.url === "/api/user") {  
    sendJSON(res, {  
      id: 1,  
      name: "张三"  
    })  
  } else {  
    sendJSON(res, {  
      message: "接口不存在"  
    }, 404)  
  }  
})  
  
server.listen(3000, () => {  
  console.log("服务器已启动: http://localhost:3000")  
})
```

十五、综合案例：简单用户接口

实现这些接口：

请求方法	路径	功能
GET	/api/users	获取用户列表
GET	/api/user?id=1	根据 id 获取用户
POST	/api/login	登录

代码:

```
const http = require("http")

const users = [
  { id: 1, name: "张三", age: 20 },
  { id: 2, name: "李四", age: 22 },
  { id: 3, name: "王五", age: 25 }
]

function sendJSON(res, data, statusCode = 200) {
  res.statusCode = statusCode
  res.setHeader("Content-Type", "application/json; charset=utf-8")
  res.end(JSON.stringify(data))
}

function getBody(req) {
  return new Promise((resolve, reject) => {
    let body = ""

    req.on("data", chunk => {
      body += chunk
    })

    req.on("end", () => {
      try {
        resolve(body ? JSON.parse(body) : {})
      } catch (err) {
        reject(err)
      }
    })

    req.on("error", err => {
      reject(err)
    })
  })
}

const server = http.createServer(async (req, res) => {
  const urlObj = new URL(req.url, "http://localhost:3000")
  const pathname = urlObj.pathname

  if (req.method === "GET" && pathname === "/api/users") {
    sendJSON(res, {
      code: 0,
      data: users
    })
    return
  }

  if (req.method === "GET" && pathname === "/api/user") {
    const id = Number(urlObj.searchParams.get("id"))
    const user = users.find(item => item.id === id)

    if (!user) {
      sendJSON(res, {
```

```

        code: 404,
        message: "用户不存在"
    }, 404)
    return
}

sendJSON(res, {
    code: 0,
    data: user
})
return
}

if (req.method === "POST" && pathname === "/api/login") {
    try {
        const body = await getBody(req)

        if (body.username === "admin" && body.password === "123456") {
            sendJSON(res, {
                code: 0,
                message: "登录成功",
                token: "mock-token"
            })
        } else {
            sendJSON(res, {
                code: 401,
                message: "用户名或密码错误"
            }, 401)
        }
    } catch (err) {
        sendJSON(res, {
            code: 400,
            message: "请求体格式错误"
        }, 400)
    }

    return
}

sendJSON(res, {
    code: 404,
    message: "接口不存在"
}, 404)
})

server.listen(3000, () => {
    console.log("用户接口服务启动: http://localhost:3000")
})

```

测试:

```
curl http://localhost:3000/api/users
```

```
curl "http://localhost:3000/api/user?id=1"
```



```
curl -X POST http://localhost:3000/api/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"123456"}'
```

十六、http 模块的特点

原生 http 模块：

优点：

- Node.js 内置，不需要安装
- 可以理解服务器底层原理
- 灵活

缺点：

- 路由写起来麻烦
- 处理请求体麻烦
- 静态资源处理麻烦
- 中间件机制需要自己实现

所以实际开发中通常会使用框架：

```
Express  
Koa  
Fastify  
NestJS
```

比如 Express：

```
app.get("/api/users", (req, res) => {  
  res.json(users)  
})
```

比原生 http 简洁很多。

本课重点

这一课你需要掌握：

1. http 是 Node.js 内置模块
2. http.createServer() 可以创建服务器
3. req 是请求对象
4. res 是响应对象
5. req.url 获取请求路径

6. `req.method` 获取请求方法
7. `res.setHeader()` 设置响应头
8. `res.statusCode` 设置状态码
9. `res.end()` 返回内容并结束响应
10. GET 参数可以用 `new URL(req.url, base)` 解析
11. POST 请求体需要通过 `data` 和 `end` 事件接收
12. 返回对象时要用 `JSON.stringify()`
13. 中文响应要设置 `charset=utf-8`

课后练习

创建一个原生 http 服务，实现以下功能：

请求方法	路径	返回
GET	/	首页
GET	/about	关于我们
GET	/api/products	商品列表 JSON
GET	/api/product?id=1	单个商品 JSON
POST	/api/order	接收订单数据并返回“下单成功”
其他	任意	404

商品数据：

```
const products = [
  { id: 1, name: "苹果", price: 5 },
  { id: 2, name: "香蕉", price: 3 },
  { id: 3, name: "橙子", price: 4 }
]
```

参考答案

```
const http = require("http")

const products = [
  { id: 1, name: "苹果", price: 5 },
  { id: 2, name: "香蕉", price: 3 },
  { id: 3, name: "橙子", price: 4 }
]

function sendHTML(res, html, statusCode = 200) {
  res.statusCode = statusCode
  res.setHeader("Content-Type", "text/html; charset=utf-8")
  res.end(html)
```

```

}

function sendJSON(res, data, statusCode = 200) {
  res.statusCode = statusCode
  res.setHeader("Content-Type", "application/json; charset=utf-8")
  res.end(JSON.stringify(data))
}

function getBody(req) {
  return new Promise((resolve, reject) => {
    let body = ""

    req.on("data", chunk => {
      body += chunk
    })

    req.on("end", () => {
      try {
        resolve(body ? JSON.parse(body) : {})
      } catch (err) {
        reject(err)
      }
    })

    req.on("error", reject)
  })
}

const server = http.createServer(async (req, res) => {
  const urlObj = new URL(req.url, "http://localhost:3000")
  const pathname = urlObj.pathname

  if (req.method === "GET" && pathname === "/") {
    sendHTML(res, "<h1>首页</h1>")
    return
  }

  if (req.method === "GET" && pathname === "/about") {
    sendHTML(res, "<h1>关于我们</h1>")
    return
  }

  if (req.method === "GET" && pathname === "/api/products") {
    sendJSON(res, {
      code: 0,
      data: products
    })
    return
  }

  if (req.method === "GET" && pathname === "/api/product") {
    const id = Number(urlObj.searchParams.get("id"))
    const product = products.find(item => item.id === id)

    if (!product) {
      sendJSON(res, {
        code: 404,

```

```
        message: "商品不存在"
      }, 404)
      return
    }

    sendJSON(res, {
      code: 0,
      data: product
    })
    return
  }

  if (req.method === "POST" && pathname === "/api/order") {
    try {
      const order = await getBody(req)

      sendJSON(res, {
        code: 0,
        message: "下单成功",
        data: order
      })
    } catch (err) {
      sendJSON(res, {
        code: 400,
        message: "请求体格式错误"
      }, 400)
    }

    return
  }

  sendHTML(res, "<h1>404 页面不存在</h1>", 404)
})

server.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

下一课建议学习：

Node.js 第五课：npm 包管理工具。

Node.js 第五课：npm 包管理工具

npm 是 Node.js 自带的包管理工具，全称是：

```
Node Package Manager
```

它主要用来：

- 初始化 Node.js 项目
- 安装第三方包

- 管理项目依赖
- 运行脚本命令
- 发布自己的包

一、什么是包？

在 Node.js 中，别人写好的功能模块可以称为一个“包”。

比如：

包名	作用
express	Web 框架
axios	发送 HTTP 请求
dayjs	日期处理
lodash	工具函数库
mysql2	连接 MySQL
mongoose	操作 MongoDB
jsonwebtoken	生成和验证 JWT
multer	文件上传

以前你要自己写很多功能，现在可以直接安装别人写好的包。

二、查看 npm 是否安装成功

安装 Node.js 时，npm 通常会一起安装。

在终端输入：

```
node -v
```

查看 Node.js 版本。

再输入：

```
npm -v
```

查看 npm 版本。

如果都能显示版本号，说明安装成功。

三、初始化项目 npm init

创建一个项目目录：

```
mkdir npm-demo  
cd npm-demo
```

初始化项目：

```
npm init
```

它会一步一步询问：

```
package name:  
version:  
description:  
entry point:  
test command:  
git repository:  
keywords:  
author:  
license:
```

如果你想全部使用默认值，可以使用：

```
npm init -y
```

执行后会生成一个文件：

```
package.json
```

四、package.json 是什么？

`package.json` 是 Node.js 项目的配置文件。

示例：

```
{  
  "name": "npm-demo",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",
```

```
"license": "ISC"
}
```

常见字段说明：

字段	说明
name	项目名称
version	项目版本
description	项目描述
main	入口文件
scripts	脚本命令
dependencies	生产依赖
devDependencies	开发依赖
license	开源协议

五、安装第三方包

比如安装 `dayjs`：

```
npm install dayjs
```

也可以简写：

```
npm i dayjs
```

安装完成后，会出现：

```
npm-demo
├─ node_modules
├─ package-lock.json
└─ package.json
```

同时 `package.json` 中会多出：

```
"dependencies": {
  "dayjs": "^1.11.13"
}
```

六、使用已安装的包

新建 `app.js`：

```
const dayjs = require("dayjs")

const now = dayjs().format("YYYY-MM-DD HH:mm:ss")

console.log(now)
```

运行：

```
node app.js
```

输出类似：

```
2026-05-28 10:30:00
```

七、node_modules 是什么？

`node_modules` 是 npm 下载第三方包后存放的位置。

例如安装 `dayjs` 后：

```
node_modules
└─ dayjs
```

项目中一般不要手动修改 `node_modules`。

也不要把 `node_modules` 上传到 Git 仓库。

通常会在 `.gitignore` 里写：

```
node_modules
```

八、package-lock.json 是什么？

`package-lock.json` 用来锁定依赖的具体版本。

比如 `package.json` 里可能是：

```
"dayjs": "^1.11.13"
```

而 `package-lock.json` 会记录实际安装的精确版本：


```
"version": "1.11.13"
```

它的作用是：

- 保证团队成员安装到一致的依赖版本
- 保证线上部署环境依赖稳定
- 提高安装速度

一般建议把 `package-lock.json` 提交到 Git。

九、依赖类型 `dependencies` 和 `devDependencies`

1. `dependencies`

生产环境也需要的包。

安装命令：

```
npm install express
```

或：

```
npm install express --save
```

现在默认就是保存到 `dependencies`。

示例：

```
"dependencies": {  
  "express": "^4.18.2"  
}
```

常见生产依赖：

- `express`
- `mysql2`
- `mongoose`
- `jsonwebtoken`
- `bcryptjs`
- `cors`

2. `devDependencies`

只在开发阶段需要，线上运行不一定需要。

安装命令：

```
npm install nodemon -D
```

或：

```
npm install nodemon --save-dev
```

示例：

```
"devDependencies": {  
  "nodemon": "^3.1.0"  
}
```

常见开发依赖：

- nodemon
- eslint
- prettier
- jest
- typescript
- vite

十、卸载包

卸载 dayjs：

```
npm uninstall dayjs
```

简写：

```
npm un dayjs
```

卸载后：

- node_modules 中删除该包
- package.json 中删除依赖记录
- package-lock.json 中更新记录

十一、安装指定版本

安装指定版本：

```
npm install express@4.18.2
```

安装最新版本：

```
npm install express@latest
```

查看某个包的所有版本：

```
npm view express versions
```

查看包信息：

```
npm view express
```

十二、版本号规则

在 `package.json` 中经常看到：

```
"express": "^4.18.2"
```

版本号一般是：

```
主版本号.次版本号.修订号  
major.minor.patch
```

例如：

```
4.18.2
```

含义：

位置	含义
4	主版本号，可能有重大变化
18	次版本号，新增功能
2	修订号，修复 bug

常见符号

1. ^

```
"express": "^4.18.2"
```

表示允许升级次版本和修订版本，但不升级主版本。

可能安装：

```
4.19.0  
4.20.1
```

不会安装：

```
5.0.0
```

2. ~

```
"express": "~4.18.2"
```

表示只允许升级修订版本。

可能安装：

```
4.18.3  
4.18.4
```

不会安装：

```
4.19.0
```

3. 不加符号

```
"express": "4.18.2"
```

表示固定安装这个版本。

十三、npm scripts 脚本命令

在 `package.json` 中可以配置脚本：

```
{  
  "scripts": {
```

```
"start": "node app.js",  
"dev": "nodemon app.js",  
"hello": "echo hello"  
}  
}
```

运行：

```
npm run hello
```

对于 `start`，可以简写：

```
npm start
```

等价于：

```
npm run start
```

十四、使用 nodemon 自动重启

开发 Node.js 时，每次修改代码都要重新运行：

```
node app.js
```

很麻烦。

可以安装 `nodemon`：

```
npm install nodemon -D
```

然后在 `package.json` 中配置：

```
{  
  "scripts": {  
    "dev": "nodemon app.js"  
  }  
}
```

新建 `app.js`：

```
console.log("服务启动")  
console.log(new Date().toLocaleString())
```

运行：

```
npm run dev
```

以后修改 `app.js`，`nodemon` 会自动重启程序。

十五、全局安装和本地安装

1. 本地安装

```
npm install nodemon -D
```

安装到当前项目的 `node_modules`。

推荐项目依赖都使用本地安装。

2. 全局安装

```
npm install nodemon -g
```

安装到电脑全局环境。

安装后可以在任何目录使用：

```
nodemon app.js
```

查看全局安装位置：

```
npm root -g
```

查看全局安装包：

```
npm list -g --depth=0
```

十六、npx 是什么？

`npx` 可以直接运行项目中安装的命令。

比如你本地安装了 `nodemon`：

```
npm install nodemon -D
```

可以执行：

```
npx nodemon app.js
```

不需要全局安装。

十七、npm install 的常见用法

安装 package.json 中所有依赖

如果你从 GitHub 下载了一个项目，通常没有 `node_modules`。

只需要执行：

```
npm install
```

或者：

```
npm i
```

npm 会根据 `package.json` 和 `package-lock.json` 安装依赖。

只安装生产依赖

```
npm install --production
```

或新版本常用：

```
npm install --omit=dev
```

这会跳过 `devDependencies`。

十八、查看已安装依赖

查看当前项目依赖：

```
npm list
```

只查看一级依赖：

```
npm list --depth=0
```

查看全局包：

```
npm list -g --depth=0
```

十九、更新依赖

查看哪些包可以更新：

```
npm outdated
```

更新包：

```
npm update
```

更新指定包：

```
npm update express
```

注意：npm update 通常不会跨越 package.json 允许的版本范围。

二十、npm 镜像源

如果你在国内访问 npm 官方源比较慢，可以切换镜像源。

查看当前源：

```
npm config get registry
```

设置淘宝 npmmirror 源：

```
npm config set registry https://registry.npmmirror.com
```

恢复官方源：

```
npm config set registry https://registry.npmjs.org
```

二十一、清理 npm 缓存

有时候安装依赖异常，可以清理缓存：

```
npm cache clean --force
```

然后重新安装：

```
rm -rf node_modules package-lock.json  
npm install
```

Windows PowerShell 可以用：

```
Remove-Item -Recurse -Force node_modules  
Remove-Item -Force package-lock.json  
npm install
```

二十二、创建一个完整 npm 小项目

项目结构：

```
npm-demo  
├─ app.js  
├─ package.json  
└─ package-lock.json
```

步骤：

```
mkdir npm-demo  
cd npm-demo  
npm init -y  
npm install dayjs  
npm install nodemon -D
```

修改 package.json：

```
{  
  "name": "npm-demo",  
  "version": "1.0.0",  
  "main": "app.js",  
  "scripts": {  
    "start": "node app.js",  
    "dev": "nodemon app.js"  
  },  
  "dependencies": {  
    "dayjs": "^1.11.13"  
  },  
  "devDependencies": {  
    "nodemon": "^3.1.0"  
  }  
}
```

```
}  
}
```

app.js :

```
const dayjs = require("dayjs")  
  
console.log("当前时间: ", dayjs().format("YYYY-MM-DD HH:mm:ss"))
```

开发运行：

```
npm run dev
```

正式运行：

```
npm start
```

二十三、常见错误

1. 没有 package.json 就安装依赖

如果你在一个没有初始化的目录里安装包，可能项目结构不清晰。

推荐先执行：

```
npm init -y
```

再安装依赖。

2. 把 node_modules 上传到 Git

不推荐上传：

```
node_modules
```

应该在 .gitignore 中添加：

```
node_modules
```

别人下载项目后执行：

```
npm install
```

即可恢复依赖。

3. 命令输错

错误：

```
npm install express -D
```

这会把 express 安装到开发依赖中。

Express 是生产环境需要的，应该：

```
npm install express
```

4. 删除 package-lock.json 后版本变化

如果删除 package-lock.json 后重新安装，可能会安装到更新的兼容版本。

团队项目中建议保留 package-lock.json。

本课重点

这一课你需要掌握：

1. npm 是 Node.js 的包管理工具
 2. npm init -y 初始化项目
 3. package.json 是项目配置文件
 4. npm install 包名 安装生产依赖
 5. npm install 包名 -D 安装开发依赖
 6. node_modules 存放下载的包
 7. package-lock.json 锁定依赖版本
 8. npm uninstall 包名 卸载包
 9. scripts 可以配置项目命令
 10. npm run 脚本名 执行脚本
 11. nodemon 可以自动重启 Node 程序
 12. dependencies 是生产依赖
 13. devDependencies 是开发依赖
-

课后练习

创建一个项目 `npm-practice`，完成以下要求：

1. 初始化项目
2. 安装 `dayjs`
3. 安装 `nodemon` 到开发依赖
4. 新建 `app.js`
5. 打印当前时间
6. 配置脚本：
 - `start`：使用 `node app.js`
 - `dev`：使用 `nodemon app.js`
7. 分别使用下面两个命令运行：

```
npm start
npm run dev
```

参考答案

命令：

```
mkdir npm-practice
cd npm-practice
npm init -y
npm install dayjs
npm install nodemon -D
```

`app.js`：

```
const dayjs = require("dayjs")

console.log("当前时间：", dayjs().format("YYYY-MM-DD HH:mm:ss"))
```

`package.json` 中的 `scripts`：

```
{
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  }
}
```

运行：

```
npm start
npm run dev
```

下一课建议学习：

Node.js 第六课：Express 框架入门。

Node.js 第六课：Express 框架入门

Express 是 Node.js 中非常常用的 Web 开发框架。

前面我们用原生 http 模块写服务器，会发现：

- 路由判断麻烦
- 处理 JSON 麻烦
- 处理 POST 请求体麻烦
- 返回响应不够方便
- 静态资源服务要自己写很多代码

而 Express 可以让这些事情变得简单。

一、Express 是什么？

Express 是基于 Node.js 的 Web 框架。

它可以用来开发：

- 网站
- 后端接口 API
- 管理系统后端
- 小程序后端
- App 后端
- 前后端分离项目接口

使用原生 http：

```
const http = require("http")

const server = http.createServer((req, res) => {
  if (req.url === "/" && req.method === "GET") {
    res.end("首页")
  }
})

server.listen(3000)
```

使用 Express：

```
const express = require("express")

const app = express()

app.get("/", (req, res) => {
```

```
    res.send("首页")
  })

  app.listen(3000)
```

明显更简洁。

二、创建 Express 项目

创建项目目录：

```
mkdir express-demo
cd express-demo
```

初始化项目：

```
npm init -y
```

安装 Express：

```
npm install express
```

安装开发工具 nodemon：

```
npm install nodemon -D
```

项目结构：

```
express-demo
├─ app.js
├─ package.json
└─ node_modules
```

三、第一个 Express 服务器

新建 app.js：

```
const express = require("express")

const app = express()

app.get("/", (req, res) => {
  res.send("Hello Express")
})
```

```
app.listen(3000, () => {  
  console.log("服务器启动: http://localhost:3000")  
})
```

运行：

```
node app.js
```

浏览器访问：

```
http://localhost:3000
```

页面显示：

```
Hello Express
```

四、配置启动脚本

修改 package.json：

```
{  
  "scripts": {  
    "start": "node app.js",  
    "dev": "nodemon app.js"  
  }  
}
```

以后可以这样启动：

```
npm start
```

开发时推荐：

```
npm run dev
```

这样修改代码后服务器会自动重启。

五、Express 的基本结构

```
const express = require("express")  
  
const app = express()
```

```
app.get("/", (req, res) => {
  res.send("首页")
})

app.listen(3000, () => {
  console.log("服务器启动成功")
})
```

解释：

```
const express = require("express")
```

引入 Express。

```
const app = express()
```

创建一个 Express 应用。

```
app.get("/", callback)
```

定义一个 GET 路由。

```
app.listen(3000)
```

监听 3000 端口，启动服务器。

六、路由基础

路由可以理解为：

不同的请求地址，对应不同的处理逻辑。

1. GET 路由

```
app.get("/", (req, res) => {
  res.send("首页")
})

app.get("/about", (req, res) => {
  res.send("关于我们")
})

app.get("/contact", (req, res) => {
```



```
res.send("联系我们")
})
```

访问：

```
http://localhost:3000/
http://localhost:3000/about
http://localhost:3000/contact
```

2. POST 路由

```
app.post("/login", (req, res) => {
  res.send("登录接口")
})
```

浏览器地址栏默认是 GET 请求，所以 POST 请求一般用：

- Postman
- Apifox
- curl
- 前端 axios/fetch

测试：

```
curl -X POST http://localhost:3000/login
```

3. PUT 路由

通常用于修改数据：

```
app.put("/user", (req, res) => {
  res.send("修改用户")
})
```

4. DELETE 路由

通常用于删除数据：

```
app.delete("/user", (req, res) => {
  res.send("删除用户")
})
```

七、Express 响应数据

Express 的 `res` 比原生 `http` 更方便。

1. `res.send()`

返回普通文本或 HTML：

```
app.get("/", (req, res) => {  
  res.send("Hello Express")  
})
```

也可以返回 HTML：

```
app.get("/html", (req, res) => {  
  res.send("<h1>首页</h1><p>Hello Express</p>")  
})
```

2. `res.json()`

返回 JSON 数据：

```
app.get("/user", (req, res) => {  
  res.json({  
    id: 1,  
    name: "张三",  
    age: 20  
  })  
})
```

访问：

```
http://localhost:3000/user
```

返回：

```
{  
  "id": 1,  
  "name": "张三",  
  "age": 20  
}
```

`res.json()` 会自动设置：

```
Content-Type: application/json
```

不需要手动 `JSON.stringify()`。

3. res.status()

设置状态码：

```
app.get("/not-found", (req, res) => {  
  res.status(404).send("页面不存在")  
})
```

也可以配合 JSON：

```
app.get("/error", (req, res) => {  
  res.status(500).json({  
    message: "服务器错误"  
  })  
})
```

4. res.redirect()

重定向：

```
app.get("/old", (req, res) => {  
  res.redirect("/new")  
})  
  
app.get("/new", (req, res) => {  
  res.send("这是新页面")  
})
```

访问：

```
http://localhost:3000/old
```

会自动跳转到：

```
http://localhost:3000/new
```

八、获取查询参数 req.query

查询参数就是 URL 中 `?` 后面的内容。

例如：

```
http://localhost:3000/search?keyword=node&page=1
```

Express 中用 `req.query` 获取：

```
app.get("/search", (req, res) => {
  console.log(req.query)

  res.json({
    keyword: req.query.keyword,
    page: req.query.page
  })
})
```

访问：

```
http://localhost:3000/search?keyword=node&page=1
```

返回：

```
{
  "keyword": "node",
  "page": "1"
}
```

注意：查询参数默认都是字符串。

如果需要数字：

```
const page = Number(req.query.page)
```

九、获取动态路由参数 `req.params`

动态路由常用于根据 id 查询详情。

```
app.get("/users/:id", (req, res) => {
  res.json({
    id: req.params.id
  })
})
```

访问：

```
http://localhost:3000/users/100
```

返回：

```
{
  "id": "100"
}
```

也可以多个参数：

```
app.get("/articles/:category/:id", (req, res) => {
  res.json({
    category: req.params.category,
    id: req.params.id
  })
})
```

访问：

```
http://localhost:3000/articles/node/10
```

返回：

```
{
  "category": "node",
  "id": "10"
}
```

十、接收 JSON 请求体

Express 需要使用中间件解析请求体。

在路由前添加：

```
app.use(express.json())
```

完整示例：

```
const express = require("express")

const app = express()

app.use(express.json())

app.post("/login", (req, res) => {
  console.log(req.body)
})
```

```
res.json({
  message: "登录成功",
  data: req.body
})
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

测试：

```
curl -X POST http://localhost:3000/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"123456"}'
```

返回：

```
{
  "message": "登录成功",
  "data": {
    "username": "admin",
    "password": "123456"
  }
}
```

十一、接收表单请求体

如果前端提交的是普通表单：

```
Content-Type: application/x-www-form-urlencoded
```

需要添加：

```
app.use(express.urlencoded({ extended: false }))
```

示例：

```
const express = require("express")

const app = express()

app.use(express.urlencoded({ extended: false }))

app.post("/form", (req, res) => {
  res.json({
    message: "接收到表单数据",
    data: req.body
  })
})
```

```
    })  
  })  
  
  app.listen(3000)
```

测试：

```
curl -X POST http://localhost:3000/form \  
-H "Content-Type: application/x-www-form-urlencoded" \  
-d "username=admin&password=123456"
```

返回：

```
{  
  "message": "接收到表单数据",  
  "data": {  
    "username": "admin",  
    "password": "123456"  
  }  
}
```

实际项目通常两个都写：

```
app.use(express.json())  
app.use(express.urlencoded({ extended: false }))
```

十二、中间件 middleware 简单理解

中间件是 Express 非常重要的概念。

它本质上是一个函数：

```
function middleware(req, res, next) {  
  // 处理逻辑  
  next()  
}
```

比如：

```
app.use((req, res, next) => {  
  console.log("请求方法: ", req.method)  
  console.log("请求地址: ", req.url)  
  next()  
})
```

完整示例：

```
const express = require("express")

const app = express()

app.use((req, res, next) => {
  console.log(`[${new Date().toLocaleString()}] ${req.method} ${req.url}`)
  next()
})

app.get("/", (req, res) => {
  res.send("首页")
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

访问 `/` 后，终端会打印请求日志。

注意：

```
next()
```

表示继续执行下一个中间件或路由。

如果不调用 `next()`，请求可能会一直卡住。

十三、静态资源托管 `express.static`

如果你想让浏览器访问 HTML、CSS、JS、图片，可以使用：

```
app.use(express.static("public"))
```

项目结构：

```
express-demo
├─ app.js
└─ public
   ├─ index.html
   ├─ style.css
   └─ logo.png
```

`app.js`：

```
const express = require("express")
const path = require("path")

const app = express()
```



```
app.use(express.static(path.join(__dirname, "public")))

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

访问：

```
http://localhost:3000/index.html
http://localhost:3000/style.css
http://localhost:3000/logo.png
```

如果 `public` 里有 `index.html`，访问：

```
http://localhost:3000/
```

会自动显示 `index.html`。

十四、实现一个简单用户接口

下面我们写一个完整的小 API。

项目结构：

```
express-demo
├─ app.js
└─ package.json
```

`app.js`：

```
const express = require("express")

const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: false }))

const users = [
  { id: 1, name: "张三", age: 20 },
  { id: 2, name: "李四", age: 22 },
  { id: 3, name: "王五", age: 25 }
]

// 首页
app.get("/", (req, res) => {
  res.send("<h1>Express 用户接口</h1>")
})

// 获取用户列表
app.get("/api/users", (req, res) => {
```

```
res.json({
  code: 0,
  message: "获取成功",
  data: users
})
})

// 根据 id 获取用户，动态路由方式
app.get("/api/users/:id", (req, res) => {
  const id = Number(req.params.id)

  const user = users.find(item => item.id === id)

  if (!user) {
    return res.status(404).json({
      code: 404,
      message: "用户不存在"
    })
  }

  res.json({
    code: 0,
    message: "获取成功",
    data: user
  })
})
```

```
// 新增用户
app.post("/api/users", (req, res) => {
  const { name, age } = req.body

  if (!name || !age) {
    return res.status(400).json({
      code: 400,
      message: "name 和 age 不能为空"
    })
  }

  const newUser = {
    id: users.length + 1,
    name,
    age: Number(age)
  }

  users.push(newUser)

  res.status(201).json({
    code: 0,
    message: "新增成功",
    data: newUser
  })
})
```

```
// 修改用户
app.put("/api/users/:id", (req, res) => {
  const id = Number(req.params.id)
  const { name, age } = req.body
```

```
const user = users.find(item => item.id === id)

if (!user) {
  return res.status(404).json({
    code: 404,
    message: "用户不存在"
  })
}

if (name) user.name = name
if (age) user.age = Number(age)

res.json({
  code: 0,
  message: "修改成功",
  data: user
})
})

// 删除用户
app.delete("/api/users/:id", (req, res) => {
  const id = Number(req.params.id)

  const index = users.findIndex(item => item.id === id)

  if (index === -1) {
    return res.status(404).json({
      code: 404,
      message: "用户不存在"
    })
  }

  const deletedUser = users.splice(index, 1)[0]

  res.json({
    code: 0,
    message: "删除成功",
    data: deletedUser
  })
})

// 404
app.use((req, res) => {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

十五、测试接口

启动服务：

```
npm run dev
```

或者：

```
node app.js
```

1. 获取用户列表

```
curl http://localhost:3000/api/users
```

2. 获取单个用户

```
curl http://localhost:3000/api/users/1
```

3. 新增用户

```
curl -X POST http://localhost:3000/api/users \
-H "Content-Type: application/json" \
-d '{"name":"赵六","age":28}'
```

4. 修改用户

```
curl -X PUT http://localhost:3000/api/users/1 \
-H "Content-Type: application/json" \
-d '{"name":"张三修改版","age":30}'
```

5. 删除用户

```
curl -X DELETE http://localhost:3000/api/users/1
```

十六、Express 路由拆分

项目变大后，不建议所有接口都写在 `app.js`。

可以拆分路由。

项目结构：

```
express-demo
├─ app.js
├─ routes
│   └─ user.js
```

routes/user.js

```
const express = require("express")

const router = express.Router()

const users = [
  { id: 1, name: "张三", age: 20 },
  { id: 2, name: "李四", age: 22 }
]

router.get("/", (req, res) => {
  res.json({
    code: 0,
    data: users
  })
})

router.get("/:id", (req, res) => {
  const id = Number(req.params.id)
  const user = users.find(item => item.id === id)

  if (!user) {
    return res.status(404).json({
      code: 404,
      message: "用户不存在"
    })
  }

  res.json({
    code: 0,
    data: user
  })
})

router.post("/", (req, res) => {
  const { name, age } = req.body

  const newUser = {
```

```
    id: users.length + 1,
    name,
    age: Number(age)
  }

  users.push(newUser)

  res.status(201).json({
    code: 0,
    message: "新增成功",
    data: newUser
  })
})

module.exports = router
```

app.js

```
const express = require("express")
const userRouter = require("./routes/user")

const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: false }))

app.get("/", (req, res) => {
  res.send("首页")
})

app.use("/api/users", userRouter)

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

这样：

```
app.use("/api/users", userRouter)
```

表示 `userRouter` 里的路由都自动加上 `/api/users` 前缀。

例如：

```
router.get("/")
```

实际访问地址是：

```
GET /api/users
```

```
router.get("/:id")
```

实际访问地址是：

```
GET /api/users/1
```

十七、常用第三方中间件

Express 经常搭配一些中间件使用。

包名	作用
cors	解决跨域
morgan	请求日志
multer	文件上传
cookie-parser	解析 Cookie
express-session	Session
jsonwebtoken	JWT 登录鉴权
bcryptjs	密码加密

比如安装 cors：

```
npm install cors
```

使用：

```
const cors = require("cors")

app.use(cors())
```

这样可以允许前端跨域请求后端接口。

十八、Express 和原生 http 对比

功能	原生 http	Express
创建服务器	可以	可以
路由管理	麻烦	简单
读取 query	手动解析	req.query
读取 params	需要自己实现	req.params

功能	原生 http	Express
读取 JSON body	手动监听 data/end	<code>express.json()</code>
返回 JSON	手动 stringify	<code>res.json()</code>
静态资源	手写 fs 读取	<code>express.static()</code>
中间件	需要自己设计	内置机制

十九、常见错误

1. 忘记安装 express

错误：

```
Error: Cannot find module 'express'
```

解决：

```
npm install express
```

2. 忘记调用 express()

错误写法：

```
const express = require("express")

app.get("/", (req, res) => {
  res.send("首页")
})
```

正确写法：

```
const express = require("express")

const app = express()
```

3. 忘记 app.listen

如果没有：

```
app.listen(3000)
```


服务器不会启动。

4. 忘记 express.json()

如果 POST JSON 时 `req.body` 是 `undefined`，一般是忘了：

```
app.use(express.json())
```

注意它必须写在路由之前：

```
app.use(express.json())

app.post("/login", (req, res) => {
  console.log(req.body)
})
```

5. 路由顺序写错

Express 按照代码顺序匹配路由。

错误示例：

```
app.use((req, res) => {
  res.status(404).send("404")
})

app.get("/user", (req, res) => {
  res.send("用户")
})
```

这样 `/user` 永远访问不到。

正确写法：

```
app.get("/user", (req, res) => {
  res.send("用户")
})

app.use((req, res) => {
  res.status(404).send("404")
})
```

404 中间件应该放在最后。

本课重点

这一课你需要掌握：

1. Express 是 Node.js 常用 Web 框架
2. 安装 Express：

```
npm install express
```

3. 创建服务器：

```
const express = require("express")
const app = express()
```

4. 定义 GET 路由：

```
app.get("/", (req, res) => {})
```

5. 定义 POST 路由：

```
app.post("/login", (req, res) => {})
```

6. 返回文本或 HTML：

```
res.send("内容")
```

7. 返回 JSON：

```
res.json({ message: "ok" })
```

8. 获取查询参数：

```
req.query
```

9. 获取动态参数：

```
req.params
```

10. 获取 JSON 请求体：

```
app.use(express.json())
```

11. 获取表单请求体：

```
app.use(express.urlencoded({ extended: false })))
```

12. 托管静态资源：

```
app.use(express.static("public"))
```

13. 使用 `express.Router()` 拆分路由

课后练习

创建一个 Express 项目，实现商品接口。

商品数据：

```
const products = [
  { id: 1, name: "苹果", price: 5 },
  { id: 2, name: "香蕉", price: 3 },
  { id: 3, name: "橙子", price: 4 }
]
```

要求实现：

方法	路径	功能
GET	/	返回首页
GET	/api/products	获取商品列表
GET	/api/products/:id	获取单个商品
POST	/api/products	新增商品
PUT	/api/products/:id	修改商品
DELETE	/api/products/:id	删除商品

参考答案

```
const express = require("express")

const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: false })))

const products = [
  { id: 1, name: "苹果", price: 5 },
  { id: 2, name: "香蕉", price: 3 },
```

```
{ id: 3, name: "橙子", price: 4 }
]

app.get("/", (req, res) => {
  res.send("<h1>商品管理接口</h1>")
})

app.get("/api/products", (req, res) => {
  res.json({
    code: 0,
    message: "获取成功",
    data: products
  })
})

app.get("/api/products/:id", (req, res) => {
  const id = Number(req.params.id)
  const product = products.find(item => item.id === id)

  if (!product) {
    return res.status(404).json({
      code: 404,
      message: "商品不存在"
    })
  }

  res.json({
    code: 0,
    message: "获取成功",
    data: product
  })
})

app.post("/api/products", (req, res) => {
  const { name, price } = req.body

  if (!name || price === undefined) {
    return res.status(400).json({
      code: 400,
      message: "name 和 price 不能为空"
    })
  }

  const newProduct = {
    id: products.length ? products[products.length - 1].id + 1 : 1,
    name,
    price: Number(price)
  }

  products.push(newProduct)

  res.status(201).json({
    code: 0,
    message: "新增成功",
    data: newProduct
  })
})
```

```
app.put("/api/products/:id", (req, res) => {
  const id = Number(req.params.id)
  const { name, price } = req.body

  const product = products.find(item => item.id === id)

  if (!product) {
    return res.status(404).json({
      code: 404,
      message: "商品不存在"
    })
  }

  if (name !== undefined) product.name = name
  if (price !== undefined) product.price = Number(price)

  res.json({
    code: 0,
    message: "修改成功",
    data: product
  })
})

app.delete("/api/products/:id", (req, res) => {
  const id = Number(req.params.id)
  const index = products.findIndex(item => item.id === id)

  if (index === -1) {
    return res.status(404).json({
      code: 404,
      message: "商品不存在"
    })
  }

  const deletedProduct = products.splice(index, 1)[0]

  res.json({
    code: 0,
    message: "删除成功",
    data: deletedProduct
  })
})

app.use((req, res) => {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

下一课建议学习：

Node.js 第七课：Express 路由和中间件深入。

Node.js 第八课：连接数据库 MySQL / MongoDB

这一课我们学习 Node.js 后端开发非常重要的一部分：连接数据库。

常见数据库分两类：

类型	数据库	特点
关系型数据库	MySQL、PostgreSQL	表结构清晰，适合订单、用户、商品等业务
非关系型数据库	MongoDB、Redis	文档结构灵活，适合内容、日志、配置等场景

本课重点学习：

- 1. Node.js 连接 MySQL
- 2. Node.js 连接 MongoDB
- 3. 在 Express 中封装数据库操作
- 4. 实现简单增删改查接口

一、MySQL 和 MongoDB 的区别

1. MySQL

MySQL 是关系型数据库。

数据像 Excel 表格一样存储。

例如用户表 users：

id	name	age	email
1	张三	20	zhangsan@test.com
2	李四	22	lisi@test.com

特点：

- 有表结构
- 使用 SQL 语句
- 适合结构化数据
- 常用于电商、后台管理、订单系统

2. MongoDB

MongoDB 是文档型数据库。

数据像 JSON 一样存储。

例如用户集合 `users`：

```
{
  "_id": "6656...",
  "name": "张三",
  "age": 20,
  "email": "zhangsan@test.com"
}
```

特点：

- 数据结构灵活
- 使用集合 `collection`
- 文档格式类似 JSON
- 适合内容系统、日志、动态字段数据

二、Node.js 连接 MySQL

Node.js 连接 MySQL 常用包：

```
mysql2
```

安装：

```
npm install mysql2
```

如果还没有 Express 项目，可以创建：

```
mkdir db-demo
cd db-demo
npm init -y
npm install express mysql2
npm install nodemon -D
```

配置 `package.json`：

```
{
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  }
}
```

三、准备 MySQL 数据库

假设你已经安装 MySQL。

创建数据库：

```
CREATE DATABASE node_demo DEFAULT CHARACTER SET utf8mb4;
```

使用数据库：

```
USE node_demo;
```

创建用户表：

```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(50) NOT NULL,  
  age INT,  
  email VARCHAR(100)  
);
```

插入测试数据：

```
INSERT INTO users (name, age, email) VALUES  
( '张三', 20, 'zhangsan@test.com'),  
( '李四', 22, 'lisi@test.com'),  
( '王五', 25, 'wangwu@test.com');
```

查看数据：

```
SELECT * FROM users;
```

四、最简单的 MySQL 连接

新建 mysql-test.js：

```
const mysql = require("mysql2")  
  
const connection = mysql.createConnection({  
  host: "localhost",  
  user: "root",  
  password: "你的数据库密码",  
  database: "node_demo"  
})  
  
connection.query("SELECT * FROM users", (err, results) => {  
  if (err) {  
    console.log("查询失败: ", err)  
  }  
})
```



```
    return
  }

  console.log(results)
})

connection.end()
```

运行：

```
node mysql-test.js
```

输出类似：

```
[
  { id: 1, name: '张三', age: 20, email: 'zhangsan@test.com' },
  { id: 2, name: '李四', age: 22, email: 'lisi@test.com' },
  { id: 3, name: '王五', age: 25, email: 'wangwu@test.com' }
]
```

五、推荐使用连接池 pool

真实项目中不推荐每次都创建一个连接。

推荐使用连接池：

```
const mysql = require("mysql2/promise")

const pool = mysql.createPool({
  host: "localhost",
  user: "root",
  password: "你的数据库密码",
  database: "node_demo",
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
})

module.exports = pool
```

连接池的好处：

- 自动管理多个连接
- 性能更好
- 适合 Web 服务并发请求

六、MySQL 项目结构

推荐这样组织项目：

```
db-demo
├─ app.js
├─ db
│   └─ mysql.js
├─ routes
│   └─ user.js
```

1. db/mysql.js

```
const mysql = require("mysql2/promise")

const pool = mysql.createPool({
  host: "localhost",
  user: "root",
  password: "你的数据库密码",
  database: "node_demo",
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
})

module.exports = pool
```

2. routes/user.js

```
const express = require("express")
const pool = require("../db/mysql")

const router = express.Router()

// 获取用户列表
router.get("/", async (req, res) => {
  try {
    const [rows] = await pool.query("SELECT * FROM users")

    res.json({
      code: 0,
      message: "获取成功",
      data: rows
    })
  } catch (err) {
    res.status(500).json({
      code: 500,
      message: "数据库查询失败"
    })
  }
})
```

```
})

// 根据 id 获取用户
router.get("/:id", async (req, res) => {
  try {
    const id = Number(req.params.id)

    const [rows] = await pool.query(
      "SELECT * FROM users WHERE id = ?",
      [id]
    )

    if (rows.length === 0) {
      return res.status(404).json({
        code: 404,
        message: "用户不存在"
      })
    }

    res.json({
      code: 0,
      message: "获取成功",
      data: rows[0]
    })
  } catch (err) {
    res.status(500).json({
      code: 500,
      message: "数据库查询失败"
    })
  }
})

// 新增用户
router.post("/", async (req, res) => {
  try {
    const { name, age, email } = req.body

    if (!name) {
      return res.status(400).json({
        code: 400,
        message: "name 不能为空"
      })
    }

    const [result] = await pool.query(
      "INSERT INTO users (name, age, email) VALUES (?, ?, ?)",
      [name, age, email]
    )

    res.status(201).json({
      code: 0,
      message: "新增成功",
      data: {
        id: result.insertId,
        name,
        age,
        email
      }
    })
  }
})
```

```

    }
  })
} catch (err) {
  res.status(500).json({
    code: 500,
    message: "数据库新增失败"
  })
}
})

// 修改用户
router.put("/:id", async (req, res) => {
  try {
    const id = Number(req.params.id)
    const { name, age, email } = req.body

    const [result] = await pool.query(
      "UPDATE users SET name = ?, age = ?, email = ? WHERE id = ?",
      [name, age, email, id]
    )

    if (result.affectedRows === 0) {
      return res.status(404).json({
        code: 404,
        message: "用户不存在"
      })
    }

    res.json({
      code: 0,
      message: "修改成功"
    })
  } catch (err) {
    res.status(500).json({
      code: 500,
      message: "数据库修改失败"
    })
  }
})

```

```

// 删除用户
router.delete("/:id", async (req, res) => {
  try {
    const id = Number(req.params.id)

    const [result] = await pool.query(
      "DELETE FROM users WHERE id = ?",
      [id]
    )

    if (result.affectedRows === 0) {
      return res.status(404).json({
        code: 404,
        message: "用户不存在"
      })
    }
  }
}

```

```
    res.json({
      code: 0,
      message: "删除成功"
    })
  } catch (err) {
    res.status(500).json({
      code: 500,
      message: "数据库删除失败"
    })
  }
})

module.exports = router
```

3. app.js

```
const express = require("express")
const userRouter = require("./routes/user")

const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: false }))

app.get("/", (req, res) => {
  res.send("Node.js MySQL Demo")
})

app.use("/api/users", userRouter)

app.use((req, res) => {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

启动:

```
npm run dev
```

测试:

```
curl http://localhost:3000/api/users
```

新增用户:

```
curl -X POST http://localhost:3000/api/users \
-H "Content-Type: application/json" \
-d '{"name":"赵六","age":28,"email":"zhaoliu@test.com"}'
```

七、为什么 SQL 要使用问号占位符？

推荐：

```
pool.query(
  "SELECT * FROM users WHERE id = ?",
  [id]
)
```

不要这样写：

```
pool.query(`SELECT * FROM users WHERE id = ${id}`)
```

原因是：防止 SQL 注入。

例如用户恶意传入：

```
1 OR 1=1
```

如果直接拼接 SQL，可能造成危险。

所以要使用参数化查询：

```
"WHERE id = ?"
```

八、Node.js 连接 MongoDB

Node.js 操作 MongoDB 常用两种方式：

工具	说明
mongodb	官方原生驱动
mongoose	更常用的 ODM，适合项目开发

本课使用 `mongoose`。

安装：

```
npm install mongoose
```

九、准备 MongoDB

假设你本地 MongoDB 地址是：

```
mongodb://127.0.0.1:27017/node_demo
```

其中：

```
node_demo
```

是数据库名，如果不存在，MongoDB 会在写入数据时自动创建。

十、最简单的 MongoDB 连接

新建 mongo-test.js：

```
const mongoose = require("mongoose")

async function main() {
  try {
    await mongoose.connect("mongodb://127.0.0.1:27017/node_demo")
    console.log("MongoDB 连接成功")
  } catch (err) {
    console.log("MongoDB 连接失败：", err)
  }
}

main()
```

运行：

```
node mongo-test.js
```

如果输出：

```
MongoDB 连接成功
```

说明连接成功。

十一、Mongoose 基本概念

Mongoose 中常见三个概念：

概念	说明
Schema	定义文档结构
Model	数据模型，用来操作数据库
Document	一条具体数据

例如用户 Schema：

```
const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
  email: String
})
```

创建 Model：

```
const User = mongoose.model("User", userSchema)
```

之后就可以：

```
User.find()
User.create()
User.findById()
User.findByIdAndUpdate()
User.findByIdAndDelete()
```

十二、MongoDB 项目结构

```
mongo-demo
├─ app.js
├─ db
│   └─ mongo.js
├─ models
│   └─ User.js
└─ routes
    └─ user.js
```

安装依赖：

```
mkdir mongo-demo
cd mongo-demo
npm init -y
```



```
npm install express mongoose
npm install nodemon -D
```

1. db/mongo.js

```
const mongoose = require("mongoose")

async function connectMongo() {
  try {
    await mongoose.connect("mongodb://127.0.0.1:27017/node_demo")
    console.log("MongoDB 连接成功")
  } catch (err) {
    console.log("MongoDB 连接失败: ", err)
    process.exit(1)
  }
}

module.exports = connectMongo
```

2. models/User.js

```
const mongoose = require("mongoose")

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true
    },
    age: {
      type: Number,
      default: 0
    },
    email: {
      type: String
    }
  },
  {
    timestamps: true
  }
)

const User = mongoose.model("User", userSchema)

module.exports = User
```

说明：

```
timestamps: true
```

会自动添加：

```
createdAt  
updatedAt
```

3. routes/user.js

```
const express = require("express")  
const User = require("../models/User")  
  
const router = express.Router()  
  
// 获取用户列表  
router.get("/", async (req, res) => {  
  try {  
    const users = await User.find().sort({ createdAt: -1 })  
  
    res.json({  
      code: 0,  
      message: "获取成功",  
      data: users  
    })  
  } catch (err) {  
    res.status(500).json({  
      code: 500,  
      message: "查询失败"  
    })  
  }  
})  
  
// 根据 id 获取用户  
router.get("/:id", async (req, res) => {  
  try {  
    const user = await User.findById(req.params.id)  
  
    if (!user) {  
      return res.status(404).json({  
        code: 404,  
        message: "用户不存在"  
      })  
    }  
  
    res.json({  
      code: 0,  
      message: "获取成功",  
      data: user  
    })  
  } catch (err) {  
    res.status(400).json({
```

```
        code: 400,  
        message: "用户 ID 格式错误"  
    })  
  }  
})
```

// 新增用户

```
router.post("/", async (req, res) => {  
  try {  
    const { name, age, email } = req.body  
  
    const user = await User.create({  
      name,  
      age,  
      email  
    })  
  
    res.status(201).json({  
      code: 0,  
      message: "新增成功",  
      data: user  
    })  
  } catch (err) {  
    res.status(400).json({  
      code: 400,  
      message: "新增失败",  
      error: err.message  
    })  
  }  
})
```

// 修改用户

```
router.put("/:id", async (req, res) => {  
  try {  
    const { name, age, email } = req.body  
  
    const user = await User.findByIdAndUpdate(  
      req.params.id,  
      { name, age, email },  
      {  
        new: true,  
        runValidators: true  
      }  
    )  
  
    if (!user) {  
      return res.status(404).json({  
        code: 404,  
        message: "用户不存在"  
      })  
    }  
  
    res.json({  
      code: 0,  
      message: "修改成功",  
      data: user  
    })  
  }  
})
```

```

    } catch (err) {
      res.status(400).json({
        code: 400,
        message: "修改失败",
        error: err.message
      })
    }
  })
})

// 删除用户
router.delete("/:id", async (req, res) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id)

    if (!user) {
      return res.status(404).json({
        code: 404,
        message: "用户不存在"
      })
    }

    res.json({
      code: 0,
      message: "删除成功",
      data: user
    })
  } catch (err) {
    res.status(400).json({
      code: 400,
      message: "删除失败"
    })
  }
})

module.exports = router

```

4. app.js

```

const express = require("express")
const connectMongo = require("./db/mongo")
const userRouter = require("./routes/user")

const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: false }))

connectMongo()

app.get("/", (req, res) => {
  res.send("Node.js MongoDB Demo")
})

```

```
app.use("/api/users", userRouter)

app.use((req, res) => {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

启动：

```
npm run dev
```

新增用户：

```
curl -X POST http://localhost:3000/api/users \
-H "Content-Type: application/json" \
-d '{"name":"张三","age":20,"email":"zhangsan@test.com"}'
```

获取用户：

```
curl http://localhost:3000/api/users
```

十三、MySQL 增删改查 SQL 总结

1. 查询全部

```
SELECT * FROM users;
```

Node.js：

```
const [rows] = await pool.query("SELECT * FROM users")
```

2. 条件查询

```
SELECT * FROM users WHERE id = ?;
```

Node.js：

```
const [rows] = await pool.query(  
  "SELECT * FROM users WHERE id = ?",  
  [id]  
)
```

3. 新增

```
INSERT INTO users (name, age, email) VALUES (?, ?, ?);
```

Node.js:

```
const [result] = await pool.query(  
  "INSERT INTO users (name, age, email) VALUES (?, ?, ?)",  
  [name, age, email]  
)
```

4. 修改

```
UPDATE users SET name = ?, age = ?, email = ? WHERE id = ?;
```

Node.js:

```
const [result] = await pool.query(  
  "UPDATE users SET name = ?, age = ?, email = ? WHERE id = ?",  
  [name, age, email, id]  
)
```

5. 删除

```
DELETE FROM users WHERE id = ?;
```

Node.js:

```
const [result] = await pool.query(  
  "DELETE FROM users WHERE id = ?",  
  [id]  
)
```

十四、MongoDB 增删改查总结

1. 查询全部

```
const users = await User.find()
```

2. 根据 ID 查询

```
const user = await User.findById(id)
```

3. 新增

```
const user = await User.create({  
  name,  
  age,  
  email  
})
```

4. 修改

```
const user = await User.findByIdAndUpdate(  
  id,  
  { name, age, email },  
  { new: true }  
)
```

5. 删除

```
const user = await User.findByIdAndDelete(id)
```

十五、环境变量管理数据库配置

真实项目不建议把数据库密码直接写死在代码里。

推荐使用 `.env` 文件。

安装：

```
npm install dotenv
```

项目根目录创建 `.env`：

```
PORT=3000

MYSQL_HOST=localhost
MYSQL_USER=root
MYSQL_PASSWORD=你的数据库密码
MYSQL_DATABASE=node_demo

MONGO_URL=mongodb://127.0.0.1:27017/node_demo
```

在入口文件最上方加载：

```
require("dotenv").config()
```

MySQL 配置：

```
const mysql = require("mysql2/promise")

const pool = mysql.createPool({
  host: process.env.MYSQL_HOST,
  user: process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE
})

module.exports = pool
```

MongoDB 配置：

```
await mongoose.connect(process.env.MONGO_URL)
```

`.env` 不要提交到 Git。

`.gitignore`：

```
node_modules
.env
```

十六、MySQL 和 MongoDB 该选哪个？

简单建议：

选 MySQL 的场景

- 用户系统
- 订单系统
- 支付系统
- 商品库存
- 后台管理系统
- 数据关系明确的业务

例如：

```
用户表
订单表
商品表
订单详情表
```

这些关系明确，MySQL 很适合。

选 MongoDB 的场景

- 博客内容
- 评论系统
- 日志系统
- 聊天记录
- 配置数据
- 字段变化频繁的数据

例如文章：

```
{
  "title": "Node.js 教程",
  "content": "...",
  "tags": ["node", "express"],
  "author": {
    "name": "张三",
    "avatar": "xxx"
  }
}
```

MongoDB 会比较灵活。

十七、常见错误

1. MySQL 连接失败

常见错误：

```
Access denied for user 'root'@'localhost'
```

原因：

- 用户名错
- 密码错
- MySQL 用户权限不足

检查：

```
user: "root",  
password: "你的真实密码"
```

2. MySQL 数据库不存在

错误：

```
Unknown database 'node_demo'
```

解决：

```
CREATE DATABASE node_demo DEFAULT CHARACTER SET utf8mb4;
```

3. MySQL 表不存在

错误：

```
Table 'node_demo.users' doesn't exist
```

解决：

```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(50) NOT NULL,  
  age INT,  
  email VARCHAR(100)  
);
```

4. MongoDB 连接失败

错误：

可能原因：

- MongoDB 服务没有启动
- 连接地址错误
- 端口不是 27017

检查：

```
mongodb://127.0.0.1:27017/node_demo
```

5. Mongoose ObjectId 格式错误

如果访问：

```
/api/users/abc
```

可能报错，因为 `abc` 不是合法的 MongoDB `_id`。

可以用 `mongoose.Types.ObjectId.isValid()` 判断：

```
const mongoose = require("mongoose")

if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
  return res.status(400).json({
    code: 400,
    message: "用户 ID 格式错误"
  })
}
```

十八、一个更规范的错误处理方式

可以把异步错误统一交给错误中间件。

封装 `utils/asyncHandler.js`：

```
function asyncHandler(fn) {
  return function (req, res, next) {
    Promise.resolve(fn(req, res, next)).catch(next)
  }
}

module.exports = asyncHandler
```

使用：

```
const asyncHandler = require("../utils/asyncHandler")

router.get("/", asyncHandler(async (req, res) => {
  const users = await User.find()

  res.json({
    code: 0,
    data: users
  })
}))
```

错误处理中间件：

```
app.use((err, req, res, next) => {
  console.error(err)

  res.status(500).json({
    code: 500,
    message: "服务器内部错误"
  })
})
```

这样代码会更简洁。

十九、本课重点

这一课你需要掌握：

1. MySQL 是关系型数据库
2. MongoDB 是文档型数据库
3. Node.js 操作 MySQL 常用 `mysql2`
4. Node.js 操作 MongoDB 常用 `mongoose`
5. MySQL 推荐使用连接池 `createPool`
6. MySQL 查询推荐使用参数化查询：

```
"WHERE id = ?"
```

7. Mongoose 通过 Schema 定义数据结构
 8. Mongoose 通过 Model 操作集合
 9. Express 中数据库操作通常放在路由或控制器中
 10. 数据库配置推荐放到 `.env`
 11. `.env` 不要提交到 Git
-

二十、课后练习

任选 MySQL 或 MongoDB，完成一个商品接口。

商品字段：

```
{
  name: "苹果",
  price: 5,
  stock: 100
}
```

实现接口：

方法	路径	功能
GET	/api/products	获取商品列表
GET	/api/products/:id	获取单个商品
POST	/api/products	新增商品
PUT	/api/products/:id	修改商品
DELETE	/api/products/:id	删除商品

MySQL 建表参考

```
CREATE TABLE products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  price DECIMAL(10, 2) NOT NULL,
  stock INT DEFAULT 0
);
```

MongoDB Model 参考

```
const mongoose = require("mongoose")

const productSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true
    },
    price: {
      type: Number,
      required: true
    },
  },
)
```

```
    stock: {
      type: Number,
      default: 0
    },
    {
      timestamps: true
    }
  )

module.exports = mongoose.model("Product", productSchema)
```

下一课建议学习：

Node.js 第九课：登录注册、密码加密与 JWT 鉴权。

Node.js 第九课：登录注册、密码加密与 JWT 鉴权

这一课学习后端项目里非常核心的功能：

1. 用户注册
2. 密码加密
3. 用户登录
4. 生成 JWT Token
5. 校验 Token
6. 保护需要登录才能访问的接口

一、为什么不能明文保存密码？

很多初学者会这样保存用户：

```
{
  username: "admin",
  password: "123456"
}
```

这是非常危险的。

如果数据库泄露，所有用户密码都会直接暴露。

正确做法是：

```
{
  username: "admin",
  password: "$2a$10$N9qo8uL0ickgx2ZMRZoMye..."
}
```

也就是保存 加密后的密码哈希值。

二、密码加密常用库 bcryptjs

安装：

```
npm install bcryptjs
```

使用：

```
const bcrypt = require("bcryptjs")
```

1. 加密密码

```
const bcrypt = require("bcryptjs")

const password = "123456"

const hash = bcrypt.hashSync(password, 10)

console.log(hash)
```

输出类似：

```
$2a$10$wV1hF6ODkCVSTczEXAMPLE...
```

这里的 10 是加密强度，通常使用 10 或 12。

2. 校验密码

```
const bcrypt = require("bcryptjs")

const password = "123456"
const hash = "$2a$10$wV1hF6ODkCVSTczEXAMPLE..."

const isMatch = bcrypt.compareSync(password, hash)

console.log(isMatch)
```

如果密码正确，返回：

```
true
```

如果错误，返回：

```
false
```

三、什么是 JWT?

JWT 全称：

```
JSON Web Token
```

它是一种常见的登录认证方案。

登录成功后，服务器生成一个 Token 返回给前端。

之后前端请求需要登录的接口时，把 Token 放在请求头中。

流程：

1. 用户提交账号密码
2. 后端验证账号密码
3. 验证成功，生成 JWT Token
4. 前端保存 Token
5. 前端请求接口时携带 Token
6. 后端校验 Token
7. 校验通过，允许访问接口

四、安装 jsonwebtoken

```
npm install jsonwebtoken
```

引入：

```
const jwt = require("jsonwebtoken")
```

五、生成 Token

```
const jwt = require("jsonwebtoken")

const token = jwt.sign(
  {
    id: 1,
    username: "admin"
```



```
    },
    "my_secret_key",
    {
      expiresIn: "2h"
    }
  )

  console.log(token)
```

说明：

```
jwt.sign(payload, secret, options)
```

参数	说明
payload	要存入 Token 的用户信息
secret	密钥
expiresIn	过期时间

常见过期时间：

```
expiresIn: "30m" // 30 分钟
expiresIn: "2h"  // 2 小时
expiresIn: "7d"  // 7 天
```

六、验证 Token

```
const jwt = require("jsonwebtoken")

try {
  const decoded = jwt.verify(token, "my_secret_key")
  console.log(decoded)
} catch (err) {
  console.log("Token 无效或已过期")
}
```

解码后可能得到：

```
{
  id: 1,
  username: "admin",
  iat: 1710000000,
  exp: 171007200
}
```

其中：

字段	说明
iat	Token 签发时间
exp	Token 过期时间

七、创建登录注册项目

创建项目：

```
mkdir auth-demo
cd auth-demo
npm init -y
npm install express bcryptjs jsonwebtoken dotenv
npm install nodemon -D
```

修改 package.json：

```
{
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  }
}
```

项目结构：

```
auth-demo
├─ app.js
├─ .env
├─ middlewares
│   └─ auth.js
└─ routes
    └─ auth.js
```

八、配置 .env

新建 .env：

```
PORT=3000
JWT_SECRET=node_auth_secret_123456
JWT_EXPIRES_IN=2h
```

注意：真实项目中 JWT_SECRET 要更复杂，并且不要提交到 Git。

.gitignore：

```
node_modules
.env
```

九、注册和登录接口

这里为了方便教学，先用数组模拟数据库。

真实项目中可以把 `users` 换成 MySQL 或 MongoDB 查询。

routes/auth.js

```
const express = require("express")
const bcrypt = require("bcryptjs")
const jwt = require("jsonwebtoken")

const router = express.Router()

// 模拟数据库
const users = []

// 注册
router.post("/register", async (req, res) => {
  const { username, password } = req.body

  if (!username || !password) {
    return res.status(400).json({
      code: 400,
      message: "用户名和密码不能为空"
    })
  }

  const existUser = users.find(user => user.username === username)

  if (existUser) {
    return res.status(409).json({
      code: 409,
      message: "用户名已存在"
    })
  }

  const hashedPassword = await bcrypt.hash(password, 10)

  const newUser = {
    id: users.length + 1,
    username,
    password: hashedPassword
  }

  users.push(newUser)
```

```

    res.status(201).json({
      code: 0,
      message: "注册成功",
      data: {
        id: newUser.id,
        username: newUser.username
      }
    })
  })
})

// 登录
router.post("/login", async (req, res) => {
  const { username, password } = req.body

  if (!username || !password) {
    return res.status(400).json({
      code: 400,
      message: "用户名和密码不能为空"
    })
  }

  const user = users.find(user => user.username === username)

  if (!user) {
    return res.status(401).json({
      code: 401,
      message: "用户名或密码错误"
    })
  }

  const isMatch = await bcrypt.compare(password, user.password)

  if (!isMatch) {
    return res.status(401).json({
      code: 401,
      message: "用户名或密码错误"
    })
  }

  const token = jwt.sign(
    {
      id: user.id,
      username: user.username
    },
    process.env.JWT_SECRET,
    {
      expiresIn: process.env.JWT_EXPIRES_IN
    }
  )

  res.json({
    code: 0,
    message: "登录成功",
    token
  })
})

```

```
module.exports = router
```

十、JWT 鉴权中间件

新建：

```
middlewares/auth.js
```

代码：

```
const jwt = require("jsonwebtoken")

function auth(req, res, next) {
  const authorization = req.headers.authorization

  if (!authorization) {
    return res.status(401).json({
      code: 401,
      message: "缺少 Authorization 请求头"
    })
  }

  const parts = authorization.split(" ")

  if (parts.length !== 2 || parts[0] !== "Bearer") {
    return res.status(401).json({
      code: 401,
      message: "Token 格式错误"
    })
  }

  const token = parts[1]

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET)

    req.user = decoded

    next()
  } catch (err) {
    return res.status(401).json({
      code: 401,
      message: "Token 无效或已过期"
    })
  }
}

module.exports = auth
```

十一、入口文件 app.js

```
require("dotenv").config()

const express = require("express")
const authRouter = require("./routes/auth")
const auth = require("./middlewares/auth")

const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: false }))

app.get("/", (req, res) => {
  res.send("Node.js 登录注册 JWT Demo")
})

// 登录注册路由
app.use("/api/auth", authRouter)

// 需要登录才能访问的接口
app.get("/api/profile", auth, (req, res) => {
  res.json({
    code: 0,
    message: "获取用户信息成功",
    data: {
      user: req.user
    }
  })
})

// 需要登录才能访问的订单接口
app.get("/api/orders", auth, (req, res) => {
  res.json({
    code: 0,
    message: "获取订单成功",
    data: [
      { id: 1, name: "订单1" },
      { id: 2, name: "订单2" }
    ]
  })
})

// 404
app.use((req, res) => {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
})

const PORT = process.env.PORT || 3000

app.listen(PORT, () => {
```

```
console.log(`服务器启动: http://localhost:${PORT}`)  
})
```

十二、测试接口

启动项目：

```
npm run dev
```

1. 注册

```
curl -X POST http://localhost:3000/api/auth/register \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"123456"}'
```

返回：

```
{  
  "code": 0,  
  "message": "注册成功",  
  "data": {  
    "id": 1,  
    "username": "admin"  
  }  
}
```

2. 登录

```
curl -X POST http://localhost:3000/api/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"123456"}'
```

返回：

```
{  
  "code": 0,  
  "message": "登录成功",  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
}
```

复制返回的 token。

3. 不带 Token 访问用户信息

```
curl http://localhost:3000/api/profile
```

返回：

```
{
  "code": 401,
  "message": "缺少 Authorization 请求头"
}
```

4. 带 Token 访问用户信息

```
curl http://localhost:3000/api/profile \
-H "Authorization: Bearer 你的token"
```

返回：

```
{
  "code": 0,
  "message": "获取用户信息成功",
  "data": {
    "user": {
      "id": 1,
      "username": "admin",
      "iat": 1710000000,
      "exp": 1710007200
    }
  }
}
```

十三、前端如何携带 Token?

前端登录成功后，一般会把 Token 保存起来。

例如：

```
localStorage.setItem("token", res.data.token)
```

请求接口时携带：

```
const token = localStorage.getItem("token")

fetch("http://localhost:3000/api/profile", {
  headers: {
```



```
    Authorization: `Bearer ${token}`  
  }  
})
```

如果使用 axios:

```
axios.get("http://localhost:3000/api/profile", {  
  headers: {  
    Authorization: `Bearer ${token}`  
  }  
})
```

十四、JWT 中不要存敏感信息

生成 Token 时，不要这样：

```
jwt.sign({  
  id: user.id,  
  username: user.username,  
  password: user.password  
}, secret)
```

错误原因：

- JWT 内容可以被前端解码看到
- JWT 只是签名防篡改，不是加密隐藏内容
- 不应该放密码、身份证号、银行卡号等敏感信息

推荐只放必要信息：

```
jwt.sign({  
  id: user.id,  
  username: user.username  
}, secret)
```

十五、用 MySQL 实现注册登录

如果你使用 MySQL，可以创建用户表：

```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

注册时：

```
const [rows] = await pool.query(
  "SELECT id FROM users WHERE username = ?",
  [username]
)

if (rows.length > 0) {
  return res.status(409).json({
    message: "用户名已存在"
  })
}

const hashedPassword = await bcrypt.hash(password, 10)

await pool.query(
  "INSERT INTO users (username, password) VALUES (?, ?)",
  [username, hashedPassword]
)
```

登录时：

```
const [rows] = await pool.query(
  "SELECT * FROM users WHERE username = ?",
  [username]
)

const user = rows[0]

const isMatch = await bcrypt.compare(password, user.password)
```

十六、用 MongoDB 实现注册登录

如果你使用 MongoDB，可以创建 User Model：

```
const mongoose = require("mongoose")

const userSchema = new mongoose.Schema(
  {
    username: {
      type: String,
      required: true,
      unique: true
    },
    password: {
      type: String,
      required: true
    }
  },
  {
    timestamps: true
  }
)
```

```
    }  
  )  
  
  module.exports = mongoose.model("User", userSchema)
```

注册时：

```
const existUser = await User.findOne({ username })  
  
if (existUser) {  
  return res.status(409).json({  
    message: "用户名已存在"  
  })  
}  
  
const hashedPassword = await bcrypt.hash(password, 10)  
  
const user = await User.create({  
  username,  
  password: hashedPassword  
})
```

登录时：

```
const user = await User.findOne({ username })  
  
const isMatch = await bcrypt.compare(password, user.password)
```

十七、登录注册接口常见状态码

状态码	场景
200	登录成功、查询成功
201	注册成功、创建成功
400	参数错误
401	未登录、Token 无效、密码错误
403	已登录但无权限
404	资源不存在
409	用户名冲突
500	服务器错误

十八、Token 过期怎么办？

Token 一般会设置过期时间：

```
expiresIn: "2h"
```

过期后，后端会返回：

```
{
  "code": 401,
  "message": "Token 无效或已过期"
}
```

前端可以：

1. 跳转登录页
2. 清除本地 Token
3. 提示用户重新登录

例如：

```
localStorage.removeItem("token")
location.href = "/login"
```

十九、真实项目中更推荐的结构

项目可以这样组织：

```
auth-project
├─ app.js
├─ .env
├─ db
│   └─ mysql.js
├─ routes
│   └─ auth.js
├─ controllers
│   └─ authController.js
├─ middlewares
│   └─ auth.js
├─ models
│   └─ User.js
└─ utils
    └─ jwt.js
```

简单项目可以先写在路由里。

项目变大后，建议拆分：

- routes：只负责路由
- controllers：负责业务逻辑

- `models`：负责数据库模型
 - `middlewares`：负责中间件
 - `utils`：负责工具函数
-

二十、常见错误

1. 忘记 `express.json()`

如果 `req.body` 是 `undefined`，检查是否写了：

```
app.use(express.json())
```

并且必须写在路由前面。

2. `JWT_SECRET` 不一致

生成 Token 用的是：

```
process.env.JWT_SECRET
```

验证 Token 也必须用同一个密钥：

```
jwt.verify(token, process.env.JWT_SECRET)
```

如果密钥不同，会验证失败。

3. Authorization 格式错误

推荐格式：

```
Authorization: Bearer token字符串
```

不是：

```
Authorization: token字符串
```

也不是：

```
token: token字符串
```

4. 把密码放进 Token

错误：

```
jwt.sign({  
  password: user.password  
}, secret)
```

不要这样做。

5. 注册时没有加密密码

错误：

```
password: password
```

正确：

```
password: await bcrypt.hash(password, 10)
```

二十一、本课重点

这一课你需要掌握：

1. 密码不能明文保存
2. bcryptjs 用来加密和校验密码
3. 注册时保存加密后的密码
4. 登录时使用 bcrypt.compare() 校验密码
5. jsonwebtoken 用来生成和验证 JWT
6. 登录成功后返回 Token
7. 前端请求接口时携带：

```
Authorization: Bearer token
```

8. 后端通过中间件校验 Token
9. 校验成功后可以把用户信息挂载到：

```
req.user
```

10. JWT 中不要存放敏感信息
 11. JWT_SECRET 应该放在 .env
-

课后练习

基于本课代码，实现以下接口：

方法	路径	是否需要登录	功能
POST	/api/auth/register	否	注册
POST	/api/auth/login	否	登录
GET	/api/profile	是	获取当前用户信息
GET	/api/orders	是	获取订单列表
POST	/api/logout	是	退出登录

说明：

JWT 本身是无状态的，简单退出登录可以由前端删除 Token 实现：

```
localStorage.removeItem("token")
```

后端 /api/logout 可以简单返回：

```
res.json({
  code: 0,
  message: "退出成功，请前端删除 Token"
})
```

下一课建议学习：

Node.js 第十课：文件上传 multer。

Node.js 第十课：文件上传 multer

这一课学习 Express 项目中常见的功能：文件上传。

比如：

- 上传头像
- 上传商品图片
- 上传文章封面
- 上传 PDF
- 上传 Excel
- 上传视频
- 多文件上传

在 Express 中，最常用的上传中间件是：

multer

一、什么是文件上传？

普通 JSON 请求一般是：

```
Content-Type: application/json
```

例如：

```
{
  "username": "admin",
  "password": "123456"
}
```

而文件上传通常使用：

```
Content-Type: multipart/form-data
```

因为文件不是普通字符串，需要用表单数据格式传输。

二、安装 multer

创建项目：

```
mkdir upload-demo
cd upload-demo
npm init -y
```

安装依赖：

```
npm install express multer
npm install nodemon -D
```

修改 package.json：

```
{
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  }
}
```


项目结构：

```
upload-demo
├─ app.js
├─ package.json
└─ uploads
```

创建上传目录：

```
mkdir uploads
```

三、最简单的单文件上传

新建 app.js：

```
const express = require("express")
const multer = require("multer")
const path = require("path")

const app = express()

const upload = multer({
  dest: "uploads/"
})

app.post("/upload", upload.single("file"), (req, res) => {
  res.json({
    code: 0,
    message: "上传成功",
    file: req.file
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

启动：

```
npm run dev
```

测试上传：

```
curl -X POST http://localhost:3000/upload \
-F "file=@./test.png"
```

注意：

```
-F "file=@./test.png"
```

这里的 `file` 必须和代码里的：

```
upload.single("file")
```

一致。

四、req.file 是什么？

上传成功后，`req.file` 大概是这样：

```
{
  fieldname: 'file',
  originalname: 'test.png',
  encoding: '7bit',
  mimetype: 'image/png',
  destination: 'uploads/',
  filename: '9f2c8d8d8d7c9a1b...',
  path: 'uploads/9f2c8d8d8d7c9a1b...',
  size: 12345
}
```

字段说明：

字段	说明
fieldname	表单字段名
originalname	原始文件名
encoding	编码
mimetype	文件类型
destination	保存目录
filename	保存后的文件名
path	文件路径
size	文件大小，单位字节

五、为什么上传后文件没有后缀？

如果使用：

```
multer({ dest: "uploads/" })
```

multer 会自动生成随机文件名，但默认不会保留扩展名。

例如原文件：

```
avatar.png
```

保存后可能变成：

```
uploads/9f2c8d8d8d7c9a1b
```

如果想保留扩展名，需要自定义存储方式。

六、自定义文件名和保存目录

使用 `multer.diskStorage()`。

```
const express = require("express")
const multer = require("multer")
const path = require("path")

const app = express()

const storage = multer.diskStorage({
  destination(req, file, cb) {
    cb(null, "uploads/")
  },
  filename(req, file, cb) {
    const ext = path.extname(file.originalname)
    const filename = Date.now() + "-" + Math.round(Math.random() * 1e9) + ext
    cb(null, filename)
  }
})

const upload = multer({
  storage
})

app.post("/upload", upload.single("file"), (req, res) => {
  res.json({
    code: 0,
    message: "上传成功",
    file: req.file
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

上传后文件名类似：

uploads/1716888888888-123456789.png

七、让上传的图片可以被浏览器访问

上传到 `uploads` 后，浏览器默认不能直接访问，需要托管静态资源。

```
app.use("/uploads", express.static(path.join(__dirname, "uploads")))
```

完整示例：

```
const express = require("express")
const multer = require("multer")
const path = require("path")

const app = express()

app.use("/uploads", express.static(path.join(__dirname, "uploads")))

const storage = multer.diskStorage({
  destination(req, file, cb) {
    cb(null, path.join(__dirname, "uploads"))
  },
  filename(req, file, cb) {
    const ext = path.extname(file.originalname)
    const filename = Date.now() + "-" + Math.round(Math.random() * 1e9) + ext
    cb(null, filename)
  }
})

const upload = multer({ storage })

app.post("/upload", upload.single("file"), (req, res) => {
  const fileUrl = `/uploads/${req.file.filename}`

  res.json({
    code: 0,
    message: "上传成功",
    data: {
      filename: req.file.filename,
      originalname: req.file.originalname,
      mimetype: req.file.mimetype,
      size: req.file.size,
      url: fileUrl
    }
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

上传后返回：

```
{
  "code": 0,
  "message": "上传成功",
  "data": {
    "filename": "1716888888888-123456789.png",
    "originalname": "test.png",
    "mimetype": "image/png",
    "size": 12345,
    "url": "/uploads/1716888888888-123456789.png"
  }
}
```

浏览器访问：

```
http://localhost:3000/uploads/1716888888888-123456789.png
```

就可以看到图片。

八、上传头像案例

接口：

```
POST /api/upload/avatar
```

字段名：

```
avatar
```

代码：

```
app.post("/api/upload/avatar", upload.single("avatar"), (req, res) => {
  if (!req.file) {
    return res.status(400).json({
      code: 400,
      message: "请选择要上传的文件"
    })
  }

  res.json({
    code: 0,
    message: "头像上传成功",
    data: {
      url: `/uploads/${req.file.filename}`
    }
  })
})
```

测试：

```
curl -X POST http://localhost:3000/api/upload/avatar \
-F "avatar=@./avatar.png"
```

注意：这里字段名是 `avatar`，所以必须对应：

```
upload.single("avatar")
```

九、同时上传文本字段和文件

前端可能同时提交：

- 商品名称
- 商品价格
- 商品图片

接口：

```
POST /api/products
```

代码：

```
app.post("/api/products", upload.single("image"), (req, res) => {
  console.log("文本字段: ", req.body)
  console.log("文件信息: ", req.file)

  const { name, price } = req.body

  res.json({
    code: 0,
    message: "商品创建成功",
    data: {
      name,
      price,
      image: `/uploads/${req.file.filename}`
    }
  })
})
```

测试：

```
curl -X POST http://localhost:3000/api/products \
-F "name=苹果" \
-F "price=5" \
-F "image=@./apple.png"
```

说明：

- 普通字段会出现在 `req.body`
- 文件字段会出现在 `req.file`

十、多文件上传 array

如果要一次上传多张图片，可以使用：

```
upload.array("images", 5)
```

表示字段名是 `images`，最多上传 5 个文件。

```
app.post("/api/upload/images", upload.array("images", 5), (req, res) => {
  res.json({
    code: 0,
    message: "多文件上传成功",
    data: req.files.map(file => ({
      originalname: file.originalname,
      filename: file.filename,
      size: file.size,
      url: `/uploads/${file.filename}`
    }))
  })
})
```

测试：

```
curl -X POST http://localhost:3000/api/upload/images \
-F "images=@./a.png" \
-F "images=@./b.png"
```

多文件上传后，文件信息在：

```
req.files
```

十一、多个不同字段上传 fields

比如：

- 头像 avatar：1 个
- 背景图 background：1 个
- 相册 photos：最多 5 个

代码：

```

app.post(
  "/api/upload/profile",
  upload.fields([
    { name: "avatar", maxCount: 1 },
    { name: "background", maxCount: 1 },
    { name: "photos", maxCount: 5 }
  ]),
  (req, res) => {
    res.json({
      code: 0,
      message: "上传成功",
      files: req.files
    })
  }
)

```

测试：

```

curl -X POST http://localhost:3000/api/upload/profile \
-F "avatar=@./avatar.png" \
-F "background=@./bg.jpg" \
-F "photos=@./p1.png" \
-F "photos=@./p2.png"

```

req.files 大概是：

```

{
  avatar: [file],
  background: [file],
  photos: [file, file]
}

```

十二、限制上传文件大小

可以通过 `limits` 限制大小。

比如限制单个文件最大 2MB：

```

const upload = multer({
  storage,
  limits: {
    fileSize: 2 * 1024 * 1024
  }
})

```

完整：

```

const upload = multer({
  storage,

```



```
limits: {
  fileSize: 2 * 1024 * 1024
}
})
```

如果超过大小，会报错，需要错误处理中间件处理。

十三、限制上传文件类型

可以使用 `fileFilter`。

只允许图片：

```
const upload = multer({
  storage,
  limits: {
    fileSize: 2 * 1024 * 1024
  },
  fileFilter(req, file, cb) {
    const allowedTypes = ["image/jpeg", "image/png", "image/gif", "image/webp"]

    if (allowedTypes.includes(file.mimetype)) {
      cb(null, true)
    } else {
      cb(new Error("只允许上传图片文件"))
    }
  }
})
```

注意：`mimetype` 是客户端传来的，不能作为绝对安全依据。真实项目还需要更严格的文件内容检测。

十四、处理 multer 上传错误

比如：

- 文件太大
- 文件类型不允许
- 字段名错误
- 超过文件数量

可以写错误处理中间件：

```
app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    if (err.code === "LIMIT_FILE_SIZE") {
      return res.status(400).json({
        code: 400,
        message: "文件不能超过 2MB"
      })
    }
  }
  next(err)
})
```

```

    })
  }

  return res.status(400).json({
    code: 400,
    message: err.message
  })
}

if (err) {
  return res.status(400).json({
    code: 400,
    message: err.message
  })
}

next()
})

```

注意：错误处理中间件要放在路由后面。

十五、完整图片上传示例

项目结构：

```

upload-demo
├─ app.js
└─ uploads

```

app.js：

```

const express = require("express")
const multer = require("multer")
const path = require("path")
const fs = require("fs")

const app = express()

const uploadDir = path.join(__dirname, "uploads")

if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir)
}

app.use("/uploads", express.static(uploadDir))

const storage = multer.diskStorage({
  destination(req, file, cb) {
    cb(null, uploadDir)
  },
  filename(req, file, cb) {
    const ext = path.extname(file.originalname).toLowerCase()

```

```

    const filename = `${Date.now()}-${Math.round(Math.random() * 1e9)}${ext}`
    cb(null, filename)
  }
})

const upload = multer({
  storage,
  limits: {
    fileSize: 2 * 1024 * 1024
  },
  fileFilter(req, file, cb) {
    const allowedTypes = ["image/jpeg", "image/png", "image/gif", "image/webp"]

    if (allowedTypes.includes(file.mimetype)) {
      cb(null, true)
    } else {
      cb(new Error("只允许上传 jpg、png、gif、webp 图片"))
    }
  }
})

app.get("/", (req, res) => {
  res.send(`
    <h1>文件上传测试</h1>
    <form action="/api/upload/avatar" method="post" enctype="multipart/form-data">
      <input type="file" name="avatar" />
      <button type="submit">上传头像</button>
    </form>
  `)
})

app.post("/api/upload/avatar", upload.single("avatar"), (req, res) => {
  if (!req.file) {
    return res.status(400).json({
      code: 400,
      message: "请选择文件"
    })
  }

  res.json({
    code: 0,
    message: "上传成功",
    data: {
      originalname: req.file.originalname,
      filename: req.file.filename,
      mimetype: req.file.mimetype,
      size: req.file.size,
      url: `/uploads/${req.file.filename}`
    }
  })
})

app.post("/api/upload/images", upload.array("images", 5), (req, res) => {
  res.json({
    code: 0,
    message: "多图上传成功",
    data: req.files.map(file => ({

```

```

        originalname: file.originalname,
        filename: file.filename,
        size: file.size,
        url: `/uploads/${file.filename}`
    )))
  })
})

app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError) {
    if (err.code === "LIMIT_FILE_SIZE") {
      return res.status(400).json({
        code: 400,
        message: "文件不能超过 2MB"
      })
    }

    if (err.code === "LIMIT_UNEXPECTED_FILE") {
      return res.status(400).json({
        code: 400,
        message: "上传字段名错误或文件数量超出限制"
      })
    }

    return res.status(400).json({
      code: 400,
      message: err.message
    })
  }

  if (err) {
    return res.status(400).json({
      code: 400,
      message: err.message
    })
  }

  next()
})

app.use((req, res) => {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})

```

运行：

```
npm run dev
```

浏览器访问：

```
http://localhost:3000
```

即可测试上传。

十六、前端 FormData 上传示例

浏览器前端代码：

```
<input type="file" id="file">
<button id="btn">上传</button>

<script>
  const fileInput = document.getElementById("file")
  const btn = document.getElementById("btn")

  btn.onclick = async function () {
    const file = fileInput.files[0]

    if (!file) {
      alert("请选择文件")
      return
    }

    const formData = new FormData()
    formData.append("avatar", file)

    const res = await fetch("http://localhost:3000/api/upload/avatar", {
      method: "POST",
      body: formData
    })

    const data = await res.json()
    console.log(data)
  }
</script>
```

注意：

使用 FormData 时，通常不要手动设置：

```
Content-Type: multipart/form-data
```

浏览器会自动设置，并带上 boundary。

错误示例：

```
headers: {
  "Content-Type": "multipart/form-data"
```

```
}
```

十七、axios 上传示例

```
const formData = new FormData()

formData.append("avatar", file)

axios.post("http://localhost:3000/api/upload/avatar", formData)
  .then(res => {
    console.log(res.data)
  })
```

如果要显示上传进度：

```
axios.post("http://localhost:3000/api/upload/avatar", formData, {
  onUploadProgress(event) {
    const percent = Math.round((event.loaded * 100) / event.total)
    console.log("上传进度: ", percent + "%")
  }
})
```

十八、上传后保存到数据库

真实项目中，上传文件后通常会把文件地址保存到数据库。

例如用户头像：

```
app.post("/api/users/:id/avatar", upload.single("avatar"), async (req, res) => {
  const userId = req.params.id
  const avatarUrl = `/uploads/${req.file.filename}`

  // 示例: MySQL
  // await pool.query("UPDATE users SET avatar = ? WHERE id = ?", [avatarUrl, userId])

  // 示例: MongoDB
  // await User.findByIdAndUpdate(userId, { avatar: avatarUrl })

  res.json({
    code: 0,
    message: "头像更新成功",
    data: {
      userId,
      avatar: avatarUrl
    }
  })
})
```

数据库中保存的通常是：

```
/uploads/xxx.png
```

或者完整 URL：

```
http://localhost:3000/uploads/xxx.png
```

十九、结合 JWT：只有登录用户能上传头像

如果前面已经写过 `auth` 中间件，可以这样：

```
app.post("/api/user/avatar", auth, upload.single("avatar"), async (req, res) => {
  const userId = req.user.id
  const avatarUrl = `/uploads/${req.file.filename}`

  res.json({
    code: 0,
    message: "头像上传成功",
    data: {
      userId,
      avatar: avatarUrl
    }
  })
})
```

请求时：

```
curl -X POST http://localhost:3000/api/user/avatar \
-H "Authorization: Bearer 你的token" \
-F "avatar=@./avatar.png"
```

中间件执行顺序：

```
auth -> upload.single("avatar") -> 路由处理函数
```

二十、文件上传安全注意事项

文件上传是一个高风险功能，真实项目一定要注意。

1. 限制文件大小

```
limits: {
  fileSize: 2 * 1024 * 1024
```

```
}
```

2. 限制文件类型

```
fileFilter(req, file, cb) {  
  const allowedTypes = ["image/jpeg", "image/png"]  
  cb(null, allowedTypes.includes(file.mimetype))  
}
```

3. 不要使用用户原始文件名直接保存

不推荐：

```
cb(null, file.originalname)
```

可能导致：

- 文件名冲突
- 路径风险
- 特殊字符问题

推荐：

```
Date.now() + "-" + random + ext
```

4. 不要把上传目录设置为代码目录

推荐单独目录：

```
uploads
```

不要让用户覆盖服务器代码文件。

5. 生产环境建议使用对象存储

例如：

- 阿里云 OSS
- 腾讯云 COS
- 七牛云

- AWS S3

真实线上项目中，文件通常不直接存在 Node 服务器本地，而是上传到对象存储。

二十一、常见错误

1. req.file 是 undefined

原因通常是字段名不一致。

后端：

```
upload.single("avatar")
```

前端必须：

```
formData.append("avatar", file)
```

或者 curl：

```
-F "avatar=@./avatar.png"
```

2. 表单缺少 enctype

HTML 表单上传文件必须写：

```
<form enctype="multipart/form-data">
```

完整：

```
<form action="/api/upload/avatar" method="post" enctype="multipart/form-data">
```

3. 前端手动设置了错误的 Content-Type

使用 FormData 时，不要手动设置：

```
"Content-Type": "multipart/form-data"
```

让浏览器自动设置。

4. 上传目录不存在

如果目录不存在，可能报错：

```
ENOENT: no such file or directory
```

解决：

```
if (!fs.existsSync(uploadDir)) {  
  fs.mkdirSync(uploadDir)  
}
```

5. 文件太大报错但没有返回友好信息

需要添加错误处理中间件：

```
app.use((err, req, res, next) => {  
  if (err instanceof multer.MulterError) {  
    return res.status(400).json({  
      message: err.message  
    })  
  }  
  
  next(err)  
})
```

本课重点

这一课你需要掌握：

1. 文件上传通常使用 `multipart/form-data`
2. Express 上传文件常用 `multer`
3. 安装：

```
npm install multer
```

4. 单文件上传：

```
upload.single("avatar")
```

5. 多文件上传：

```
upload.array("images", 5)
```

6. 多字段上传：

```
upload.fields([
  { name: "avatar", maxCount: 1 }
])
```

7. 文件信息在：

```
req.file
```

或：

```
req.files
```

8. 普通文本字段在：

```
req.body
```

- 9. 使用 `multer.diskStorage()` 自定义文件名
- 10. 使用 `express.static()` 让上传文件可访问
- 11. 使用 `limits.fileSize` 限制大小
- 12. 使用 `fileFilter` 限制类型
- 13. 真实项目中要注意上传安全

课后练习

实现一个商品图片上传接口。

要求：

方法	路径	功能
POST	/api/products	创建商品，包含商品名、价格、图片
POST	/api/upload/images	多图上传，最多 5 张
GET	/uploads/文件名	访问上传后的图片

商品创建字段：

```
name
price
image
```

curl 测试：

```
curl -X POST http://localhost:3000/api/products \
-F "name=苹果" \
-F "price=5" \
-F "image=@./apple.png"
```

返回示例：

```
{
  "code": 0,
  "message": "商品创建成功",
  "data": {
    "name": "苹果",
    "price": 5,
    "image": "/uploads/xxx.png"
  }
}
```

下一课建议学习：

Node.js 第十一课：项目部署 PM2 / Nginx / Linux。

Node.js 第十一课：项目部署 PM2 / Nginx / Linux

这一课学习如何把本地写好的 Node.js / Express 项目部署到服务器上，让别人可以通过公网访问。

本课内容：

1. Linux 服务器基础
2. 安装 Node.js
3. 上传项目到服务器
4. 配置 .env
5. 使用 PM2 启动 Node.js 服务
6. 使用 Nginx 做反向代理
7. 配置防火墙和端口
8. 项目更新与重启
9. 常见部署错误

一、部署整体流程

一个 Node.js 项目上线，大致流程是：

```
购买服务器
↓
登录 Linux 服务器
↓
安装 Node.js / npm
```

↓
上传项目代码
↓
安装依赖 `npm install`
↓
配置环境变量 `.env`
↓
用 `PM2` 启动项目
↓
配置 `Nginx` 反向代理
↓
绑定域名
↓
配置 `HTTPS`

简单理解：

用户浏览器 -> `Nginx` -> `Node.js Express` 服务

例如：

用户访问：`http://你的域名.com`
`Nginx` 接收 `80` 端口请求
`Nginx` 转发给本机 `Node.js` 的 `3000` 端口
`Express` 返回接口数据
`Nginx` 再返回给用户

二、为什么需要 PM2?

如果你直接运行：

```
node app.js
```

关闭终端后，程序可能就停止了。

而且如果程序报错崩溃，也不会自动重启。

所以生产环境通常使用 `PM2`。

`PM2` 的作用：

- 后台运行 `Node.js` 项目
 - 程序崩溃自动重启
 - 查看日志
 - 查看运行状态
 - 支持开机自启
 - 支持多进程模式
-

三、为什么需要 Nginx?

Node.js 本身可以监听端口，比如：

```
app.listen(3000)
```

但是实际部署中通常不会直接让用户访问：

```
http://服务器IP:3000
```

而是让用户访问：

```
http://服务器IP  
http://你的域名.com  
https://你的域名.com
```

这就需要 Nginx。

Nginx 的作用：

- 监听 80 / 443 端口
- 反向代理到 Node.js 服务
- 配置域名
- 配置 HTTPS
- 托管静态资源
- 负载均衡
- 限流
- gzip 压缩

四、准备一个可部署的 Express 项目

假设项目结构：

```
my-node-app  
├─ app.js  
├─ package.json  
├─ package-lock.json  
├─ .env  
├─ routes  
└─ uploads
```

app.js 示例：

```
require("dotenv").config()  
  
const express = require("express")
```

```
const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: false }))

app.get("/", (req, res) => {
  res.send("Node.js 项目部署成功")
})

app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok",
    time: new Date().toISOString()
  })
})

const PORT = process.env.PORT || 3000

app.listen(PORT, () => {
  console.log(`服务器启动: http://localhost:${PORT}`)
})
```

.env :

```
PORT=3000
NODE_ENV=production
JWT_SECRET=your_secret_key
```

package.json :

```
{
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "dependencies": {
    "dotenv": "^16.0.0",
    "express": "^4.18.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}
```

本地测试:

```
npm install
npm start
```

访问:

```
http://localhost:3000
```

五、登录 Linux 服务器

通常使用 SSH 登录。

假设服务器 IP 是：

```
1.2.3.4
```

登录：

```
ssh root@1.2.3.4
```

如果是普通用户：

```
ssh ubuntu@1.2.3.4
```

第一次登录可能提示：

```
Are you sure you want to continue connecting?
```

输入：

```
yes
```

然后输入服务器密码。

六、Linux 常用命令

部署时常用命令：

```
pwd
```

查看当前目录。

```
ls
```

查看文件。

```
ls -la
```


查看隐藏文件。

```
cd /path
```

进入目录。

```
mkdir app
```

创建目录。

```
rm -rf node_modules
```

删除目录。

```
cp a.txt b.txt
```

复制文件。

```
mv a.txt b.txt
```

移动或重命名。

```
cat app.log
```

查看文件内容。

```
tail -f app.log
```

实时查看日志。

```
nano .env
```

编辑文件。

```
vim .env
```

编辑文件，适合熟悉 Vim 的用户。

七、安装 Node.js

推荐使用 `nvm` 管理 Node.js 版本。

1. 安装 nvm

执行：

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

如果没有 `curl`，先安装：

Ubuntu / Debian：

```
apt update  
apt install curl -y
```

CentOS：

```
yum install curl -y
```

安装后让配置生效：

```
source ~/.bashrc
```

或者重新登录服务器。

检查：

```
nvm -v
```

2. 安装 Node.js LTS

```
nvm install --lts
```

使用 LTS 版本：

```
nvm use --lts
```

检查版本：

```
node -v  
npm -v
```

八、上传项目到服务器

常见方式有三种：

1. Git 拉取代码
2. scp 上传
3. FTP/SFTP 工具上传

推荐使用 Git。

方式一：Git 拉取项目

服务器安装 Git：

Ubuntu / Debian：

```
apt update
apt install git -y
```

CentOS：

```
yum install git -y
```

进入部署目录：

```
mkdir -p /var/www
cd /var/www
```

克隆项目：

```
git clone https://github.com/你的用户名/你的项目.git my-node-app
```

进入项目：

```
cd my-node-app
```

安装依赖：

```
npm install
```

如果是生产环境，只安装生产依赖：

```
npm install --omit=dev
```

方式二：scp 上传项目

在你本地电脑执行：

```
scp -r ./my-node-app root@1.2.3.4:/var/www/
```

然后登录服务器：

```
ssh root@1.2.3.4
cd /var/www/my-node-app
npm install
```

注意：

不要上传：

```
node_modules
```

可以在本地 `.gitignore` 中加入：

```
node_modules
.env
```

九、配置 `.env`

如果 `.env` 没有上传，需要在服务器创建：

```
cd /var/www/my-node-app
nano .env
```

写入：

```
PORT=3000
NODE_ENV=production
JWT_SECRET=your_production_secret
```

如果有数据库：

```
MYSQL_HOST=127.0.0.1
MYSQL_USER=root
MYSQL_PASSWORD=你的数据库密码
MYSQL_DATABASE=node_demo

MONGO_URL=mongodb://127.0.0.1:27017/node_demo
```

保存后退出。

`nano` 保存方式：

```
Ctrl + O  
回车  
Ctrl + X
```

十、安装 PM2

全局安装：

```
npm install pm2 -g
```

检查：

```
pm2 -v
```

十一、用 PM2 启动项目

进入项目目录：

```
cd /var/www/my-node-app
```

启动：

```
pm2 start app.js --name my-node-app
```

查看状态：

```
pm2 list
```

你会看到类似：

id	name	mode	status	cpu	memory
0	my-node-app	fork	online	0%	50mb

访问服务器本机测试：

```
curl http://127.0.0.1:3000
```

如果返回：

```
Node.js 项目部署成功
```

说明 Node 服务已经运行。

十二、PM2 常用命令

查看应用列表

```
pm2 list
```

查看日志

```
pm2 logs
```

查看指定应用日志：

```
pm2 logs my-node-app
```

重启应用

```
pm2 restart my-node-app
```

停止应用

```
pm2 stop my-node-app
```

删除应用

```
pm2 delete my-node-app
```

查看详细信息

```
pm2 show my-node-app
```

重新加载，尽量不中断服务

```
pm2 reload my-node-app
```

十三、配置 PM2 开机自启

如果服务器重启，默认 PM2 进程可能不会自动恢复。

执行：

```
pm2 startup
```

它会输出一段命令，例如：

```
sudo env PATH=$PATH:/root/.nvm/versions/node/v20.x.x/bin pm2 startup systemd -u root --hp /root
```

复制它输出的命令并执行。

然后保存当前进程列表：

```
pm2 save
```

以后服务器重启后，PM2 会自动恢复项目。

十四、使用 ecosystem.config.js 管理 PM2

项目根目录创建：

```
ecosystem.config.js
```

内容：

```
module.exports = {
  apps: [
    {
      name: "my-node-app",
      script: "app.js",
      instances: 1,
      autorestart: true,
      watch: false,
      max_memory_restart: "300M",
      env: {
```

```
    NODE_ENV: "production",  
    PORT: 3000  
  }  
}  
]  
}
```

启动：

```
pm2 start ecosystem.config.js
```

重启：

```
pm2 restart ecosystem.config.js
```

保存：

```
pm2 save
```

如果想使用多进程模式：

```
instances: "max",  
exec_mode: "cluster"
```

示例：

```
module.exports = {  
  apps: [  
    {  
      name: "my-node-app",  
      script: "app.js",  
      instances: "max",  
      exec_mode: "cluster",  
      autorestart: true,  
      watch: false,  
      max_memory_restart: "300M",  
      env: {  
        NODE_ENV: "production",  
        PORT: 3000  
      }  
    }  
  ]  
}
```

不过初学阶段用：

```
instances: 1
```

即可。

十五、安装 Nginx

Ubuntu / Debian

```
apt update  
apt install nginx -y
```

启动 Nginx：

```
systemctl start nginx
```

设置开机自启：

```
systemctl enable nginx
```

查看状态：

```
systemctl status nginx
```

CentOS

```
yum install nginx -y  
systemctl start nginx  
systemctl enable nginx  
systemctl status nginx
```

安装成功后，浏览器访问：

```
http://服务器IP
```

如果看到 Nginx 默认欢迎页，说明 Nginx 正常。

十六、配置 Nginx 反向代理

假设 Node.js 服务运行在：

```
127.0.0.1:3000
```

我们希望用户访问：

```
http://服务器IP
```

自动转发到：

```
http://127.0.0.1:3000
```

1. 新建 Nginx 配置

Ubuntu / Debian 通常使用：

```
nano /etc/nginx/sites-available/my-node-app
```

写入：

```
server {  
    listen 80;  
    server_name 你的服务器IP或域名;  
  
    location / {  
        proxy_pass http://127.0.0.1:3000;  
  
        proxy_http_version 1.1;  
  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection "upgrade";  
    }  
}
```

如果暂时没有域名，可以写服务器 IP：

```
server_name 1.2.3.4;
```

如果有域名：

```
server_name example.com www.example.com;
```

2. 启用配置

创建软链接：

```
ln -s /etc/nginx/sites-available/my-node-app /etc/nginx/sites-enabled/
```

检查 Nginx 配置是否正确：

```
nginx -t
```

如果显示：

```
syntax is ok
test is successful
```

说明配置正确。

重载 Nginx：

```
systemctl reload nginx
```

浏览器访问：

```
http://服务器IP
```

应该能看到：

```
Node.js 项目部署成功
```

3. CentOS 常见配置位置

CentOS 可能没有 `sites-available` 目录，可以直接创建：

```
nano /etc/nginx/conf.d/my-node-app.conf
```

内容同上：

```
server {
    listen 80;
    server_name 你的服务器IP或域名;

    location / {
        proxy_pass http://127.0.0.1:3000;

        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
}  
}
```

检查并重载：

```
nginx -t  
systemctl reload nginx
```

十七、配置防火墙和安全组

部署项目时常见问题：

本机 `curl 127.0.0.1:3000` 可以
但是浏览器访问服务器 IP 不行

可能是：

1. 云服务器安全组没开放端口
2. Linux 防火墙没开放端口
3. Nginx 没启动
4. Nginx 配置错误

1. 云服务器安全组

如果使用阿里云、腾讯云、华为云等，需要在控制台开放：

端口	用途
22	SSH 登录
80	HTTP
443	HTTPS
3000	Node.js 端口，可选

如果使用 Nginx 代理，公网一般只需要开放：

80
443
22

不建议把 3000 直接暴露到公网。

2. Ubuntu UFW 防火墙

查看状态：

```
ufw status
```

开放 80 和 443：

```
ufw allow 80
ufw allow 443
```

开放 SSH：

```
ufw allow 22
```

启用：

```
ufw enable
```

十八、绑定域名

如果你有域名，例如：

```
example.com
```

需要去域名 DNS 控制台添加解析：

```
类型：A
主机记录：@
记录值：服务器公网 IP
```

如果要支持：

```
www.example.com
```

再添加：

```
类型：A
主机记录：www
记录值：服务器公网 IP
```

然后 Nginx 配置：

```
server {
    listen 80;
    server_name example.com www.example.com;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

重载：

```
nginx -t
systemctl reload nginx
```

十九、配置 HTTPS

HTTPS 常用免费证书工具：

```
certbot
```

Ubuntu / Debian 安装：

```
apt install certbot python3-certbot-nginx -y
```

申请证书：

```
certbot --nginx -d example.com -d www.example.com
```

按提示操作即可。

成功后访问：

```
https://example.com
```

查看自动续期：

```
certbot renew --dry-run
```

二十、部署静态资源和上传文件

如果项目中有上传文件目录：

```
uploads
```

Express 中通常有：

```
app.use("/uploads", express.static(path.join(__dirname, "uploads")))
```

部署时要注意：

1. 服务器上要有 uploads 目录
2. Node.js 进程对该目录有写入权限
3. 不要在每次更新代码时删除上传目录

创建目录：

```
mkdir -p /var/www/my-node-app/uploads
```

如果权限不足：

```
chmod 755 /var/www/my-node-app/uploads
```

如果使用普通用户运行项目，可能需要修改所有者：

```
chown -R ubuntu:ubuntu /var/www/my-node-app/uploads
```

二十一、项目更新上线流程

如果你用 Git 部署，更新项目通常这样：

```
cd /var/www/my-node-app
git pull
npm install --omit=dev
pm2 restart my-node-app
```

如果修改了 Nginx 配置：

```
nginx -t
systemctl reload nginx
```

如果修改了 .env：

```
pm2 restart my-node-app
```

二十二、部署脚本 deploy.sh

可以写一个简单脚本。

项目根目录创建：

```
nano deploy.sh
```

内容：

```
#!/bin/bash

echo "开始部署..."

cd /var/www/my-node-app || exit

echo "拉取最新代码..."
git pull

echo "安装依赖..."
npm install --omit=dev

echo "重启 PM2..."
pm2 restart my-node-app

echo "部署完成"
```

赋予执行权限：

```
chmod +x deploy.sh
```

运行：

```
./deploy.sh
```

二十三、Nginx 配置静态前端项目

如果你有一个前端项目，比如 Vue / React 打包后生成：

```
dist
```

可以让 Nginx 直接托管前端，接口代理给 Node.js。

假设：

```
/var/www/frontend/dist
/var/www/my-node-app
```


Nginx 配置：

```
server {  
    listen 80;  
    server_name example.com;  
  
    root /var/www/frontend/dist;  
    index index.html;  
  
    location / {  
        try_files $uri $uri/ /index.html;  
    }  
  
    location /api/ {  
        proxy_pass http://127.0.0.1:3000/api/;  
  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

说明：

```
location / {  
    try_files $uri $uri/ /index.html;  
}
```

这是为 Vue Router / React Router 的 history 模式准备的。

```
location /api/ {  
    proxy_pass http://127.0.0.1:3000/api/;  
}
```

表示接口请求转发给 Node.js。

二十四、Node.js 获取真实客户端 IP

如果用了 Nginx 代理，Express 中直接获取：

```
req.ip
```

可能得到：

```
127.0.0.1
```

因为请求是从 Nginx 转发来的。

可以在 Express 中设置：

```
app.set("trust proxy", true)
```

然后：

```
app.get("/ip", (req, res) => {  
  res.json({  
    ip: req.ip,  
    ips: req.ips  
  })  
})
```

Nginx 配置中也要有：

```
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

二十五、生产环境建议

1. 不要用 nodemon 跑生产环境

开发环境：

```
npm run dev
```

生产环境：

```
pm2 start app.js --name my-node-app
```

不要在服务器上长期使用：

```
nodemon app.js
```

2. 不要把数据库密码写死在代码里

错误：

```
password: "123456"
```

推荐：

```
password: process.env.MYSQL_PASSWORD
```

3. 不要提交 .env

.gitignore :

```
node_modules
.env
uploads
```

注意：如果上传文件目录里有真实用户文件，一般也不要提交到 Git。

4. 生产环境关闭详细错误

开发时可以返回：

```
error: err.message
```

生产环境不要把数据库错误、堆栈信息直接返回给用户。

推荐：

```
app.use((err, req, res, next) => {
  console.error(err)

  res.status(500).json({
    code: 500,
    message: "服务器内部错误"
  })
})
```

5. 设置 NODE_ENV

.env :

```
NODE_ENV=production
```

代码中可以判断：

```
if (process.env.NODE_ENV !== "production") {
  console.log(err)
}
```

二十六、常见错误排查

1. PM2 显示 online，但网页打不开

检查 Node 服务：

```
curl http://127.0.0.1:3000
```

如果本机 curl 不通，说明 Node 服务有问题。

看日志：

```
pm2 logs my-node-app
```

2. Node 服务正常，但公网打不开

检查 Nginx：

```
systemctl status nginx
```

检查配置：

```
nginx -t
```

检查 80 端口是否开放：

```
ufw status
```

检查云服务器安全组是否开放 80。

3. 端口被占用

错误：

```
EADDRINUSE: address already in use :::3000
```

说明 3000 端口已经被占用。

查看进程：

```
lsof -i:3000
```

如果没有 `lsyf`：

```
apt install lsyf -y
```

杀掉进程：

```
kill -9 进程ID
```

或者修改 `.env`：

```
PORT=3001
```

然后重启：

```
pm2 restart my-node-app
```

4. Cannot find module

错误：

```
Cannot find module 'express'
```

说明依赖没安装。

解决：

```
cd /var/www/my-node-app
npm install
pm2 restart my-node-app
```

5. .env 没生效

检查入口文件最上方是否写了：

```
require("dotenv").config()
```

必须在使用 `process.env` 前执行。

6. Nginx 502 Bad Gateway

502 通常表示 Nginx 找不到后端服务。

检查：

```
pm2 list
curl http://127.0.0.1:3000
```

如果 Node 没启动：

```
pm2 start app.js --name my-node-app
```

如果 Node 端口不是 3000，需要修改 Nginx：

```
proxy_pass http://127.0.0.1:你的端口;
```

然后：

```
nginx -t
systemctl reload nginx
```

7. 413 Request Entity Too Large

上传文件时报错：

```
413 Request Entity Too Large
```

说明 Nginx 限制了上传大小。

在 Nginx server 中添加：

```
client_max_body_size 20M;
```

示例：

```
server {
    listen 80;
    server_name example.com;

    client_max_body_size 20M;

    location / {
        proxy_pass http://127.0.0.1:3000;
    }
}
```

重载：

```
nginx -t
systemctl reload nginx
```

二十七、完整部署示例总结

假设：

```
服务器 IP: 1.2.3.4
项目目录: /var/www/my-node-app
Node 端口: 3000
PM2 名称: my-node-app
```

1. 安装环境

```
apt update
apt install git curl nginx -y
```

安装 Node.js：

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install --lts
```

安装 PM2：

```
npm install pm2 -g
```

2. 拉取项目

```
mkdir -p /var/www
cd /var/www
git clone https://github.com/你的用户名/你的项目.git my-node-app
cd my-node-app
npm install --omit=dev
```

3. 配置环境变量

```
nano .env
```

```
PORT=3000
NODE_ENV=production
JWT_SECRET=your_secret
```

4. PM2 启动

```
pm2 start app.js --name my-node-app
pm2 save
pm2 startup
```

测试:

```
curl http://127.0.0.1:3000
```

5. Nginx 配置

```
nano /etc/nginx/sites-available/my-node-app
```

写入:

```
server {
    listen 80;
    server_name 1.2.3.4;

    location / {
        proxy_pass http://127.0.0.1:3000;

        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

启用:

```
ln -s /etc/nginx/sites-available/my-node-app /etc/nginx/sites-enabled/
nginx -t
systemctl reload nginx
```

访问:

http://1.2.3.4

本课重点

这一课你需要掌握：

1. 生产环境不要直接用 `node app.js`
2. PM2 用来管理 Node.js 后台进程
3. 常用启动命令：

```
pm2 start app.js --name my-node-app
```

4. 查看状态：

```
pm2 list
```

5. 查看日志：

```
pm2 logs
```

6. 重启项目：

```
pm2 restart my-node-app
```

7. Nginx 用来做反向代理
8. 用户访问 80 / 443, Nginx 转发给 Node.js 的 3000 端口
9. `.env` 保存生产环境配置
10. 云服务器安全组要开放 80、443、22
11. 修改代码后通常执行：

```
git pull
npm install --omit=dev
pm2 restart my-node-app
```

课后练习

把你之前写的 Express 项目部署到 Linux 服务器上，要求：

1. 项目监听端口使用 `.env` 配置
2. 使用 PM2 启动项目
3. 使用 Nginx 代理到 Node.js 服务

4. 浏览器访问服务器 IP 可以看到首页
5. `/api/health` 返回 JSON:

```
{
  "code": 0,
  "message": "ok"
}
```

6. 配置 PM2 开机自启
7. 写一个 `deploy.sh` 脚本用于更新部署

下一课建议学习:

Node.js 第十二课: 项目结构、MVC 分层与实战项目搭建。

Node.js 第十二课: 项目结构、MVC 分层与实战项目搭建

这一课我们学习真实项目中非常重要的内容:

如何组织一个可维护的 Node.js / Express 后端项目。

之前我们写代码时, 很多逻辑都放在 `app.js` 或路由文件里。小项目没问题, 但真实项目会越来越复杂, 比如:

- 用户注册登录
- 商品管理
- 订单管理
- 文件上传
- JWT 鉴权
- 数据库操作
- 参数校验
- 错误处理
- 日志记录
- 权限控制

如果全部写在一个文件里, 后期会非常难维护。

所以我们需要学习 **项目结构设计** 和 **MVC 分层思想**。

一、什么是 MVC?

MVC 是一种常见的软件分层思想。

MVC 分为三层:

层	全称	作用
M	Model	数据模型，负责数据库操作
V	View	视图，负责页面展示
C	Controller	控制器，负责业务逻辑

在前后端分离项目中，Node.js 后端通常不直接负责页面展示，所以 View 层会弱化。

我们更常见的是这种结构：

```
路由 routes
控制器 controllers
模型 models
中间件 middlewares
配置 config
工具 utils
服务 services
```

二、为什么要分层？

如果不分层，代码可能是这样：

```
app.post("/api/auth/register", async (req, res) => {
  const { username, password } = req.body

  if (!username || !password) {
    return res.status(400).json({
      message: "用户名和密码不能为空"
    })
  }

  const existUser = await User.findOne({ username })

  if (existUser) {
    return res.status(409).json({
      message: "用户名已存在"
    })
  }

  const hashedPassword = await bcrypt.hash(password, 10)

  const user = await User.create({
    username,
    password: hashedPassword
  })

  res.json({
    message: "注册成功",
    data: user
  })
})
```

```
  })  
})
```

功能能跑，但问题是：

- 路由太长
- 参数校验、业务逻辑、数据库操作混在一起
- 不方便复用
- 不方便测试
- 后期维护困难

分层后会变成：

```
router.post("/register", authController.register)
```

业务逻辑放到 controller：

```
exports.register = async (req, res) => {  
  // 注册逻辑  
}
```

这样结构更清晰。

三、推荐项目目录结构

我们搭建一个比较标准的 Express 项目结构：

```
node-mvc-demo  
├─ app.js  
├─ server.js  
├─ package.json  
├─ .env  
├─ .gitignore  
├─ config  
│   └─ db.js  
├─ controllers  
│   ├── authController.js  
│   └─ userController.js  
├─ models  
│   └─ User.js  
├─ routes  
│   ├── authRoutes.js  
│   └─ userRoutes.js  
├─ middlewares  
│   ├── authMiddleware.js  
│   ├── errorMiddleware.js  
│   └─ notFoundMiddleware.js  
├─ utils  
│   ├── asyncHandler.js  
│   └─ generateToken.js
```

```
| └─ response.js
└─ uploads
```

四、各目录作用说明

1. app.js

负责创建 Express 应用，注册中间件和路由。

```
const express = require("express")

const app = express()

app.use(express.json())

module.exports = app
```

2. server.js

负责启动服务、连接数据库。

```
require("dotenv").config()

const app = require("./app")
const connectDB = require("./config/db")

connectDB()

const PORT = process.env.PORT || 3000

app.listen(PORT, () => {
  console.log(`服务器启动: http://localhost:${PORT}`)
})
```

3. config

放配置文件，比如数据库连接。

```
config/db.js
```

4. models

放数据库模型。

```
models/User.js
```

5. routes

放路由文件。

```
routes/authRoutes.js  
routes/userRoutes.js
```

6. controllers

放控制器，处理具体业务逻辑。

```
controllers/authController.js  
controllers/userController.js
```

7. middlewares

放中间件。

例如：

- 登录鉴权
 - 错误处理
 - 404 处理
 - 文件上传
 - 权限校验
-

8. utils

放工具函数。

例如：

- 生成 Token
- 统一响应格式
- 异步错误捕获
- 密码加密工具

五、创建项目

```
mkdir node-mvc-demo
cd node-mvc-demo
npm init -y
```

安装依赖：

```
npm install express mongoose dotenv bcryptjs jsonwebtoken cors
npm install nodemon -D
```

修改 package.json：

```
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}
```

六、创建基础目录

```
mkdir config controllers models routes middlewares utils uploads
touch server.js app.js .env .gitignore
```

Linux / macOS 可以这样：

```
touch config/db.js
touch models/User.js
touch routes/authRoutes.js routes/userRoutes.js
touch controllers/authController.js controllers/userController.js
touch middlewares/authMiddleware.js middlewares/errorMiddleware.js
middlewares/notFoundMiddleware.js
touch utils/asyncHandler.js utils/generateToken.js utils/response.js
```

Windows 可以手动创建这些文件。

七、配置 .env

.env：

```
PORT=3000
NODE_ENV=development
```

```
MONGO_URL=mongodb://127.0.0.1:27017/node_mvc_demo
JWT_SECRET=node_mvc_secret_123456
JWT_EXPIRES_IN=2h
```

.gitignore :

```
node_modules
.env
uploads
```

八、数据库连接 config/db.js

这里使用 MongoDB + Mongoose。

config/db.js :

```
const mongoose = require("mongoose")

async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGO_URL)
    console.log("MongoDB 连接成功")
  } catch (err) {
    console.error("MongoDB 连接失败: ", err.message)
    process.exit(1)
  }
}

module.exports = connectDB
```

九、用户模型 models/User.js

models/User.js :

```
const mongoose = require("mongoose")

const userSchema = new mongoose.Schema(
  {
    username: {
      type: String,
      required: [true, "用户名不能为空"],
      unique: true,
      trim: true
    },
    password: {
      type: String,
      required: [true, "密码不能为空"]
    }
  }
)
```



```
    nickname: {
      type: String,
      default: ""
    },
    avatar: {
      type: String,
      default: ""
    },
    role: {
      type: String,
      enum: ["user", "admin"],
      default: "user"
    }
  },
  {
    timestamps: true
  }
)

module.exports = mongoose.model("User", userSchema)
```

十、工具函数：生成 Token

utils/generateToken.js：

```
const jwt = require("jsonwebtoken")

function generateToken(user) {
  return jwt.sign(
    {
      id: user._id,
      username: user.username,
      role: user.role
    },
    process.env.JWT_SECRET,
    {
      expiresIn: process.env.JWT_EXPIRES_IN
    }
  )
}

module.exports = generateToken
```

十一、工具函数：异步错误捕获

在 Express 中，异步函数经常需要 `try...catch`。

我们可以封装一个工具函数。

utils/asyncHandler.js :

```
function asyncHandler(fn) {
  return function (req, res, next) {
    Promise.resolve(fn(req, res, next)).catch(next)
  }
}

module.exports = asyncHandler
```

使用前：

```
router.get("/", async (req, res, next) => {
  try {
    const users = await User.find()
    res.json(users)
  } catch (err) {
    next(err)
  }
})
```

使用后：

```
router.get("/", asyncHandler(async (req, res) => {
  const users = await User.find()
  res.json(users)
})))
```

十二、工具函数：统一响应格式

utils/response.js :

```
function success(res, data = null, message = "操作成功", code = 0) {
  res.json({
    code,
    message,
    data
  })
}

function fail(res, message = "操作失败", statusCode = 400, code = statusCode) {
  res.status(statusCode).json({
    code,
    message
  })
}

module.exports = {
  success,
  fail
}
```

```
    fail
  }
}
```

使用：

```
const { success, fail } = require("../utils/response")

success(res, user, "获取成功")
fail(res, "用户不存在", 404)
```

十三、注册登录控制器 authController.js

controllers/authController.js：

```
const bcrypt = require("bcryptjs")
const User = require("../models/User")
const generateToken = require("../utils/generateToken")
const asyncHandler = require("../utils/asyncHandler")
const { success, fail } = require("../utils/response")

exports.register = asyncHandler(async (req, res) => {
  const { username, password, nickname } = req.body

  if (!username || !password) {
    return fail(res, "用户名和密码不能为空", 400)
  }

  const existUser = await User.findOne({ username })

  if (existUser) {
    return fail(res, "用户名已存在", 409)
  }

  const hashedPassword = await bcrypt.hash(password, 10)

  const user = await User.create({
    username,
    password: hashedPassword,
    nickname: nickname || ""
  })

  success(
    res,
    {
      id: user._id,
      username: user.username,
      nickname: user.nickname,
      role: user.role
    },
    "注册成功"
  )
})
```

```

exports.login = asyncHandler(async (req, res) => {
  const { username, password } = req.body

  if (!username || !password) {
    return fail(res, "用户名和密码不能为空", 400)
  }

  const user = await User.findOne({ username })

  if (!user) {
    return fail(res, "用户名或密码错误", 401)
  }

  const isMatch = await bcrypt.compare(password, user.password)

  if (!isMatch) {
    return fail(res, "用户名或密码错误", 401)
  }

  const token = generateToken(user)

  success(
    res,
    {
      token,
      user: {
        id: user._id,
        username: user.username,
        nickname: user.nickname,
        avatar: user.avatar,
        role: user.role
      }
    },
    "登录成功"
  )
})

```

十四、用户控制器 userController.js

controllers/userController.js :

```

const User = require("../models/User")
const asyncHandler = require("../utils/asyncHandler")
const { success, fail } = require("../utils/response")

exports.getProfile = asyncHandler(async (req, res) => {
  const user = await User.findById(req.user.id).select("-password")

  if (!user) {
    return fail(res, "用户不存在", 404)
  }
})

```

```

    success(res, user, "获取用户信息成功")
  })

exports.getUserList = asyncHandler(async (req, res) => {
  const users = await User.find().select("-password").sort({ createdAt: -1 })

  success(res, users, "获取用户列表成功")
})

exports.updateProfile = asyncHandler(async (req, res) => {
  const { nickname, avatar } = req.body

  const user = await User.findByIdAndUpdate(
    req.user.id,
    {
      nickname,
      avatar
    },
    {
      new: true,
      runValidators: true
    }
  ).select("-password")

  if (!user) {
    return fail(res, "用户不存在", 404)
  }

  success(res, user, "修改用户信息成功")
})

```

十五、JWT 鉴权中间件

middlewares/authMiddleware.js :

```

const jwt = require("jsonwebtoken")
const { fail } = require("../utils/response")

function authMiddleware(req, res, next) {
  const authorization = req.headers.authorization

  if (!authorization) {
    return fail(res, "缺少 Authorization 请求头", 401)
  }

  const parts = authorization.split(" ")

  if (parts.length !== 2 || parts[0] !== "Bearer") {
    return fail(res, "Token 格式错误", 401)
  }

  const token = parts[1]

```

```
try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET)
  req.user = decoded
  next()
} catch (err) {
  return fail(res, "Token 无效或已过期", 401)
}

function adminMiddleware(req, res, next) {
  if (!req.user || req.user.role !== "admin") {
    return fail(res, "没有管理员权限", 403)
  }

  next()
}

module.exports = {
  authMiddleware,
  adminMiddleware
}
```

十六、404 中间件

middlewares/notFoundMiddleware.js :

```
function notFoundMiddleware(req, res) {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
}

module.exports = notFoundMiddleware
```

十七、错误处理中间件

middlewares/errorMiddleware.js :

```
function errorMiddleware(err, req, res, next) {
  console.error("错误: ", err)

  const statusCode = res.statusCode && res.statusCode !== 200
    ? res.statusCode
    : 500

  res.status(statusCode).json({
    code: statusCode,
    message: process.env.NODE_ENV === "production"
```

```
      ? "服务器内部错误"
      : err.message
    })
  }

  module.exports = errorMiddleware
```

十八、认证路由 authRoutes.js

routes/authRoutes.js :

```
const express = require("express")
const authController = require("../controllers/authController")

const router = express.Router()

router.post("/register", authController.register)
router.post("/login", authController.login)

module.exports = router
```

最终访问地址：

```
POST /api/auth/register
POST /api/auth/login
```

十九、用户路由 userRoutes.js

routes/userRoutes.js :

```
const express = require("express")
const userController = require("../controllers/userController")
const {
  authMiddleware,
  adminMiddleware
} = require("../middlewares/authMiddleware")

const router = express.Router()

router.get("/profile", authMiddleware, userController.getProfile)

router.put("/profile", authMiddleware, userController.updateProfile)

router.get("/", authMiddleware, adminMiddleware, userController.getUserList)

module.exports = router
```

最终访问地址：

```
GET /api/users/profile
PUT /api/users/profile
GET /api/users
```

其中：

```
GET /api/users
```

需要管理员权限。

二十、app.js

app.js：

```
const express = require("express")
const cors = require("cors")
const path = require("path")

const authRoutes = require("./routes/authRoutes")
const userRoutes = require("./routes/userRoutes")
const notFoundMiddleware = require("./middlewares/notFoundMiddleware")
const errorMiddleware = require("./middlewares/errorMiddleware")

const app = express()

app.use(cors())

app.use(express.json())
app.use(express.urlencoded({ extended: false }))

app.use("/uploads", express.static(path.join(__dirname, "uploads")))

app.get("/", (req, res) => {
  res.send("Node.js MVC 项目启动成功")
})

app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok",
    time: new Date().toISOString()
  })
})

app.use("/api/auth", authRoutes)
app.use("/api/users", userRoutes)

app.use(notFoundMiddleware)
```



```
app.use(errorMiddleware)
```

```
module.exports = app
```

二十一、server.js

server.js :

```
require("dotenv").config()

const app = require("./app")
const connectDB = require("./config/db")

connectDB()

const PORT = process.env.PORT || 3000

app.listen(PORT, () => {
  console.log(`服务器启动: http://localhost:${PORT}`)
})
```

二十二、启动项目

启动 MongoDB 后，运行：

```
npm run dev
```

看到：

```
MongoDB 连接成功
服务器启动: http://localhost:3000
```

说明项目启动成功。

二十三、测试接口

1. 健康检查

```
curl http://localhost:3000/api/health
```

返回：

```
{
  "code": 0,
  "message": "ok",
  "time": "2026-05-28T..."
}
```

2. 注册

```
curl -X POST http://localhost:3000/api/auth/register \
-H "Content-Type: application/json" \
-d "{\"username\":\"admin\",\"password\":\"123456\",\"nickname\":\"管理员\"}"
```

3. 登录

```
curl -X POST http://localhost:3000/api/auth/login \
-H "Content-Type: application/json" \
-d "{\"username\":\"admin\",\"password\":\"123456\"}"
```

返回：

```
{
  "code": 0,
  "message": "登录成功",
  "data": {
    "token": "xxx",
    "user": {
      "id": "xxx",
      "username": "admin",
      "nickname": "管理员",
      "avatar": "",
      "role": "user"
    }
  }
}
```

复制 token。

4. 获取当前用户信息

```
curl http://localhost:3000/api/users/profile \
-H "Authorization: Bearer 你的token"
```

5. 修改当前用户信息

```
curl -X PUT http://localhost:3000/api/users/profile \
-H "Content-Type: application/json" \
-H "Authorization: Bearer 你的token" \
-d '{"nickname":"新昵称","avatar":"/uploads/avatar.png"}'
```

二十四、MVC 中每一层的职责

1. routes：只负责路由匹配

路由层应该尽量简单：

```
router.post("/login", authController.login)
```

不建议在路由里写大量业务逻辑。

2. controllers：负责业务逻辑

控制器负责处理：

- 获取请求参数
- 调用 Model 查询数据库
- 判断业务规则
- 返回响应

例如：

```
exports.login = asyncHandler(async (req, res) => {
  // 登录逻辑
})
```

3. models：负责数据结构

模型负责定义数据库字段：

```
const userSchema = new mongoose.Schema({
  username: String,
  password: String
})
```

4. middlewares：负责通用流程

中间件适合处理：

- 登录校验
 - 权限校验
 - 错误处理
 - 日志记录
 - 文件上传
 - 参数解析
-

5. utils：负责工具函数

工具函数应该是通用的，比如：

- 生成 Token
 - 统一响应
 - 时间格式化
 - 异步错误捕获
-

二十五、进一步拆分：Service 层

有些项目还会增加 `services` 层。

```
controllers
services
models
```

职责：

层	作用
Controller	处理请求和响应
Service	处理核心业务逻辑
Model	操作数据库

例如：

```
controllers/authController.js
services/authService.js
models/User.js
```

authService.js 示例

```
const bcrypt = require("bcryptjs")
const User = require("../models/User")

async function registerUser({ username, password, nickname }) {
  const existUser = await User.findOne({ username })

  if (existUser) {
    const error = new Error("用户名已存在")
    error.statusCode = 409
    throw error
  }

  const hashedPassword = await bcrypt.hash(password, 10)

  const user = await User.create({
    username,
    password: hashedPassword,
    nickname
  })

  return user
}

module.exports = {
  registerUser
}
```

Controller 中：

```
const authService = require("../services/authService")

exports.register = asyncHandler(async (req, res) => {
  const user = await authService.registerUser(req.body)

  res.json({
    code: 0,
    message: "注册成功",
    data: user
  })
})
```

初学阶段可以先不加 Service 层，等项目复杂后再加。

二十六、实战项目：任务管理系统 TODO API

我们用 MVC 结构搭建一个简单实战项目：任务管理系统。

功能：

方法	路径	功能	是否登录
POST	/api/auth/register	注册	否
POST	/api/auth/login	登录	否
GET	/api/todos	获取我的任务	是
POST	/api/todos	创建任务	是
PUT	/api/todos/:id	修改任务	是
DELETE	/api/todos/:id	删除任务	是

二十七、Todo 模型

新建：

```
models/ToDo.js
```

```
const mongoose = require("mongoose")

const todoSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: [true, "任务标题不能为空"],
      trim: true
    },
    completed: {
      type: Boolean,
      default: false
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "User",
      required: true
    }
  },
  {
    timestamps: true
  }
)

module.exports = mongoose.model("Todo", todoSchema)
```

这里的：

```
user: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "User"
}
```

表示每个任务属于某个用户。

二十八、Todo 控制器

新建：

controllers/todoController.js

```
const Todo = require("../models/Todo")
const asyncHandler = require("../utils/asyncHandler")
const { success, fail } = require("../utils/response")

exports.getTodos = asyncHandler(async (req, res) => {
  const todos = await Todo.find({
    user: req.user.id
  }).sort({ createdAt: -1 })

  success(res, todos, "获取任务列表成功")
})

exports.createTodo = asyncHandler(async (req, res) => {
  const { title } = req.body

  if (!title) {
    return fail(res, "任务标题不能为空", 400)
  }

  const todo = await Todo.create({
    title,
    user: req.user.id
  })

  success(res, todo, "创建任务成功")
})

exports.updateTodo = asyncHandler(async (req, res) => {
  const { title, completed } = req.body

  const todo = await Todo.findOneAndUpdate(
    {
      _id: req.params.id,
      user: req.user.id
    },
    {
      title,
      completed
    },
    {
      new: true,
      runValidators: true
    }
  )
})
```

```

    if (!todo) {
      return fail(res, "任务不存在", 404)
    }

    success(res, todo, "修改任务成功")
  })

exports.deleteTodo = asyncHandler(async (req, res) => {
  const todo = await Todo.findOneAndDelete({
    _id: req.params.id,
    user: req.user.id
  })

  if (!todo) {
    return fail(res, "任务不存在", 404)
  }

  success(res, todo, "删除任务成功")
})

```

注意这里查询条件中有：

```
user: req.user.id
```

这样可以保证用户只能操作自己的任务。

二十九、Todo 路由

新建：

```
routes/todoRoutes.js
```

```

const express = require("express")
const todoController = require("../controllers/todoController")
const { authMiddleware } = require("../middlewares/authMiddleware")

const router = express.Router()

router.use(authMiddleware)

router.get("/", todoController.getTodos)
router.post("/", todoController.createTodo)
router.put("/:id", todoController.updateTodo)
router.delete("/:id", todoController.deleteTodo)

module.exports = router

```

这里：


```
router.use(authMiddleware)
```

表示这个路由文件下所有接口都需要登录。

三十、在 app.js 注册 Todo 路由

修改 app.js：

```
const todoRoutes = require("./routes/todoRoutes")
```

然后添加：

```
app.use("/api/todos", todoRoutes)
```

完整片段：

```
app.use("/api/auth", authRoutes)
app.use("/api/users", userRoutes)
app.use("/api/todos", todoRoutes)
```

三十一、测试 Todo 接口

1. 创建任务

```
curl -X POST http://localhost:3000/api/todos \
-H "Content-Type: application/json" \
-H "Authorization: Bearer 你的token" \
-d '{"title":"学习 Node.js MVC"}'
```

2. 获取我的任务

```
curl http://localhost:3000/api/todos \
-H "Authorization: Bearer 你的token"
```

3. 修改任务

```
curl -X PUT http://localhost:3000/api/todos/任务ID \
-H "Content-Type: application/json" \
```

```
-H "Authorization: Bearer 你的token" \  
-d "{\"title\":\"学习 Express MVC\",\"completed\":true}"
```

4. 删除任务

```
curl -X DELETE http://localhost:3000/api/todos/任务ID \  
-H "Authorization: Bearer 你的token"
```

三十二、项目最终结构

完成后结构大概是：

```
node-mvc-demo  
├─ app.js  
├─ server.js  
├─ package.json  
├─ .env  
├─ .gitignore  
├─ config  
│   └─ db.js  
├─ controllers  
│   ├── authController.js  
│   ├── userController.js  
│   └─ todoController.js  
├─ models  
│   ├── User.js  
│   └─ Todo.js  
├─ routes  
│   ├── authRoutes.js  
│   ├── userRoutes.js  
│   └─ todoRoutes.js  
├─ middlewares  
│   ├── authMiddleware.js  
│   ├── errorMiddleware.js  
│   └─ notFoundMiddleware.js  
├─ utils  
│   ├── asyncHandler.js  
│   ├── generateToken.js  
│   └─ response.js  
└─ uploads
```

三十三、开发中的常见规范

1. 路由文件命名

推荐：

```
authRoutes.js  
userRoutes.js  
productRoutes.js  
orderRoutes.js
```

2. 控制器命名

推荐：

```
authController.js  
userController.js  
productController.js
```

3. 模型命名

推荐首字母大写：

```
User.js  
Product.js  
Order.js
```

4. 中间件命名

推荐：

```
authMiddleware.js  
errorMiddleware.js  
uploadMiddleware.js
```

5. 不要在 controller 里写太多重复代码

可以把重复逻辑封装到：

```
utils  
services  
middlewares
```

三十四、常见错误

1. 忘记导出 router

错误：

```
const router = express.Router()

router.get("/", handler)
```

少了：

```
module.exports = router
```

2. 路由路径重复

如果 `app.js` 中：

```
app.use("/api/users", userRoutes)
```

那么 `userRoutes.js` 里不要再写：

```
router.get("/api/users/profile", ...)
```

应该写：

```
router.get("/profile", ...)
```

最终路径才是：

```
/api/users/profile
```

3. 忘记连接数据库

`server.js` 里要调用：

```
connectDB()
```

4. 忘记加载 `.env`

入口文件顶部要写：

```
require("dotenv").config()
```

而且要在使用 `process.env` 之前执行。

5. 错误处理中间件位置不对

推荐顺序：

```
app.use("/api/auth", authRoutes)

app.use(notFoundMiddleware)

app.use(errorMiddleware)
```

404 和错误处理中间件要放在路由后面。

本课重点

这一课你需要掌握：

1. MVC 是一种项目分层思想
 2. `routes` 负责路由匹配
 3. `controllers` 负责业务逻辑
 4. `models` 负责数据库模型
 5. `middlewares` 负责中间件
 6. `utils` 负责工具函数
 7. `app.js` 负责 Express 应用配置
 8. `server.js` 负责启动服务和连接数据库
 9. 项目变大后可以增加 `services` 层
 10. 分层的目标是让项目更清晰、更容易维护
-

课后练习

基于本课 MVC 项目结构，实现一个商品管理模块。

商品字段：

```
{
  name: "苹果",
  price: 5,
  stock: 100,
```

```
    cover: "/uploads/apple.png"
  }
```

要求创建：

```
models/Product.js
controllers/productController.js
routes/productRoutes.js
```

实现接口：

方法	路径	功能
GET	/api/products	获取商品列表
GET	/api/products/:id	获取商品详情
POST	/api/products	新增商品
PUT	/api/products/:id	修改商品
DELETE	/api/products/:id	删除商品

最后在 `app.js` 中注册：

```
app.use("/api/products", productRoutes)
```

下一课建议学习：

Node.js 第十三课：参数校验、统一错误处理与接口规范设计。

Node.js 第十三课：参数校验、统一错误处理与接口规范设计

这一课我们学习真实后端项目中非常重要的三件事：

1. 参数校验
2. 统一错误处理
3. 接口规范设计

前面我们写接口时，经常这样判断参数：

```
if (!username || !password) {
  return res.status(400).json({
    code: 400,
    message: "用户名和密码不能为空"
  })
}
```

小项目可以这样写，但项目变大后会出现很多问题：

- 每个接口都写很多 if
- 错误格式不统一
- 参数校验逻辑重复
- 返回数据格式不统一
- 前端不好处理错误
- 后端维护困难

所以真实项目中，一般会做统一封装。

一、为什么需要参数校验？

用户提交的数据不能直接相信。

比如注册接口：

```
{
  "username": "",
  "password": "123"
}
```

如果不校验，可能导致：

- 用户名为空
- 密码太短
- 邮箱格式错误
- 年龄不是数字
- 商品价格为负数
- 数据库报错
- 安全风险

所以接口收到数据后，要先校验，再进入业务逻辑。

二、常见需要校验的内容

类型	示例
必填	用户名不能为空
字符串长度	密码至少 6 位
格式	邮箱格式、手机号格式
数字范围	年龄必须大于 0
枚举值	状态只能是 pending/done
MongoDB ID	必须是合法 ObjectId
数组	tags 必须是数组

类型	示例
文件	图片类型、大小限制

三、常用参数校验库

Node.js 常见校验库：

库	特点
Joi	功能强大，老牌常用
Zod	TypeScript 友好，现代项目常用
express-validator	Express 专用
Yup	前端表单常用，也可用于后端

这一课我们用 **Joi**，因为它简单、成熟、适合 Express 入门。

四、安装 Joi

在 MVC 项目中安装：

```
npm install joi
```

如果你还没有项目，可以基于上一课的 MVC 项目继续。

五、原始写法的问题

例如注册接口：

```
exports.register = asyncHandler(async (req, res) => {
  const { username, password, email } = req.body

  if (!username) {
    return res.status(400).json({
      code: 400,
      message: "用户名不能为空"
    })
  }

  if (!password) {
    return res.status(400).json({
      code: 400,
      message: "密码不能为空"
    })
  }
})
```



```
    })  
  }  
  
  if (password.length < 6) {  
    return res.status(400).json({  
      code: 400,  
      message: "密码至少 6 位"  
    })  
  }  
}  
  
// 后续注册逻辑  
})
```

问题：

1. 控制器变得很长
2. 每个接口都要写重复判断
3. 错误响应格式不一定统一
4. 参数规则不好复用

六、使用 Joi 定义校验规则

新建目录：

```
validators
```

项目结构：

```
node-mvc-demo  
├── validators  
│   ├── authValidator.js  
│   ├── userValidator.js  
│   └── productValidator.js
```

validators/authValidator.js

```
const Joi = require("joi")  
  
const registerSchema = Joi.object({  
  username: Joi.string()  
    .trim()  
    .min(3)  
    .max(20)  
    .required()  
    .messages({  
      "string.empty": "用户名不能为空",  
      "string.min": "用户名至少 3 个字符",  
    })  
})
```

```
    "string.max": "用户名最多 20 个字符",
    "any.required": "用户名不能为空"
  )),

password: Joi.string()
  .min(6)
  .max(30)
  .required()
  .messages({
    "string.empty": "密码不能为空",
    "string.min": "密码至少 6 位",
    "string.max": "密码最多 30 位",
    "any.required": "密码不能为空"
  )),

email: Joi.string()
  .email()
  .allow("")
  .messages({
    "string.email": "邮箱格式不正确"
  )),

nickname: Joi.string()
  .trim()
  .max(30)
  .allow("")
  .messages({
    "string.max": "昵称最多 30 个字符"
  })
})

const loginSchema = Joi.object({
  username: Joi.string()
    .trim()
    .required()
    .messages({
      "string.empty": "用户名不能为空",
      "any.required": "用户名不能为空"
    }),

  password: Joi.string()
    .required()
    .messages({
      "string.empty": "密码不能为空",
      "any.required": "密码不能为空"
    })
})

module.exports = {
  registerSchema,
  loginSchema
}
```

七、封装通用校验中间件

新建：

```
middlewares/validateMiddleware.js
```

代码：

```
function validate(schema, property = "body") {
  return function (req, res, next) {
    const { error, value } = schema.validate(req[property], {
      abortEarly: false,
      stripUnknown: true
    })

    if (error) {
      const messages = error.details.map(item => item.message)

      return res.status(400).json({
        code: 400,
        message: "参数校验失败",
        errors: messages
      })
    }

    req[property] = value

    next()
  }
}

module.exports = validate
```

说明：

```
abortEarly: false
```

表示不只返回第一个错误，而是返回所有错误。

```
stripUnknown: true
```

表示自动去掉未定义的多余字段。

例如用户传：

```
{
  "username": "admin",
  "password": "123456",
  "isAdmin": true
}
```

如果 Schema 中没有 `isAdmin`，会被自动移除。

八、在路由中使用参数校验

修改：

routes/authRoutes.js

```
const express = require("express")
const authController = require("../controllers/authController")
const validate = require("../middlewares/validateMiddleware")
const {
  registerSchema,
  loginSchema
} = require("../validators/authValidator")

const router = express.Router()

router.post(
  "/register",
  validate(registerSchema),
  authController.register
)

router.post(
  "/login",
  validate(loginSchema),
  authController.login
)

module.exports = router
```

这样请求流程变成：

请求 -> Joi 参数校验 -> controller 业务逻辑

如果校验不通过，直接返回错误，不会进入 controller。

九、Controller 变得更简洁

之前 Controller 里要写很多参数判断。

现在可以简化：

```
exports.register = asyncHandler(async (req, res) => {
  const { username, password, nickname, email } = req.body
```

```
const existUser = await User.findOne({ username })

if (existUser) {
  return fail(res, "用户名已存在", 409)
}

const hashedPassword = await bcrypt.hash(password, 10)

const user = await User.create({
  username,
  password: hashedPassword,
  nickname,
  email
})

success(res, {
  id: user._id,
  username: user.username,
  nickname: user.nickname,
  email: user.email,
  role: user.role
}, "注册成功")
})
```

Controller 专注于业务逻辑，不再负责大量基础参数校验。

十、校验 query 参数

比如商品列表支持分页：

```
GET /api/products?page=1&pageSize=10&keyword=apple
```

新建：

```
validators/productValidator.js
```

```
const Joi = require("joi")

const productListQuerySchema = Joi.object({
  page: Joi.number()
    .integer()
    .min(1)
    .default(1)
    .messages({
      "number.base": "page 必须是数字",
      "number.integer": "page 必须是整数",
      "number.min": "page 不能小于 1"
    })
    ,
  pageSize: Joi.number()
    .integer()
```

```

    .min(1)
    .max(100)
    .default(10)
    .messages({
      "number.base": "pageSize 必须是数字",
      "number.integer": "pageSize 必须是整数",
      "number.min": "pageSize 不能小于 1",
      "number.max": "pageSize 不能大于 100"
    }),

    keyword: Joi.string()
      .trim()
      .allow("")
      .default("")
  })

module.exports = {
  productListQuerySchema
}

```

路由中使用：

```

router.get(
  "/",
  validate(productListQuerySchema, "query"),
  productController.getProductList
)

```

Controller：

```

exports.getProductList = asyncHandler(async (req, res) => {
  const { page, pageSize, keyword } = req.query

  res.json({
    code: 0,
    message: "获取成功",
    data: {
      page,
      pageSize,
      keyword
    }
  })
})

```

十一、校验 params 参数

比如：

```
GET /api/products/:id
```

如果使用 MongoDB，需要校验 `id` 是否是合法 `ObjectId`。

安装 Joi 扩展不是必须，我们可以简单用正则：

```
const Joi = require("joi")

const mongoIdSchema = Joi.object({
  id: Joi.string()
    .pattern(/^[0-9a-fA-F]{24}$/)
    .required()
    .messages({
      "string.pattern.base": "ID 格式不正确",
      "any.required": "ID 不能为空"
    })
})

module.exports = {
  mongoIdSchema
}
```

路由：

```
router.get(
  "/:id",
  validate(mongoIdSchema, "params"),
  productController.getProductDetail
)
```

十二、商品参数校验示例

```
const Joi = require("joi")

const createProductSchema = Joi.object({
  name: Joi.string()
    .trim()
    .min(1)
    .max(100)
    .required()
    .messages({
      "string.empty": "商品名称不能为空",
      "string.max": "商品名称最多 100 个字符",
      "any.required": "商品名称不能为空"
    }),
  price: Joi.number()
    .min(0)
    .required()
    .messages({
      "number.base": "商品价格必须是数字",
      "number.min": "商品价格不能小于 0",
      "any.required": "商品价格不能为空"
    })
})
```

```

    }),

    stock: Joi.number()
      .integer()
      .min(0)
      .default(0)
      .messages({
        "number.base": "库存必须是数字",
        "number.integer": "库存必须是整数",
        "number.min": "库存不能小于 0"
      }),

    cover: Joi.string()
      .allow("")
      .default("")
  })

const updateProductSchema = Joi.object({
  name: Joi.string()
    .trim()
    .min(1)
    .max(100),

  price: Joi.number()
    .min(0),

  stock: Joi.number()
    .integer()
    .min(0),

  cover: Joi.string()
    .allow("")
}).min(1).messages({
  "object.min": "至少需要提交一个要修改的字段"
})

module.exports = {
  createProductSchema,
  updateProductSchema
}

```

十三、统一响应格式设计

真实项目中，建议接口返回格式统一。

推荐格式：

成功响应

```

{
  "code": 0,
  "message": "操作成功",

```



```
"data": {}  
}
```

例如：

```
{  
  "code": 0,  
  "message": "登录成功",  
  "data": {  
    "token": "xxx",  
    "user": {  
      "id": "1",  
      "username": "admin"  
    }  
  }  
}
```

失败响应

```
{  
  "code": 400,  
  "message": "参数错误"  
}
```

如果有多个参数错误：

```
{  
  "code": 400,  
  "message": "参数校验失败",  
  "errors": [  
    "用户名不能为空",  
    "密码至少 6 位"  
  ]  
}
```

分页响应

```
{  
  "code": 0,  
  "message": "获取成功",  
  "data": {  
    "list": [],  
    "pagination": {  
      "page": 1,  
      "pageSize": 10,  
      "total": 100,  
      "totalPages": 10  
    }  
  }  
}
```

```
}  
}  
}
```

十四、优化 response 工具函数

修改：

utils/response.js

```
function success(res, data = null, message = "操作成功", code = 0) {  
  return res.json({  
    code,  
    message,  
    data  
  })  
}  
  
function fail(res, message = "操作失败", statusCode = 400, code = statusCode, errors =  
undefined) {  
  const body = {  
    code,  
    message  
  }  
  
  if (errors) {  
    body.errors = errors  
  }  
  
  return res.status(statusCode).json(body)  
}  
  
function pageSuccess(res, list, pagination, message = "获取成功") {  
  return res.json({  
    code: 0,  
    message,  
    data: {  
      list,  
      pagination  
    }  
  })  
}  
  
module.exports = {  
  success,  
  fail,  
  pageSuccess  
}
```

十五、优化 validateMiddleware

使用 `fail` 工具函数：

```
const { fail } = require("../utils/response")

function validate(schema, property = "body") {
  return function (req, res, next) {
    const { error, value } = schema.validate(req[property], {
      abortEarly: false,
      stripUnknown: true,
      convert: true
    })

    if (error) {
      const errors = error.details.map(item => item.message)

      return fail(res, "参数校验失败", 400, 400, errors)
    }

    req[property] = value

    next()
  }
}

module.exports = validate
```

说明：

```
convert: true
```

表示 Joi 会尝试转换类型。

例如：

```
?page=1
```

原本 query 参数是字符串 "1"，Joi 可以转换成数字 1。

十六、统一错误处理的意义

后端错误可能来自很多地方：

- 数据库连接失败
- 查询数据库失败
- JWT 解析失败
- 参数格式错误
- 文件上传失败

- 业务逻辑错误
- 代码运行异常

如果每个地方都手动 `try...catch`，代码会很乱。

更好的方式是：

```
业务中 throw 错误 -> asyncHandler 捕获 -> errorMiddleware 统一返回
```

十七、自定义 AppError 错误类

新建：

```
utils/AppError.js
```

```
class AppError extends Error {
  constructor(message, statusCode = 500, code = statusCode, errors = undefined) {
    super(message)

    this.statusCode = statusCode
    this.code = code
    this.errors = errors

    Error.captureStackTrace(this, this.constructor)
  }
}

module.exports = AppError
```

使用：

```
const AppError = require("../utils/AppError")

throw new AppError("用户不存在", 404)
```

十八、优化错误处理中间件

修改：

```
middlewares/errorMiddleware.js
```

```
const AppError = require("../utils/AppError")

function errorMiddleware(err, req, res, next) {
  console.error("错误: ", err)
```

```
const statusCode = err.statusCode || 500
const code = err.code || statusCode

const response = {
  code,
  message: err.message || "服务器内部错误"
}

if (err.errors) {
  response.errors = err.errors
}

if (process.env.NODE_ENV !== "production") {
  response.stack = err.stack
}

res.status(statusCode).json(response)
}

module.exports = errorMiddleware
```

生产环境不建议返回 `stack`：

```
NODE_ENV=production
```

返回：

```
{
  "code": 500,
  "message": "服务器内部错误"
}
```

开发环境可以返回更多信息，方便调试。

十九、在 Controller 中使用 AppError

原来：

```
if (!user) {
  return fail(res, "用户不存在", 404)
}
```

也可以改成：

```
if (!user) {
  throw new AppError("用户不存在", 404)
}
```

示例：

```
const User = require("../models/User")
const asyncHandler = require("../utils/asyncHandler")
const AppError = require("../utils/AppError")
const { success } = require("../utils/response")

exports.getProfile = asyncHandler(async (req, res) => {
  const user = await User.findById(req.user.id).select("-password")

  if (!user) {
    throw new AppError("用户不存在", 404)
  }

  success(res, user, "获取用户信息成功")
})
```

这样所有错误都会进入统一错误处理中间件。

二十、处理 Mongoose 常见错误

Mongoose 常见错误：

错误	场景
CastError	ObjectId 格式错误
ValidationError	Schema 校验失败
MongoServerError 11000	唯一索引重复

可以在 `errorMiddleware` 中统一处理。

```
function errorMiddleware(err, req, res, next) {
  console.error("错误: ", err)

  let statusCode = err.statusCode || 500
  let code = err.code || statusCode
  let message = err.message || "服务器内部错误"
  let errors = err.errors

  if (err.name === "CastError") {
    statusCode = 400
    code = 400
    message = "ID 格式不正确"
  }

  if (err.name === "ValidationError") {
    statusCode = 400
    code = 400
    message = "数据校验失败"
    errors = Object.values(err.errors).map(item => item.message)
  }
}
```

```
}

if (err.code === 11000) {
  statusCode = 409
  code = 409
  message = "数据已存在"
}

const response = {
  code,
  message
}

if (errors) {
  response.errors = errors
}

if (process.env.NODE_ENV !== "production") {
  response.stack = err.stack
}

res.status(statusCode).json(response)
}

module.exports = errorMiddleware
```

二十一、接口状态码规范

常用 HTTP 状态码：

状态码	含义	常见场景
200	OK	查询成功、登录成功
201	Created	创建成功
204	No Content	删除成功但无返回内容
400	Bad Request	参数错误
401	Unauthorized	未登录、Token 无效
403	Forbidden	已登录但无权限
404	Not Found	资源不存在
409	Conflict	用户名重复、资源冲突
422	Unprocessable Entity	语义校验失败
500	Server Error	服务器错误

实际项目中常用：

```
res.status(200).json( ... )
res.status(201).json( ... )
```

```
res.status(400).json( ... )
res.status(401).json( ... )
res.status(403).json( ... )
res.status(404).json( ... )
res.status(500).json( ... )
```

二十二、接口路径规范

推荐使用 RESTful 风格。

以商品为例：

方法	路径	说明
GET	/api/products	获取商品列表
GET	/api/products/:id	获取商品详情
POST	/api/products	创建商品
PUT	/api/products/:id	整体更新商品
PATCH	/api/products/:id	局部更新商品
DELETE	/api/products/:id	删除商品

不推荐：

```
/getProductList
/addProduct
/deleteProduct
/updateProduct
```

推荐：

```
GET /api/products
POST /api/products
DELETE /api/products/:id
PUT /api/products/:id
```

二十三、分页接口规范

请求：

```
GET /api/products?page=1&pageSize=10&keyword=苹果
```

响应：


```

{
  "code": 0,
  "message": "获取成功",
  "data": {
    "list": [
      {
        "id": "xxx",
        "name": "苹果",
        "price": 5
      }
    ],
    "pagination": {
      "page": 1,
      "pageSize": 10,
      "total": 100,
      "totalPages": 10
    }
  }
}

```

Controller 示例:

```

const { pageSuccess } = require("../utils/response")

exports.getProductList = asyncHandler(async (req, res) => {
  const { page, pageSize, keyword } = req.query

  const filter = {}

  if (keyword) {
    filter.name = {
      $regex: keyword,
      $options: "i"
    }
  }

  const skip = (page - 1) * pageSize

  const [list, total] = await Promise.all([
    Product.find(filter)
      .skip(skip)
      .limit(pageSize)
      .sort({ createdAt: -1 }),
    Product.countDocuments(filter)
  ])

  pageSuccess(res, list, {
    page,
    pageSize,
    total,
    totalPages: Math.ceil(total / pageSize)
  })
})

```

二十四、统一接口版本

建议接口加统一前缀：

```
/api
```

如果项目比较大，还可以加版本：

```
/api/v1/users  
/api/v1/products  
/api/v2/products
```

在 `app.js` 中：

```
app.use("/api/v1/auth", authRoutes)  
app.use("/api/v1/users", userRoutes)  
app.use("/api/v1/products", productRoutes)
```

这样以后接口升级更方便。

二十五、完整实战：商品模块参数校验 + 错误处理

下面实现一个完整商品模块。

1. Product 模型

`models/Product.js`：

```
const mongoose = require("mongoose")  
  
const productSchema = new mongoose.Schema(  
  {  
    name: {  
      type: String,  
      required: [true, "商品名称不能为空"],  
      trim: true  
    },  
    price: {  
      type: Number,  
      required: [true, "商品价格不能为空"],  
      min: [0, "商品价格不能小于 0"]  
    },  
    stock: {  
      type: Number,  
      default: 0,  

```

```
    min: [0, "库存不能小于 0"]
  },
  cover: {
    type: String,
    default: ""
  }
},
{
  timestamps: true
}
)

module.exports = mongoose.model("Product", productSchema)
```

2. Product 校验器

validators/productValidator.js :

```
const Joi = require("joi")

const mongoIdSchema = Joi.object({
  id: Joi.string()
    .pattern(/^[0-9a-fA-F]{24}$/)
    .required()
    .messages({
      "string.pattern.base": "ID 格式不正确",
      "any.required": "ID 不能为空"
    })
})

const productListQuerySchema = Joi.object({
  page: Joi.number()
    .integer()
    .min(1)
    .default(1),

  pageSize: Joi.number()
    .integer()
    .min(1)
    .max(100)
    .default(10),

  keyword: Joi.string()
    .trim()
    .allow("")
    .default("")
})

const createProductSchema = Joi.object({
  name: Joi.string()
    .trim()
    .required()
    .messages({
```

```

        "string.empty": "商品名称不能为空",
        "any.required": "商品名称不能为空"
    })),

    price: Joi.number()
        .min(0)
        .required()
        .messages({
            "number.base": "商品价格必须是数字",
            "number.min": "商品价格不能小于 0",
            "any.required": "商品价格不能为空"
        }),

    stock: Joi.number()
        .integer()
        .min(0)
        .default(0),

    cover: Joi.string()
        .allow("")
        .default("")
    })

const updateProductSchema = Joi.object({
    name: Joi.string()
        .trim()
        .min(1),

    price: Joi.number()
        .min(0),

    stock: Joi.number()
        .integer()
        .min(0),

    cover: Joi.string()
        .allow("")
    }).min(1).messages({
    "object.min": "至少需要提交一个要修改的字段"
})

module.exports = {
    mongoIdSchema,
    productListQuerySchema,
    createProductSchema,
    updateProductSchema
}

```

3. Product 控制器

controllers/productController.js :

```
const Product = require("../models/Product")
const asyncHandler = require("../utils/asyncHandler")
const AppError = require("../utils/AppError")
const { success, pageSuccess } = require("../utils/response")

exports.getProductList = asyncHandler(async (req, res) => {
  const { page, pageSize, keyword } = req.query

  const filter = {}

  if (keyword) {
    filter.name = {
      $regex: keyword,
      $options: "i"
    }
  }

  const skip = (page - 1) * pageSize

  const [list, total] = await Promise.all([
    Product.find(filter)
      .skip(skip)
      .limit(pageSize)
      .sort({ createdAt: -1 }),

    Product.countDocuments(filter)
  ])

  pageSuccess(res, list, {
    page,
    pageSize,
    total,
    totalPages: Math.ceil(total / pageSize)
  })
})

exports.getProductDetail = asyncHandler(async (req, res) => {
  const product = await Product.findById(req.params.id)

  if (!product) {
    throw new AppError("商品不存在", 404)
  }

  success(res, product, "获取商品详情成功")
})

exports.createProduct = asyncHandler(async (req, res) => {
  const product = await Product.create(req.body)

  res.status(201).json({
    code: 0,
    message: "创建商品成功",
    data: product
  })
})
```

```

exports.updateProduct = asyncHandler(async (req, res) => {
  const product = await Product.findByIdAndUpdate(
    req.params.id,
    req.body,
    {
      new: true,
      runValidators: true
    }
  )

  if (!product) {
    throw new AppError("商品不存在", 404)
  }

  success(res, product, "修改商品成功")
})

exports.deleteProduct = asyncHandler(async (req, res) => {
  const product = await Product.findByIdAndDelete(req.params.id)

  if (!product) {
    throw new AppError("商品不存在", 404)
  }

  success(res, product, "删除商品成功")
})

```

4. Product 路由

routes/productRoutes.js :

```

const express = require("express")
const productController = require("../controllers/productController")
const validate = require("../middlewares/validateMiddleware")
const {
  mongoIdSchema,
  productListQuerySchema,
  createProductSchema,
  updateProductSchema
} = require("../validators/productValidator")

const router = express.Router()

router.get(
  "/",
  validate(productListQuerySchema, "query"),
  productController.getProductList
)

router.get(
  "/:id",
  validate(mongoIdSchema, "params"),
  productController.getProductDetail
)

```

```
)

router.post(
  "/",
  validate(createProductSchema),
  productController.createProduct
)

router.put(
  "/:id",
  validate(mongoIdSchema, "params"),
  validate(updateProductSchema),
  productController.updateProduct
)

router.delete(
  "/:id",
  validate(mongoIdSchema, "params"),
  productController.deleteProduct
)

module.exports = router
```

5. app.js 注册路由

```
const productRoutes = require("./routes/productRoutes")

app.use("/api/products", productRoutes)
```

二十六、测试商品接口

1. 创建商品

```
curl -X POST http://localhost:3000/api/products \
-H "Content-Type: application/json" \
-d '{"name":"苹果","price":5,"stock":100}'
```

2. 参数错误测试

```
curl -X POST http://localhost:3000/api/products \
-H "Content-Type: application/json" \
-d '{"name":"","price":-1}'
```

返回类似：

```
{
  "code": 400,
  "message": "参数校验失败",
  "errors": [
    "商品名称不能为空",
    "商品价格不能小于 0"
  ]
}
```

3. 获取分页商品

```
curl "http://localhost:3000/api/products?page=1&pageSize=10&keyword=苹果"
```

4. ID 格式错误

```
curl http://localhost:3000/api/products/abc
```

返回：

```
{
  "code": 400,
  "message": "参数校验失败",
  "errors": [
    "ID 格式不正确"
  ]
}
```

二十七、接口文档规范建议

后端开发时，最好给前端提供接口文档。

常见工具：

工具	说明
Apifox	国内常用，接口调试和文档
Postman	国际常用
Swagger / OpenAPI	自动生成接口文档
YApi	老牌接口管理平台

接口文档至少包括：

- 1. 接口名称
- 2. 请求方法
- 3. 请求路径
- 4. 请求头
- 5. 请求参数
- 6. 响应示例
- 7. 错误码说明

例如：

接口：创建商品
方法：POST
路径：/api/products
Content-Type: application/json

请求体：
{
 "name": "苹果",
 "price": 5,
 "stock": 100
}

成功响应：
{
 "code": 0,
 "message": "创建商品成功",
 "data": {
 "_id": "xxx",
 "name": "苹果",
 "price": 5,
 "stock": 100
 }
}

二十八、常见错误码设计

可以设计业务错误码：

code	含义
0	成功
400	参数错误
401	未登录或 Token 无效
403	无权限
404	资源不存在
409	资源冲突
500	服务器错误

也可以设计更细的业务码：

code	含义
10001	用户名已存在
10002	用户名或密码错误
10003	Token 无效
20001	商品不存在
30001	订单不存在

初学阶段用 HTTP 状态码作为 code 就够了。

二十九、推荐的 Express 中间件顺序

app.js 推荐顺序：

```
const express = require("express")
const cors = require("cors")

const app = express()

// 1. 跨域
app.use(cors())

// 2. 请求体解析
app.use(express.json())
app.use(express.urlencoded({ extended: false }))

// 3. 静态资源
app.use("/uploads", express.static("uploads"))

// 4. 健康检查
app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok"
  })
})

// 5. 业务路由
app.use("/api/auth", authRoutes)
app.use("/api/users", userRoutes)
app.use("/api/products", productRoutes)

// 6. 404
app.use(notFoundMiddleware)

// 7. 统一错误处理
app.use(errorMiddleware)
```

```
module.exports = app
```

顺序很重要：

普通中间件 -> 路由 -> 404 -> 错误处理中间件

三十、常见问题

1. Joi 校验 query 后数字还是字符串？

确保配置：

```
convert: true
```

```
schema.validate(req.query, {  
  convert: true  
})
```

2. Joi 自定义错误消息不生效？

检查 `.messages()` 中的 key 是否正确。

常见 key：

```
string.empty  
string.min  
string.max  
number.base  
number.min  
any.required  
object.min
```

3. 错误处理中间件不执行？

错误处理中间件必须有 4 个参数：

```
function errorMiddleware(err, req, res, next) {}
```

并且要放在所有路由后面：

```
app.use(errorMiddleware)
```

4. async 函数报错服务崩溃?

需要使用 `asyncHandler` 包裹:

```
router.get("/", asyncHandler(async (req, res) => {  
  throw new Error("测试错误")  
}))
```

5. 参数校验通过后多余字段还在?

使用:

```
stripUnknown: true
```

本课重点

这一课你需要掌握:

1. 后端必须校验用户输入
2. 参数校验可以使用 `Joi`
3. 校验逻辑建议放在 `validators` 目录
4. 使用 `validateMiddleware` 统一校验
5. 可以校验:
 - `req.body`
 - `req.query`
 - `req.params`
6. 成功响应格式要统一
7. 错误响应格式要统一
8. 使用 `AppError` 表示业务错误
9. 使用 `errorMiddleware` 统一处理错误
10. RESTful 接口路径更清晰
11. 分页接口要统一返回 `list + pagination`
12. 错误处理中间件必须放在路由后面

课后练习

基于上一课 MVC 项目，完成以下任务：

1. 安装 Joi

```
npm install joi
```

2. 新增目录

```
validators
```

3. 创建商品校验器

```
validators/productValidator.js
```

实现：

- 创建商品校验
- 修改商品校验
- 商品 ID 校验
- 分页查询校验

4. 创建通用校验中间件

```
middlewares/validateMiddleware.js
```

支持校验：

```
validate(schema, "body")
validate(schema, "query")
validate(schema, "params")
```

5. 创建统一错误类

```
utils/AppError.js
```

6. 改造商品接口

要求：

接口	校验
GET /api/products	page、pageSize、keyword
GET /api/products/:id	id
POST /api/products	name、price、stock、cover
PUT /api/products/:id	id + body

接口	校验
DELETE /api/products/:id	id

7. 统一响应格式

成功：

```
{
  "code": 0,
  "message": "操作成功",
  "data": {}
}
```

失败：

```
{
  "code": 400,
  "message": "参数校验失败",
  "errors": []
}
```

下一课建议学习：

Node.js 第十四课：日志系统、接口调试与性能优化基础。

Node.js 第十四课：日志系统、接口调试与性能优化基础

这一课学习真实项目中非常实用的三块内容：

1. 日志系统
2. 接口调试
3. 性能优化基础

前面我们已经学习了：

- Express
- 路由和中间件
- 数据库
- JWT 鉴权
- 文件上传
- MVC 分层
- 参数校验
- 统一错误处理

现在项目已经越来越接近真实后端项目了。真实项目上线后，最常见的问题是：

- 接口报错了，但不知道为什么

- 用户说接口慢，但你不知道慢在哪里
- 服务器崩了，但没有记录
- 数据库查询异常，但没有日志
- 前端说接口返回不对，后端无法快速复现
- 线上问题只能靠猜

所以我们需要学习 **日志、调试和基础性能优化**。

一、为什么需要日志系统？

很多初学者只会用：

```
console.log("xxx")
```

开发时可以，但线上项目不够用。

真实项目需要记录：

- 请求日志
- 错误日志
- 登录日志
- 数据库异常
- 文件上传异常
- 慢接口
- 服务器启动信息
- 用户操作记录

例如：

```
[2026-05-28 10:30:00] GET /api/products 200 35ms
[2026-05-28 10:31:00] POST /api/auth/login 401 用户名或密码错误
[2026-05-28 10:32:00] ERROR MongoDB 连接失败
```

这些日志可以帮助你定位问题。

二、日志分为哪些类型？

常见日志分类：

日志类型	说明
访问日志	每次请求的方法、路径、状态码、耗时
错误日志	系统错误、数据库错误、接口异常
业务日志	用户登录、下单、支付、上传文件

日志类型	说明
调试日志	开发阶段排查问题
性能日志	慢接口、慢 SQL、内存占用

三、日志级别

常见日志级别：

级别	说明
error	错误，必须关注
warn	警告，可能有风险
info	普通信息
http	HTTP 请求日志
debug	调试信息

一般生产环境重点保留：

```
error
warn
info
http
```

开发环境可以开启：

```
debug
```

四、使用 morgan 记录请求日志

Express 项目中，记录 HTTP 请求日志常用：

```
morgan
```

安装：

```
npm install morgan
```

使用：

```
const morgan = require("morgan")
```



```
app.use(morgan("dev"))
```

完整示例：

```
const express = require("express")
const morgan = require("morgan")

const app = express()

app.use(morgan("dev"))

app.get("/", (req, res) => {
  res.send("首页")
})

app.listen(3000, () => {
  console.log("服务器启动: http://localhost:3000")
})
```

访问接口后，终端会输出：

```
GET / 200 5.123 ms - 6
```

五、morgan 常用模式

1. dev

开发环境常用：

```
app.use(morgan("dev"))
```

输出：

```
GET /api/users 200 12.345 ms - 123
```

2. combined

生产环境常用，信息更完整：

```
app.use(morgan("combined"))
```

输出类似：

```
:::1 - - [28/May/2026:10:30:00 +0800] "GET /api/users HTTP/1.1" 200 123 "-" "Mozilla/5.0"
```

3. tiny

简洁模式：

```
app.use(morgan("tiny"))
```

输出：

```
GET /api/users 200 123 - 12.345 ms
```

六、把请求日志写入文件

只打印到控制台还不够，生产环境通常要写入文件。

新建目录：

```
logs
```

代码：

```
const fs = require("fs")
const path = require("path")
const morgan = require("morgan")

const logDir = path.join(__dirname, "logs")

if (!fs.existsSync(logDir)) {
  fs.mkdirSync(logDir)
}

const accessLogStream = fs.createWriteStream(
  path.join(logDir, "access.log"),
  {
    flags: "a"
  }
)

app.use(morgan("combined", {
  stream: accessLogStream
}))
```

这样请求日志会写入：

```
logs/access.log
```

七、使用 winston 做专业日志系统

`morgan` 主要适合记录 HTTP 请求日志。

如果要记录错误日志、业务日志，推荐使用：

```
winston
```

安装：

```
npm install winston
```

八、封装 logger 工具

项目结构：

```
node-mvc-demo
├─ utils
│   └─ logger.js
├─ logs
│   └─ app.log
│   └─ error.log
```

创建：

```
utils/logger.js
```

代码：

```
const fs = require("fs")
const path = require("path")
const winston = require("winston")

const logDir = path.join(__dirname, "..", "logs")

if (!fs.existsSync(logDir)) {
  fs.mkdirSync(logDir)
}

const logger = winston.createLogger({
  level: process.env.NODE_ENV === "production" ? "info" : "debug",

  format: winston.format.combine(
    winston.format.timestamp({
      format: "YYYY-MM-DD HH:mm:ss"
    }),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
```

```

    transports: [
      new winston.transports.File({
        filename: path.join(logDir, "error.log"),
        level: "error"
      }),

      new winston.transports.File({
        filename: path.join(logDir, "app.log")
      })
    ]
  })

  if (process.env.NODE_ENV !== "production") {
    logger.add(
      new winston.transports.Console({
        format: winston.format.combine(
          winston.format.colorize(),
          winston.format.simple()
        )
      })
    )
  }
}

module.exports = logger

```

九、使用 logger 记录日志

在任意文件中使用：

```

const logger = require("../utils/logger")

logger.info("服务器启动成功")
logger.warn("库存不足")
logger.error("数据库连接失败")
logger.debug("调试信息")

```

例如在 `server.js` 中：

```

require("dotenv").config()

const app = require("./app")
const connectDB = require("./config/db")
const logger = require("../utils/logger")

connectDB()

const PORT = process.env.PORT || 3000

app.listen(PORT, () => {

```

```
logger.info(`服务器启动: http://localhost:${PORT}`)  
})
```

十、在错误处理中间件中记录错误日志

修改:

middlewares/errorMiddleware.js

```
const logger = require("../utils/logger")  
  
function errorMiddleware(err, req, res, next) {  
  logger.error({  
    message: err.message,  
    stack: err.stack,  
    method: req.method,  
    url: req.originalUrl,  
    ip: req.ip,  
    body: req.body,  
    query: req.query,  
    params: req.params  
  })  
  
  let statusCode = err.statusCode || 500  
  let code = err.code || statusCode  
  let message = err.message || "服务器内部错误"  
  let errors = err.errors  
  
  if (err.name === "CastError") {  
    statusCode = 400  
    code = 400  
    message = "ID 格式不正确"  
  }  
  
  if (err.name === "ValidationError") {  
    statusCode = 400  
    code = 400  
    message = "数据校验失败"  
    errors = Object.values(err.errors).map(item => item.message)  
  }  
  
  if (err.code === 11000) {  
    statusCode = 409  
    code = 409  
    message = "数据已存在"  
  }  
  
  const response = {  
    code,  
    message  
  }  
}
```

```
if (errors) {
  response.errors = errors
}

if (process.env.NODE_ENV !== "production") {
  response.stack = err.stack
}

res.status(statusCode).json(response)
}

module.exports = errorMiddleware
```

这样线上报错时，你可以查看：

```
logs/error.log
```

十一、请求日志中间件：记录接口耗时

除了 morgan，我们也可以自己写一个请求耗时中间件。

新建：

```
middlewares/requestLogger.js
```

```
const logger = require("../utils/logger")

function requestLogger(req, res, next) {
  const start = Date.now()

  res.on("finish", () => {
    const duration = Date.now() - start

    logger.info({
      type: "request",
      method: req.method,
      url: req.originalUrl,
      statusCode: res.statusCode,
      duration: `${duration}ms`,
      ip: req.ip
    })
  })

  next()
}

module.exports = requestLogger
```

在 app.js 中使用：

```
const requestLogger = require("../middlewares/requestLogger")

app.use(requestLogger)
```

日志示例：

```
{
  "level": "info",
  "message": {
    "type": "request",
    "method": "GET",
    "url": "/api/products",
    "statusCode": 200,
    "duration": "25ms",
    "ip": "::1"
  },
  "timestamp": "2026-05-28 10:30:00"
}
```

十二、记录慢接口

比如超过 500ms 的接口记录为警告。

修改 requestLogger.js：

```
const logger = require("../utils/logger")

function requestLogger(req, res, next) {
  const start = Date.now()

  res.on("finish", () => {
    const duration = Date.now() - start

    const logData = {
      type: "request",
      method: req.method,
      url: req.originalUrl,
      statusCode: res.statusCode,
      duration,
      ip: req.ip
    }

    if (duration > 500) {
      logger.warn({
        ...logData,
        message: "慢接口"
      })
    } else {
      logger.info(logData)
    }
  })
}
```

```
    next()
  }

  module.exports = requestLogger
```

这样当接口耗时超过 500ms，会记录警告日志。

十三、日志文件切割

如果一直写入：

```
app.log
error.log
```

时间长了文件会非常大。

生产环境一般需要日志切割，例如：

```
app-2026-05-28.log
app-2026-05-29.log
error-2026-05-28.log
```

可以使用：

```
winston-daily-rotate-file
```

安装：

```
npm install winston-daily-rotate-file
```

修改 `utils/logger.js`：

```
const fs = require("fs")
const path = require("path")
const winston = require("winston")
require("winston-daily-rotate-file")

const logDir = path.join(__dirname, "..", "logs")

if (!fs.existsSync(logDir)) {
  fs.mkdirSync(logDir)
}

const dailyRotateApp = new winston.transports.DailyRotateFile({
  filename: path.join(logDir, "app-%DATE%.log"),
  datePattern: "YYYY-MM-DD",
  maxSize: "20m",
```



```
    maxFiles: "14d"
  })

  const dailyRotateError = new winston.transports.DailyRotateFile({
    filename: path.join(logDir, "error-%DATE%.log"),
    datePattern: "YYYY-MM-DD",
    level: "error",
    maxSize: "20m",
    maxFiles: "30d"
  })

  const logger = winston.createLogger({
    level: process.env.NODE_ENV === "production" ? "info" : "debug",

    format: winston.format.combine(
      winston.format.timestamp({
        format: "YYYY-MM-DD HH:mm:ss"
      }),
      winston.format.errors({ stack: true }),
      winston.format.json()
    ),

    transports: [
      dailyRotateApp,
      dailyRotateError
    ]
  })

  if (process.env.NODE_ENV !== "production") {
    logger.add(
      new winston.transports.Console({
        format: winston.format.combine(
          winston.format.colorize(),
          winston.format.simple()
        )
      })
    )
  }
}

module.exports = logger
```

参数说明：

参数	说明
datePattern	按日期切割
maxSize	单个日志文件最大大小
maxFiles	保留多久的日志

十四、接口调试工具

开发接口时，常用调试工具：

工具	作用
curl	命令行测试接口
Postman	图形化接口调试
Apifox	接口文档 + 调试 + Mock
VS Code REST Client	在 VS Code 中测试接口
浏览器 DevTools	查看前端请求
Swagger	生成接口文档

十五、使用 curl 调试接口

1. GET 请求

```
curl http://localhost:3000/api/products
```

2. 带 query 参数

```
curl "http://localhost:3000/api/products?page=1&pageSize=10"
```

3. POST JSON 请求

```
curl -X POST http://localhost:3000/api/products \
-H "Content-Type: application/json" \
-d '{"name":"苹果","price":5,"stock":100}'
```

4. 带 Token 请求

```
curl http://localhost:3000/api/users/profile \
-H "Authorization: Bearer 你的token"
```

5. 上传文件

```
curl -X POST http://localhost:3000/api/upload/avatar \
-F "avatar=@./avatar.png"
```

6. 查看响应头

```
curl -i http://localhost:3000/api/health
```

7. 只看状态码

```
curl -o /dev/null -s -w "%{http_code}\n" http://localhost:3000/api/health
```

8. 查看请求耗时

```
curl -o /dev/null -s -w "time_total: %{time_total}s\n" http://localhost:3000/api/products
```

十六、使用 VS Code REST Client

安装 VS Code 插件：

```
REST Client
```

在项目根目录创建：

```
api.http
```

内容：

```
### 健康检查
GET http://localhost:3000/api/health

### 注册
POST http://localhost:3000/api/auth/register
Content-Type: application/json

{
  "username": "admin",
  "password": "123456",
  "nickname": "管理员"
```

```
}

### 登录
POST http://localhost:3000/api/auth/login
Content-Type: application/json

{
  "username": "admin",
  "password": "123456"
}

### 获取商品列表
GET http://localhost:3000/api/products?page=1&pageSize=10

### 创建商品
POST http://localhost:3000/api/products
Content-Type: application/json

{
  "name": "苹果",
  "price": 5,
  "stock": 100
}
```

如果要使用变量：

```
@baseUrl = http://localhost:3000
@token = 你的token

### 获取个人信息
GET {{baseUrl}}/api/users/profile
Authorization: Bearer {{token}}
```

这样接口调试会非常方便。

十七、接口调试时应该检查什么？

调试接口时，不要只看“能不能请求成功”，还要检查：

1. 请求方法是否正确

```
GET / POST / PUT / DELETE
```

2. 请求路径是否正确

例如：

```
/api/products  
/api/products/:id
```

3. 请求头是否正确

JSON 请求需要：

```
Content-Type: application/json
```

JWT 需要：

```
Authorization: Bearer token
```

4. 请求体是否正确

JSON 格式要合法：

```
{  
  "name": "苹果",  
  "price": 5  
}
```

错误 JSON：

```
{  
  "name": "苹果",  
  "price": 5,  
}
```

最后多了一个逗号，会解析失败。

5. 响应状态码是否正确

例如：

```
200 查询成功  
201 创建成功  
400 参数错误  
401 未登录  
403 无权限  
404 不存在  
500 服务器错误
```

6. 响应格式是否统一

成功：

```
{
  "code": 0,
  "message": "操作成功",
  "data": {}
}
```

失败：

```
{
  "code": 400,
  "message": "参数校验失败",
  "errors": []
}
```

十八、性能优化基础：先测量，再优化

性能优化不要靠猜。

正确流程：

```
发现慢接口
↓
查看日志耗时
↓
定位是数据库慢、代码慢、网络慢还是文件慢
↓
针对性优化
↓
再次测试
```

常见性能瓶颈：

瓶颈	表现
数据库查询慢	接口耗时高
没有分页	一次返回大量数据
没有索引	查询越来越慢
同步阻塞代码	并发能力差
文件过大	上传下载慢
日志过多	磁盘压力大
JSON 数据太大	响应慢

瓶颈	表现
CPU 密集任务	Node.js 主线程阻塞

十九、优化一：列表接口必须分页

不要这样：

```
const products = await Product.find()
```

如果数据库有 10 万条数据，一次查出来会很慢。

推荐：

```
const page = Number(req.query.page) || 1
const pageSize = Number(req.query.pageSize) || 10
const skip = (page - 1) * pageSize

const [list, total] = await Promise.all([
  Product.find()
    .skip(skip)
    .limit(pageSize)
    .sort({ createdAt: -1 }),
  Product.countDocuments()
])
```

返回：

```
res.json({
  code: 0,
  data: {
    list,
    pagination: {
      page,
      pageSize,
      total,
      totalPages: Math.ceil(total / pageSize)
    }
  }
})
```

二十、优化二：只查询需要的字段

不要返回敏感字段或不必要字段。

例如用户列表不应该返回密码：

```
const users = await User.find().select("-password")
```

只返回指定字段：

```
const users = await User.find().select("username nickname avatar")
```

商品列表只返回部分字段：

```
const products = await Product.find()  
  .select("name price stock cover")
```

好处：

- 减少数据库读取
- 减少网络传输
- 避免泄露敏感信息

二十一、优化三：给常用查询字段加索引

如果经常按用户名查询：

```
User.findOne({ username })
```

应该给 `username` 加索引。

Mongoose Schema：

```
username: {  
  type: String,  
  required: true,  
  unique: true,  
  index: true  
}
```

如果商品经常按名称搜索：

```
productSchema.index({ name: 1 })
```

如果经常按创建时间排序：

```
productSchema.index({ createdAt: -1 })
```

复合索引：


```
productSchema.index({
  category: 1,
  createdAt: -1
})
```

注意：

索引不是越多越好。索引会加快查询，但会增加写入和存储成本。

二十二、优化四：避免在接口中做耗时同步操作

Node.js 是单线程事件循环模型。

如果你写了耗时同步代码，会阻塞其他请求。

不推荐：

```
const fs = require("fs")

app.get("/file", (req, res) => {
  const data = fs.readFileSync("./big.txt", "utf-8")
  res.send(data)
})
```

推荐异步：

```
const fs = require("fs/promises")

app.get("/file", async (req, res, next) => {
  try {
    const data = await fs.readFile("./big.txt", "utf-8")
    res.send(data)
  } catch (err) {
    next(err)
  }
})
```

二十三、优化五：使用 Promise.all 并行执行无依赖任务

不推荐：

```
const user = await User.findById(userId)
const orders = await Order.find({ user: userId })
const coupons = await Coupon.find({ user: userId })
```

如果这三个查询互不依赖，可以并行：

```
const [user, orders, coupons] = await Promise.all([
  User.findById(userId),
  Order.find({ user: userId }),
  Coupon.find({ user: userId })
])
```

这样接口耗时可能明显减少。

二十四、优化六：开启 gzip 压缩

如果接口返回数据较大，可以开启 gzip 压缩。

安装：

```
npm install compression
```

使用：

```
const compression = require("compression")

app.use(compression())
```

建议在 `app.js` 靠前位置使用：

```
app.use(compression())
app.use(express.json())
```

Nginx 也可以开启 gzip。

二十五、优化七：限制请求体大小

防止用户提交超大 JSON 导致服务器压力过大。

```
app.use(express.json({
  limit: "1mb"
}))

app.use(express.urlencoded({
  extended: false,
  limit: "1mb"
}))
```

文件上传则使用 `multer` 限制：

```
limits: {
  fileSize: 2 * 1024 * 1024
}
```

二十六、优化八：使用缓存

对于一些不经常变化的数据，可以缓存。

例如：

- 首页配置
- 商品分类
- 系统字典
- 热门商品
- 用户权限信息

简单内存缓存示例：

```
let categoryCache = null
let categoryCacheTime = 0

app.get("/api/categories", async (req, res) => {
  const now = Date.now()

  if (categoryCache && now - categoryCacheTime < 60 * 1000) {
    return res.json({
      code: 0,
      message: "获取成功",
      data: categoryCache,
      cache: true
    })
  }

  const categories = await Category.find()

  categoryCache = categories
  categoryCacheTime = now

  res.json({
    code: 0,
    message: "获取成功",
    data: categories,
    cache: false
  })
})
```

真实项目更常用：

Redis

后面可以单独学习 Redis 缓存。

二十七、优化九：使用 lean() 提升 Mongoose 查询性能

Mongoose 默认返回的是 Document 对象，带很多方法。

如果只是查询并返回 JSON，不需要修改文档，可以使用：

```
const products = await Product.find().lean()
```

例如：

```
const list = await Product.find(filter)
  .select("name price stock cover")
  .skip(skip)
  .limit(pageSize)
  .sort({ createdAt: -1 })
  .lean()
```

lean() 返回普通 JavaScript 对象，通常更快、更省内存。

适合：

- 列表查询
- 详情展示
- 只读接口

不适合：

- 需要调用 document.save()
- 需要使用 Mongoose document 方法

二十八、优化十：避免返回过大的 JSON

如果一个接口返回：

```
{
  "list": [几万条数据]
}
```

前端会慢，后端也慢。

应该：

1. 分页
2. 限制 pageSize 最大值

3. 只返回需要字段
4. 大文件不要用 JSON 返回

例如 Joi 校验限制：

```
pageSize: Joi.number()  
  .integer()  
  .min(1)  
  .max(100)  
  .default(10)
```

二十九、优化十一：合理使用 PM2 cluster

Node.js 单个进程只能使用一个 CPU 核心。

生产环境可以用 PM2 cluster 模式开启多进程。

ecosystem.config.js：

```
module.exports = {  
  apps: [  
    {  
      name: "my-node-app",  
      script: "server.js",  
      instances: "max",  
      exec_mode: "cluster",  
      autorestart: true,  
      watch: false,  
      max_memory_restart: "300M",  
      env: {  
        NODE_ENV: "production",  
        PORT: 3000  
      }  
    }  
  ]  
}
```

启动：

```
pm2 start ecosystem.config.js
```

说明：

```
instances: "max"
```

表示根据 CPU 核心数启动多个进程。

初学阶段可以先用：

```
instances: 1
```

等项目上线有并发需求后，再使用 cluster。

三十、查看 Node.js 服务资源占用

1. PM2 查看

```
pm2 list
```

可以看到：

```
CPU
Memory
Status
Restart
```

查看详细信息：

```
pm2 show my-node-app
```

监控：

```
pm2 monit
```

2. Linux 查看

查看 CPU 和内存：

```
top
```

更友好的工具：

```
htop
```

安装：

```
apt install htop -y
```

查看磁盘：

```
df -h
```

查看目录大小：

```
du -sh logs
```

查看端口：

```
lsof -i:3000
```

三十一、调试线上问题的基本流程

当用户反馈：

接口访问失败

你可以按这个顺序排查：

1. 服务是否运行？

```
pm2 list
```

2. 查看应用日志

```
pm2 logs my-node-app
```

或者：

```
tail -f logs/error.log
```

3. 本机 curl 测试 Node 服务

```
curl http://127.0.0.1:3000/api/health
```

4. 测试 Nginx 访问

```
curl http://服务器IP/api/health
```

5. 检查 Nginx 状态

```
systemctl status nginx
```

6. 检查 Nginx 配置

```
nginx -t
```

7. 检查服务器资源

```
top  
df -h
```

三十二、常见性能问题和解决方案

问题	原因	解决
列表接口很慢	没分页	加分页
搜索很慢	没索引	加索引
返回很慢	数据太大	select 字段、分页
CPU 很高	同步计算、死循环	改异步、拆任务
内存越来越高	内存泄漏	检查缓存、全局变量
上传失败	文件太大	multer/Nginx 设置大小
502	Node 服务挂了	PM2 logs 排查
504	接口超时	优化慢查询、Nginx 超时配置

三十三、给 Express 增加基础安全和性能中间件

可以安装：


```
npm install helmet compression
```

使用：

```
const helmet = require("helmet")
const compression = require("compression")

app.use(helmet())
app.use(compression())
```

helmet 可以设置一些安全相关 HTTP 头。

compression 可以开启响应压缩。

完整片段：

```
const express = require("express")
const cors = require("cors")
const helmet = require("helmet")
const compression = require("compression")

const app = express()

app.use(helmet())
app.use(compression())
app.use(cors())

app.use(express.json({ limit: "1mb" }))
app.use(express.urlencoded({ extended: false, limit: "1mb" })))
```

三十四、推荐 app.js 中间件顺序

```
const express = require("express")
const cors = require("cors")
const helmet = require("helmet")
const compression = require("compression")
const morgan = require("morgan")

const requestLogger = require("../middlewares/requestLogger")
const notFoundMiddleware = require("../middlewares/notFoundMiddleware")
const errorMiddleware = require("../middlewares/errorMiddleware")

const app = express()

// 1. 安全响应头
app.use(helmet())

// 2. gzip 压缩
app.use(compression())
```

```
// 3. 跨域
app.use(cors())

// 4. 请求日志
if (process.env.NODE_ENV !== "production") {
  app.use(morgan("dev"))
}

app.use(requestLogger)

// 5. 请求体解析，并限制大小
app.use(express.json({ limit: "1mb" }))
app.use(express.urlencoded({ extended: false, limit: "1mb" }))

// 6. 静态资源
app.use("/uploads", express.static("uploads"))

// 7. 健康检查
app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok",
    time: new Date().toISOString()
  })
})

// 8. 业务路由
app.use("/api/auth", authRoutes)
app.use("/api/users", userRoutes)
app.use("/api/products", productRoutes)

// 9. 404
app.use(notFoundMiddleware)

// 10. 错误处理
app.use(errorMiddleware)

module.exports = app
```

三十五、完整日志系统整合示例

安装：

```
npm install winston winston-daily-rotate-file morgan helmet compression
```

项目结构：

```
node-mvc-demo
├─ app.js
├─ server.js
├─ logs
└─ utils
```

```
|   └─ logger.js
└─ middlewares
    └─ requestLogger.js
    └─ errorMiddleware.js
```

utils/logger.js

```
const fs = require("fs")
const path = require("path")
const winston = require("winston")
require("winston-daily-rotate-file")

const logDir = path.join(__dirname, "..", "logs")

if (!fs.existsSync(logDir)) {
  fs.mkdirSync(logDir)
}

const logger = winston.createLogger({
  level: process.env.NODE_ENV === "production" ? "info" : "debug",

  format: winston.format.combine(
    winston.format.timestamp({
      format: "YYYY-MM-DD HH:mm:ss"
    }),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),

  transports: [
    new winston.transports.DailyRotateFile({
      filename: path.join(logDir, "app-%DATE%.log"),
      datePattern: "YYYY-MM-DD",
      maxSize: "20m",
      maxFiles: "14d"
    }),

    new winston.transports.DailyRotateFile({
      filename: path.join(logDir, "error-%DATE%.log"),
      datePattern: "YYYY-MM-DD",
      level: "error",
      maxSize: "20m",
      maxFiles: "30d"
    })
  ]
})

if (process.env.NODE_ENV !== "production") {
  logger.add(
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.simple()
      )
    })
  )
}
```

```
    )
  })
}

module.exports = logger
```

middlewares/requestLogger.js

```
const logger = require("../utils/logger")

function requestLogger(req, res, next) {
  const start = Date.now()

  res.on("finish", () => {
    const duration = Date.now() - start

    const logData = {
      type: "request",
      method: req.method,
      url: req.originalUrl,
      statusCode: res.statusCode,
      duration,
      ip: req.ip
    }

    if (duration > 500) {
      logger.warn({
        ...logData,
        message: "慢接口"
      })
    } else {
      logger.info(logData)
    }
  })

  next()
}

module.exports = requestLogger
```

middlewares/errorMiddleware.js

```
const logger = require("../utils/logger")

function errorMiddleware(err, req, res, next) {
  logger.error({
    message: err.message,
    stack: err.stack,
  })
}
```

```

    method: req.method,
    url: req.originalUrl,
    ip: req.ip,
    body: req.body,
    query: req.query,
    params: req.params
  })

  let statusCode = err.statusCode || 500
  let code = err.code || statusCode
  let message = err.message || "服务器内部错误"
  let errors = err.errors

  if (err.name === "CastError") {
    statusCode = 400
    code = 400
    message = "ID 格式不正确"
  }

  if (err.name === "ValidationError") {
    statusCode = 400
    code = 400
    message = "数据校验失败"
    errors = Object.values(err.errors).map(item => item.message)
  }

  if (err.code === 11000) {
    statusCode = 409
    code = 409
    message = "数据已存在"
  }

  const response = {
    code,
    message
  }

  if (errors) {
    response.errors = errors
  }

  if (process.env.NODE_ENV !== "production") {
    response.stack = err.stack
  }

  res.status(statusCode).json(response)
}

module.exports = errorMiddleware

```

三十六、本课重点

这一课你需要掌握：

1. 日志可以帮助定位线上问题
2. `morgan` 常用于记录 HTTP 请求日志
3. `winston` 常用于记录应用日志和错误日志
4. 日志应该区分级别：
 - `error`
 - `warn`
 - `info`
 - `debug`
5. 生产环境建议写入日志文件
6. 日志文件需要切割，避免无限增大
7. 错误处理中间件中应该记录错误日志
8. 请求日志可以记录接口耗时
9. 慢接口要重点关注
10. 接口调试可以使用：
 - `curl`
 - Postman
 - Apifox
 - VS Code REST Client
11. 性能优化要先测量再优化
12. 列表接口必须分页
13. 查询要只返回需要字段
14. 常用查询字段要加索引
15. 避免同步阻塞代码
16. 可以使用 `Promise.all` 并行查询
17. 可以使用 `compression` 开启 `gzip`
18. 可以用 `helmet` 增加基础安全响应头
19. 生产环境可使用 `PM2 cluster` 提升多核利用率

课后练习

基于你的 MVC 项目完成以下任务：

1. 安装依赖

```
npm install morgan winston winston-daily-rotate-file helmet compression
```

2. 创建日志工具

创建：

```
utils/logger.js
```

要求：

- 开发环境输出到控制台
- 生产环境写入文件
- 错误日志单独保存
- 日志按日期切割

3. 创建请求日志中间件

创建：

```
middlewares/requestLogger.js
```

要求记录：

- 请求方法
- 请求地址
- 状态码
- 请求耗时
- IP 地址

如果接口耗时超过 500ms ，使用 `logger.warn()` 记录。

4. 改造错误处理中间件

要求：

- 发生错误时写入 `error.log`
- 返回统一错误格式
- 生产环境不返回 `stack`

5. 增加性能优化中间件

在 `app.js` 中添加：

```
app.use(helmet())
app.use(compression())
```

并限制 JSON 请求体大小：

```
app.use(express.json({ limit: "1mb" })))
```

6. 创建 api.http 文件

写入：

```
### 健康检查
GET http://localhost:3000/api/health

### 商品列表
GET http://localhost:3000/api/products?page=1&pageSize=10

### 创建商品
POST http://localhost:3000/api/products
Content-Type: application/json

{
  "name": "苹果",
  "price": 5,
  "stock": 100
}
```

下一课建议学习：

Node.js 第十五课：Redis 缓存、限流与验证码基础。

Node.js 第十五课：Redis 缓存、限流与验证码基础

这一课学习后端项目中非常常见的 Redis 应用：

1. Redis 基础
2. Node.js 连接 Redis
3. 接口缓存
4. 登录限流
5. 短信/邮箱验证码基础
6. Redis 常见使用场景

Redis 是后端开发非常重要的工具，常用于提升性能和做临时数据存储。

一、Redis 是什么？

Redis 是一个高性能的内存数据库。

它的数据主要存放在内存中，所以读写速度非常快。

常见用途：

- 缓存接口数据
- 保存验证码
- 保存登录 Token 黑名单
- 限制接口访问频率
- 分布式锁
- 排行榜
- 计数器
- Session 存储

二、Redis 和 MySQL / MongoDB 的区别

对比	Redis	MySQL / MongoDB
存储位置	主要在内存	主要在磁盘
速度	非常快	相对较慢
数据持久性	可持久化，但常做缓存	更适合长期存储
使用场景	缓存、验证码、限流	用户、订单、商品等核心数据
查询能力	简单 key-value	更复杂查询

简单理解：

MySQL / MongoDB：存正式数据
Redis：存临时数据、热点数据

三、Redis 常见数据类型

类型	说明	示例
String	字符串	token、验证码、缓存 JSON
Hash	哈希对象	用户信息
List	列表	消息队列
Set	集合	去重数据
ZSet	有序集合	排行榜

初学阶段最常用的是：

String

比如：

```
key: user:1  
value: {"id":1,"name":"张三"}
```

四、安装 Redis

1. macOS

如果你使用 Homebrew：

```
brew install redis  
brew services start redis
```

测试：

```
redis-cli ping
```

返回：

```
PONG
```

说明 Redis 正常运行。

2. Ubuntu / Debian

```
sudo apt update  
sudo apt install redis-server -y
```

启动：

```
sudo systemctl start redis-server  
sudo systemctl enable redis-server
```

测试：

```
redis-cli ping
```

3. Windows

Windows 推荐使用 Docker 或 WSL。

Docker 方式：

```
docker run -d --name redis -p 6379:6379 redis
```

测试：

```
docker exec -it redis redis-cli ping
```

返回：

```
PONG
```

五、Redis 常用命令

进入 Redis 命令行：

```
redis-cli
```

1. 设置值

```
SET name zhangsan
```

2. 获取值

```
GET name
```

返回：

```
zhangsan
```

3. 设置过期时间

```
SET code 123456 EX 60
```

表示 code 这个 key 60 秒后自动过期。

4. 查看剩余过期时间

```
TTL code
```

5. 删除 key

```
DEL code
```

6. 判断 key 是否存在

```
EXISTS code
```

7. 自增

```
INCR login:fail:admin
```

每执行一次，值加 1。

六、Node.js 连接 Redis

创建项目：

```
mkdir redis-demo  
cd redis-demo  
npm init -y
```

安装依赖：

```
npm install express redis dotenv  
npm install nodemon -D
```

配置 package.json：

```
{  
  "scripts": {  
    "start": "node app.js",  
    "dev": "nodemon app.js"
```

```
}  
}
```

七、配置 .env

.env :

```
PORT=3000  
REDIS_URL=redis://127.0.0.1:6379
```

八、封装 Redis 客户端

项目结构：

```
redis-demo  
├─ app.js  
├─ .env  
├─ config  
  └─ redis.js
```

创建：

```
mkdir config
```

config/redis.js :

```
const { createClient } = require("redis")  
  
const redisClient = createClient({  
  url: process.env.REDIS_URL  
})  
  
redisClient.on("error", err => {  
  console.error("Redis 错误: ", err)  
})  
  
async function connectRedis() {  
  if (!redisClient.isOpen) {  
    await redisClient.connect()  
    console.log("Redis 连接成功")  
  }  
}  
  
module.exports = {  
  redisClient,
```

```
    connectRedis
  }
}
```

九、测试 Redis 读写

app.js :

```
require("dotenv").config()

const express = require("express")
const { redisClient, connectRedis } = require("./config/redis")

const app = express()

app.use(express.json())

app.get("/", (req, res) => {
  res.send("Redis Demo")
})

app.get("/set", async (req, res) => {
  await redisClient.set("name", "张三")

  res.json({
    code: 0,
    message: "设置成功"
  })
})

app.get("/get", async (req, res) => {
  const name = await redisClient.get("name")

  res.json({
    code: 0,
    data: name
  })
})

const PORT = process.env.PORT || 3000

async function start() {
  await connectRedis()

  app.listen(PORT, () => {
    console.log(`服务器启动: http://localhost:${PORT}`)
  })
}

start()
```

启动:

```
npm run dev
```

访问：

```
http://localhost:3000/set  
http://localhost:3000/get
```

十、Redis 存储 JSON 数据

Redis 的 value 通常是字符串。

如果要存对象，需要：

```
JSON.stringify()
```

取出来后需要：

```
JSON.parse()
```

示例：

```
app.get("/set-user", async (req, res) => {  
  const user = {  
    id: 1,  
    name: "张三",  
    age: 20  
  }  
  
  await redisClient.set("user:1", JSON.stringify(user))  
  
  res.json({  
    code: 0,  
    message: "用户缓存成功"  
  })  
})  
  
app.get("/get-user", async (req, res) => {  
  const value = await redisClient.get("user:1")  
  
  const user = value ? JSON.parse(value) : null  
  
  res.json({  
    code: 0,  
    data: user  
  })  
})
```

十一、设置缓存过期时间

缓存不能一直存在，否则数据可能过期或占用内存。

设置 60 秒过期：

```
await redisClient.set("user:1", JSON.stringify(user), {
  EX: 60
})
```

说明：

```
EX: 60
```

表示 60 秒后过期。

十二、接口缓存基础

假设商品列表查询数据库比较慢。

没有缓存时：

```
请求 -> 查询数据库 -> 返回数据
```

有缓存后：

```
第一次请求 -> 查询数据库 -> 保存到 Redis -> 返回数据
第二次请求 -> 直接读 Redis -> 返回数据
```

十三、模拟商品列表缓存

示例数据：

```
const products = [
  { id: 1, name: "苹果", price: 5 },
  { id: 2, name: "香蕉", price: 3 },
  { id: 3, name: "橙子", price: 4 }
]
```

接口：

```
app.get("/api/products", async (req, res) => {
  const cacheKey = "products:list"

  const cacheData = await redisClient.get(cacheKey)
```



```

if (cacheData) {
  return res.json({
    code: 0,
    message: "获取成功",
    cache: true,
    data: JSON.parse(cacheData)
  })
}

// 模拟数据库查询
const data = products

await redisClient.set(cacheKey, JSON.stringify(data), {
  EX: 60
})

res.json({
  code: 0,
  message: "获取成功",
  cache: false,
  data
})
})

```

第一次访问：

```

{
  "cache": false
}

```

第二次访问：

```

{
  "cache": true
}

```

十四、封装缓存中间件

可以把缓存逻辑封装成中间件。

新建：

middlewares/cacheMiddleware.js

```

const { redisClient } = require("../config/redis")

function cacheMiddleware(prefix, expireSeconds = 60) {
  return async function (req, res, next) {
    const cacheKey = `${prefix}:${req.originalUrl}`

```

```

const cacheData = await redisClient.get(cacheKey)

if (cacheData) {
  return res.json({
    code: 0,
    message: "获取成功",
    cache: true,
    data: JSON.parse(cacheData)
  })
}

res.sendResponse = res.json

res.json = async function (body) {
  if (body && body.code === 0) {
    await redisClient.set(cacheKey, JSON.stringify(body.data), {
      EX: expireSeconds
    })
  }

  res.sendResponse(body)
}

next()
}
}

module.exports = cacheMiddleware

```

使用：

```

const cacheMiddleware = require("../middlewares/cacheMiddleware")

app.get(
  "/api/products",
  cacheMiddleware("products", 60),
  async (req, res) => {
    res.json({
      code: 0,
      message: "获取成功",
      data: products
    })
  }
)

```

十五、缓存更新问题

缓存虽然能提升速度，但也会带来一个问题：

数据库数据变了，但 Redis 里还是旧数据

解决方式常见有：

1. 设置较短过期时间

例如缓存 60 秒：

```
EX: 60
```

简单，但可能短时间内读到旧数据。

2. 修改数据后删除缓存

例如新增商品后：

```
await redisClient.del("products:list")
```

这样下次查询会重新查数据库并写入缓存。

3. 使用更精确的缓存 key

分页时，缓存 key 要包含 query 参数：

```
products:/api/products?page=1&pageSize=10  
products:/api/products?page=2&pageSize=10
```

十六、接口限流基础

限流是为了防止接口被频繁请求。

常见场景：

- 登录接口防暴力破解
- 验证码接口防刷
- 注册接口防刷
- 短信接口防刷
- API 防恶意请求

例如：

```
同一个 IP 1 分钟最多请求 10 次
```

超过就返回：

```
{
  "code": 429,
  "message": "请求过于频繁，请稍后再试"
}
```

HTTP 状态码：

429 Too Many Requests

十七、使用 Redis 实现简单限流

思路：

每个 IP + 接口生成一个 key
每请求一次 INCR
第一次请求时设置过期时间
超过限制就拒绝

新建：

middlewares/rateLimitMiddleware.js

```
const { redisClient } = require("../config/redis")

function rateLimit(options = {}) {
  const {
    windowSeconds = 60,
    max = 10,
    prefix = "rate_limit"
  } = options

  return async function (req, res, next) {
    const ip = req.ip || req.connection.remoteAddress
    const key = `${prefix}:${ip}:${req.path}`

    const count = await redisClient.incr(key)

    if (count === 1) {
      await redisClient.expire(key, windowSeconds)
    }

    if (count > max) {
      return res.status(429).json({
        code: 429,
        message: "请求过于频繁，请稍后再试"
      })
    }

    next()
  }
}
```

```
    }  
  }  
  
  module.exports = rateLimit
```

使用：

```
const rateLimit = require("../middlewares/rateLimitMiddleware")  
  
app.post(  
  "/api/auth/login",  
  rateLimit({  
    windowSeconds: 60,  
    max: 5,  
    prefix: "login_limit"  
  }),  
  async (req, res) => {  
    res.json({  
      code: 0,  
      message: "登录接口"  
    })  
  }  
)
```

表示：

同一个 IP 对 /api/auth/login 1 分钟最多请求 5 次

十八、注意 Nginx 代理下的真实 IP

如果项目部署在 Nginx 后面，`req.ip` 可能是：

127.0.0.1

需要在 Express 中配置：

```
app.set("trust proxy", true)
```

Nginx 配置也要有：

```
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

十九、验证码基础

验证码常见类型：

类型	说明
图片验证码	图形验证码
短信验证码	发送到手机号
邮箱验证码	发送到邮箱
登录验证码	防暴力登录
注册验证码	验证手机号或邮箱

Redis 很适合保存验证码，因为验证码通常只需要短时间有效。

例如：

```
key: verify:email:test@example.com
value: 482913
expire: 300 秒
```

二十、生成 6 位数字验证码

工具函数：

```
function generateCode(length = 6) {
  let code = ""

  for (let i = 0; i < length; i++) {
    code += Math.floor(Math.random() * 10)
  }

  return code
}

module.exports = generateCode
```

或者简单写法：

```
const code = Math.floor(100000 + Math.random() * 900000).toString()
```

二十一、发送邮箱验证码示例

这里先不真正接入邮箱服务，只模拟发送。

接口：

POST /api/code/email

请求体：

```
{
  "email": "test@example.com"
}
```

代码：

```
app.post("/api/code/email", async (req, res) => {
  const { email } = req.body

  if (!email) {
    return res.status(400).json({
      code: 400,
      message: "邮箱不能为空"
    })
  }

  const code = Math.floor(100000 + Math.random() * 900000).toString()

  const key = `verify:email:${email}`

  await redisClient.set(key, code, {
    EX: 5 * 60
  })

  console.log(`发送给 ${email} 的验证码是: ${code}`)

  res.json({
    code: 0,
    message: "验证码已发送, 有效期 5 分钟"
  })
})
```

测试：

```
curl -X POST http://localhost:3000/api/code/email \
-H "Content-Type: application/json" \
-d '{"email": "test@example.com"}'
```

控制台会输出验证码。

二十二、校验邮箱验证码

接口：

POST /api/code/verify-email

请求体:

```
{  
  "email": "test@example.com",  
  "code": "123456"  
}
```

代码:

```
app.post("/api/code/verify-email", async (req, res) => {  
  const { email, code } = req.body  
  
  if (!email || !code) {  
    return res.status(400).json({  
      code: 400,  
      message: "邮箱和验证码不能为空"  
    })  
  }  
  
  const key = `verify:email:${email}`  
  
  const savedCode = await redisClient.get(key)  
  
  if (!savedCode) {  
    return res.status(400).json({  
      code: 400,  
      message: "验证码不存在或已过期"  
    })  
  }  
  
  if (savedCode !== code) {  
    return res.status(400).json({  
      code: 400,  
      message: "验证码错误"  
    })  
  }  
  
  await redisClient.del(key)  
  
  res.json({  
    code: 0,  
    message: "验证码校验成功"  
  })  
})
```

注意:

```
await redisClient.del(key)
```

验证码验证成功后要删除, 防止重复使用。

二十三、验证码接口也要限流

验证码接口很容易被刷。

例如：

```
app.post(  
  "/api/code/email",  
  rateLimit({  
    windowSeconds: 60,  
    max: 3,  
    prefix: "email_code_limit"  
  }),  
  async (req, res) => {  
    // 发送验证码逻辑  
  }  
)
```

表示：

同一个 IP 1 分钟最多请求 3 次邮箱验证码

还可以针对邮箱限流：

```
const emailKey = `email_code_cooldown:${email}`  
const exists = await redisClient.exists(emailKey)  
  
if (exists) {  
  return res.status(429).json({  
    code: 429,  
    message: "验证码发送太频繁，请稍后再试"  
  })  
}  
  
await redisClient.set(emailKey, "1", {  
  EX: 60  
})
```

表示：

同一个邮箱 60 秒只能发送一次验证码

二十四、完整验证码发送接口

```
app.post(  
  "/api/code/email",  
  rateLimit({
```

```

windowSeconds: 60,
max: 5,
prefix: "email_code_ip_limit"
}),
async (req, res) => {
  const { email } = req.body

  if (!email) {
    return res.status(400).json({
      code: 400,
      message: "邮箱不能为空"
    })
  }

  const cooldownKey = `email_code_cooldown:${email}`

  const inCooldown = await redisClient.exists(cooldownKey)

  if (inCooldown) {
    return res.status(429).json({
      code: 429,
      message: "验证码发送太频繁, 请 60 秒后再试"
    })
  }

  const code = Math.floor(100000 + Math.random() * 900000).toString()

  await redisClient.set(`verify:email:${email}`, code, {
    EX: 5 * 60
  })

  await redisClient.set(cooldownKey, "1", {
    EX: 60
  })

  console.log(`发送给 ${email} 的验证码是: ${code}`)

  res.json({
    code: 0,
    message: "验证码已发送, 有效期 5 分钟"
  })
}
)

```

二十五、结合注册使用验证码

注册请求:

```

{
  "username": "admin",
  "email": "test@example.com",
  "password": "123456",

```

```
"code": "482913"
}
```

注册逻辑中先校验验证码：

```
const savedCode = await redisClient.get(`verify:email:${email}`)

if (!savedCode) {
  return res.status(400).json({
    code: 400,
    message: "验证码不存在或已过期"
  })
}

if (savedCode !== code) {
  return res.status(400).json({
    code: 400,
    message: "验证码错误"
  })
}

await redisClient.del(`verify:email:${email}`)

// 继续注册逻辑
```

流程：

1. 用户请求发送验证码
2. 后端生成验证码，保存到 Redis，设置 5 分钟过期
3. 用户提交注册信息和验证码
4. 后端从 Redis 取验证码
5. 校验成功后删除验证码
6. 创建用户

二十六、JWT 黑名单：退出登录进阶

JWT 默认是无状态的。

用户退出登录时，前端删除 Token 即可：

```
localStorage.removeItem("token")
```

但如果想让 Token 立刻失效，可以把 Token 加入 Redis 黑名单。

1. 退出登录时加入黑名单

```
app.post("/api/logout", authMiddleware, async (req, res) => {
  const token = req.token

  await redisClient.set(`jwt:blacklist:${token}`, "1", {
    EX: 2 * 60 * 60
  })

  res.json({
    code: 0,
    message: "退出成功"
  })
})
```

2. 鉴权中间件中检查黑名单

```
const isBlacklisted = await redisClient.exists(`jwt:blacklist:${token}`)

if (isBlacklisted) {
  return res.status(401).json({
    code: 401,
    message: "Token 已失效"
  })
}
```

注意：黑名单的过期时间应该和 JWT 剩余有效时间一致，避免 Redis 存太久。

二十七、Redis 在 MVC 项目中的结构

推荐结构：

```
node-mvc-demo
├── config
│   └── redis.js
├── middlewares
│   ├── cacheMiddleware.js
│   └── rateLimitMiddleware.js
├── utils
│   └── generateCode.js
├── controllers
│   └── codeController.js
└── routes
    └── codeRoutes.js
```

二十八、完整 app.js 示例

```
require("dotenv").config()

const express = require("express")
const { connectRedis, redisClient } = require("./config/redis")
const rateLimit = require("./middlewares/rateLimitMiddleware")

const app = express()

app.set("trust proxy", true)

app.use(express.json())

app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok"
  })
})

const products = [
  { id: 1, name: "苹果", price: 5 },
  { id: 2, name: "香蕉", price: 3 }
]

app.get("/api/products", async (req, res) => {
  const cacheKey = "products:list"

  const cacheData = await redisClient.get(cacheKey)

  if (cacheData) {
    return res.json({
      code: 0,
      message: "获取成功",
      cache: true,
      data: JSON.parse(cacheData)
    })
  }

  await redisClient.set(cacheKey, JSON.stringify(products), {
    EX: 60
  })

  res.json({
    code: 0,
    message: "获取成功",
    cache: false,
    data: products
  })
})

app.post(
  "/api/code/email",
  rateLimit({
    windowSeconds: 60,
    max: 5,
    prefix: "email_code_ip_limit"
  })
)
```

```

    }},
    async (req, res) => {
      const { email } = req.body

      if (!email) {
        return res.status(400).json({
          code: 400,
          message: "邮箱不能为空"
        })
      }

      const cooldownKey = `email_code_cooldown:${email}`

      const inCooldown = await redisClient.exists(cooldownKey)

      if (inCooldown) {
        return res.status(429).json({
          code: 429,
          message: "验证码发送太频繁, 请 60 秒后再试"
        })
      }

      const code = Math.floor(100000 + Math.random() * 900000).toString()

      await redisClient.set(`verify:email:${email}`, code, {
        EX: 5 * 60
      })

      await redisClient.set(cooldownKey, "1", {
        EX: 60
      })

      console.log(`发送给 ${email} 的验证码是: ${code}`)

      res.json({
        code: 0,
        message: "验证码已发送, 有效期 5 分钟"
      })
    }
  )

  app.post("/api/code/verify-email", async (req, res) => {
    const { email, code } = req.body

    const savedCode = await redisClient.get(`verify:email:${email}`)

    if (!savedCode) {
      return res.status(400).json({
        code: 400,
        message: "验证码不存在或已过期"
      })
    }

    if (savedCode !== code) {
      return res.status(400).json({
        code: 400,
        message: "验证码错误"
      })
    }
  })
}

```

```
    })  
  }  
  
  await redisClient.del(`verify:email:${email}`)  
  
  res.json({  
    code: 0,  
    message: "验证码校验成功"  
  })  
})  
  
async function start() {  
  await connectRedis()  
  
  const PORT = process.env.PORT || 3000  
  
  app.listen(PORT, () => {  
    console.log(`服务器启动: http://localhost:${PORT}`)  
  })  
}  
  
start()
```

二十九、Redis key 命名规范

建议使用清晰的 key 名称。

常见格式：

模块:业务:标识

例如：

```
user:profile:123  
products:list  
verify:email:test@example.com  
rate_limit:127.0.0.1:/api/login  
jwt:blacklist:token
```

好处：

- 易读
- 易管理
- 易删除
- 易排查问题

三十、Redis 使用注意事项

1. 不要缓存敏感信息

不建议直接缓存：

- 明文密码
 - 身份证号
 - 银行卡号
 - 私密 Token
-

2. 缓存要设置过期时间

不推荐：

```
await redisClient.set("products:list", data)
```

推荐：

```
await redisClient.set("products:list", data, {  
  EX: 60  
})
```

3. 缓存数据要考虑更新

如果数据库修改了，记得删除相关缓存。

```
await redisClient.del("products:list")
```

4. 验证码验证成功后要删除

```
await redisClient.del(`verify:email:${email}`)
```

防止重复使用。

5. 限流要结合 IP 和业务标识

例如：

```
rate_limit:ip:path
```


或者：

```
rate_limit:userId:action
```

三十一、常见错误

1. Redis 连接失败

错误：

```
ECONNREFUSED 127.0.0.1:6379
```

原因：

- Redis 没启动
- 地址错了
- 端口错了

检查：

```
redis-cli ping
```

2. JSON.parse 报错

如果 Redis 里存的不是 JSON 字符串：

```
JSON.parse(value)
```

会报错。

建议：

```
let data = null

try {
  data = JSON.parse(value)
} catch (err) {
  data = value
}
```

3. 缓存 key 太粗导致数据错乱

错误示例：

```
const cacheKey = "products:list"
```

如果接口支持分页：

```
/api/products?page=1  
/api/products?page=2
```

都用同一个 key，就会错乱。

推荐：

```
const cacheKey = `products:list:${req.originalUrl}`
```

4. 限流误伤所有用户

如果部署在 Nginx 后面，但没有配置真实 IP，所有请求 IP 都可能是：

```
127.0.0.1
```

解决：

```
app.set("trust proxy", true)
```

并配置 Nginx 请求头。

本课重点

这一课你需要掌握：

1. Redis 是高性能内存数据库
2. Redis 常用于缓存、验证码、限流、Token 黑名单
3. Node.js 使用 `redis` 包连接 Redis
4. Redis 存对象要用 `JSON.stringify`
5. 取对象要用 `JSON.parse`
6. 缓存要设置过期时间
7. 修改数据库后应删除相关缓存
8. 限流可以用 `INCR + EXPIRE` 实现
9. 验证码适合存 Redis，并设置短过期时间
10. 验证码验证成功后要删除
11. 验证码接口必须限流
12. Redis key 命名要规范

课后练习

基于你的 MVC 项目，完成以下功能：

1. 安装 Redis 客户端

```
npm install redis
```

2. 创建 Redis 配置

```
config/redis.js
```

要求：

- 使用 `.env` 中的 `REDIS_URL`
- 项目启动时连接 Redis

3. 给商品列表加缓存

接口：

```
GET /api/products?page=1&pageSize=10
```

要求：

- 第一次查数据库
- 第二次读 Redis
- 缓存 60 秒
- 新增、修改、删除商品后清除商品列表缓存

4. 给登录接口加限流

要求：

```
同一个 IP 1 分钟最多登录 5 次
```

超过返回：

```
{
  "code": 429,
  "message": "请求过于频繁，请稍后再试"
}
```

5. 实现邮箱验证码

接口：

方法	路径	功能
POST	/api/code/email	发送验证码
POST	/api/code/verify-email	校验验证码

要求：

- 验证码 6 位数字
- 有效期 5 分钟
- 同一个邮箱 60 秒只能发送一次
- 验证成功后删除验证码

下一课建议学习：

Node.js 第十六课：单元测试、接口测试与 Jest 入门。

Node.js 第十六课：单元测试、接口测试与 Jest 入门

这一课学习 Node.js 项目中非常重要但很多初学者容易忽略的内容：

1. 什么是测试
2. 为什么要写测试
3. Jest 入门
4. 单元测试
5. 异步函数测试
6. Express 接口测试
7. 使用 Supertest 测试 API
8. 测试数据库相关逻辑
9. 在 MVC 项目中组织测试代码

一、为什么需要测试？

很多初学者写完代码后，只会手动打开浏览器或 Apifox 测试接口。

比如：

注册接口能不能用？
登录接口能不能用？

商品列表有没有返回？
删除接口会不会报错？

手动测试的问题是：

- 很耗时间
- 容易漏测
- 每次改代码都要重新点一遍
- 项目大了以后很难保证旧功能不坏
- 上线前没有信心

自动化测试可以帮你自动检查代码是否正常。

例如你写好测试后，只需要运行：

```
npm test
```

它会自动帮你测试：

```
注册是否成功  
登录是否成功  
参数错误是否返回 400  
商品接口是否返回数组  
Token 错误是否返回 401
```

二、测试分为哪些类型？

常见测试类型：

类型	说明
单元测试	测试一个函数或一个模块
接口测试	测试 HTTP API
集成测试	测试多个模块组合是否正常
端到端测试	模拟真实用户完整操作流程
性能测试	测试接口并发和响应速度

本课重点学习：

单元测试 + 接口测试

三、什么是 Jest？

Jest 是 JavaScript 生态中非常常用的测试框架。

它可以用于：

- Node.js 后端测试
- React 前端测试
- 工具函数测试
- 异步函数测试
- Mock 模拟测试

安装：

```
npm install jest -D
```

查看版本：

```
npx jest --version
```

四、创建测试项目

创建项目：

```
mkdir jest-demo  
cd jest-demo  
npm init -y
```

安装 Jest：

```
npm install jest -D
```

修改 package.json：

```
{  
  "scripts": {  
    "test": "jest"  
  }  
}
```

项目结构：

```
jest-demo  
├─ package.json  
├─ src  
│   └─ math.js  
└─ tests  
    └─ math.test.js
```

创建目录：

```
mkdir src tests
```

五、第一个单元测试

1. 写一个函数

src/math.js：

```
function add(a, b) {  
  return a + b  
}  
  
function minus(a, b) {  
  return a - b  
}  
  
module.exports = {  
  add,  
  minus  
}
```

2. 写测试文件

tests/math.test.js：

```
const { add, minus } = require("../src/math")  
  
test("add: 1 + 2 应该等于 3", () => {  
  expect(add(1, 2)).toBe(3)  
})  
  
test("minus: 5 - 2 应该等于 3", () => {  
  expect(minus(5, 2)).toBe(3)  
})
```

3. 运行测试

```
npm test
```

输出类似：

```
PASS tests/math.test.js
✓ add: 1 + 2 应该等于 3
✓ minus: 5 - 2 应该等于 3
```

说明测试通过。

六、test 和 expect 是什么？

1. test()

`test()` 用来定义一个测试用例。

```
test("测试描述", () => {
  // 测试代码
})
```

例如：

```
test("1 + 1 等于 2", () => {
  expect(1 + 1).toBe(2)
})
```

2. expect()

`expect()` 用来断言结果。

```
expect(实际结果).匹配器(期望结果)
```

例如：

```
expect(2 + 2).toBe(4)
```

意思是：

```
我期望 2 + 2 的结果是 4
```

七、常用匹配器 matcher

1. toBe

用于比较基本类型：


```
expect(1 + 1).toBe(2)
expect("hello").toBe("hello")
expect(true).toBe(true)
```

2. toEqual

用于比较对象或数组：

```
expect({ name: "张三" }).toEqual({ name: "张三" })

expect([1, 2, 3]).toEqual([1, 2, 3])
```

注意：

```
expect({ name: "张三" }).toBe({ name: "张三" })
```

通常会失败，因为两个对象不是同一个引用。

对象和数组一般用：

```
toEqual()
```

3. toContain

判断数组或字符串是否包含某个值：

```
expect([1, 2, 3]).toContain(2)

expect("hello node").toContain("node")
```

4. toBeTruthy / toBeFalsy

判断真假值：

```
expect(1).toBeTruthy()
expect("").toBeFalsy()
```

5. toBeNull / toBeUndefined

```
expect(null).toBeNull()
expect(undefined).toBeUndefined()
```

6. toHaveProperty

判断对象是否有某个属性：

```
const user = {
  id: 1,
  username: "admin"
}

expect(user).toHaveProperty("id")
expect(user).toHaveProperty("username", "admin")
```

7. toMatch

匹配正则：

```
expect("test@example.com").toMatch(/@/)
```

8. toThrow

测试函数是否抛出错误：

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("除数不能为 0")
  }

  return a / b
}

test("除数为 0 时应该抛错", () => {
  expect(() => divide(10, 0)).toThrow("除数不能为 0")
})
```

八、describe 分组测试

当测试用例越来越多，可以使用 `describe()` 分组。

```
const { add, minus } = require("../src/math")

describe("math 工具函数", () => {
  test("add", () => {
    expect(add(1, 2)).toBe(3)
  })

  test("minus", () => {
    expect(minus(5, 2)).toBe(3)
  })
})
```

输出会更清晰。

九、测试异步函数

真实项目中很多函数都是异步的，比如数据库查询、Redis 读取、接口请求。

1. 被测试函数

src/userService.js :

```
function getUserById(id) {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve({
        id,
        username: "admin"
      })
    }, 100)
  })
}

module.exports = {
  getUserById
}
```

2. 使用 async/await 测试

tests/userService.test.js :

```
const { getUserById } = require("../src/userService")

test("根据 id 获取用户", async () => {
  const user = await getUserById(1)
```

```
expect(user).toEqual({
  id: 1,
  username: "admin"
})
})
```

十、测试 Promise reject

src/authService.js :

```
async function login(username, password) {
  if (username !== "admin" || password !== "123456") {
    throw new Error("用户名或密码错误")
  }

  return {
    token: "mock-token",
    user: {
      id: 1,
      username: "admin"
    }
  }
}

module.exports = {
  login
}
```

测试:

```
const { login } = require("../src/authService")

test("登录成功", async () => {
  const result = await login("admin", "123456")

  expect(result).toHaveProperty("token")
  expect(result.user.username).toBe("admin")
})

test("登录失败", async () => {
  await expect(login("admin", "wrong")).rejects.toThrow("用户名或密码错误")
})
```

十一、beforeEach 和 afterEach

有时每个测试前都要准备数据。

```
let users = []

beforeEach(() => {
  users = [
    { id: 1, username: "admin" },
    { id: 2, username: "test" }
  ]
})

afterEach(() => {
  users = []
})

test("用户数量应该是 2", () => {
  expect(users.length).toBe(2)
})
```

常用钩子：

方法	说明
beforeAll	所有测试开始前执行一次
afterAll	所有测试结束后执行一次
beforeEach	每个测试前执行
afterEach	每个测试后执行

十二、测试 Express 接口：Supertest

测试 Express API 常用：

```
supertest
```

安装：

```
npm install supertest -D
```

十三、准备 Express 项目

项目结构：

```
express-test-demo
├─ app.js
├─ server.js
└─ package.json
```

```
└─ tests
   └─ app.test.js
```

安装依赖：

```
npm install express
npm install jest supertest -D
```

1. app.js

注意：为了方便测试，app.js 只导出 app，不直接监听端口。

```
const express = require("express")

const app = express()

app.use(express.json())

const users = [
  { id: 1, username: "admin" },
  { id: 2, username: "test" }
]

app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok"
  })
})

app.get("/api/users", (req, res) => {
  res.json({
    code: 0,
    data: users
  })
})

app.post("/api/users", (req, res) => {
  const { username } = req.body

  if (!username) {
    return res.status(400).json({
      code: 400,
      message: "用户名不能为空"
    })
  }

  const newUser = {
    id: users.length + 1,
    username
  }
})
```

```
users.push(newUser)

res.status(201).json({
  code: 0,
  message: "创建成功",
  data: newUser
})
})

module.exports = app
```

2. server.js

```
const app = require("./app")

const PORT = process.env.PORT || 3000

app.listen(PORT, () => {
  console.log(`服务器启动: http://localhost:${PORT}`)
})
```

为什么要拆成两个文件？

app.js: 用于配置 Express，并导出 app，方便测试
server.js: 用于真正启动服务

测试时不需要真的监听端口。

十四、写接口测试

tests/app.test.js：

```
const request = require("supertest")
const app = require("../app")

describe("基础接口测试", () => {
  test("GET /api/health 应该返回 ok", async () => {
    const res = await request(app).get("/api/health")

    expect(res.statusCode).toBe(200)
    expect(res.body.code).toBe(0)
    expect(res.body.message).toBe("ok")
  })

  test("GET /api/users 应该返回用户数组", async () => {
    const res = await request(app).get("/api/users")

    expect(res.statusCode).toBe(200)
```

```

    expect(res.body.code).toBe(0)
    expect(Array.isArray(res.body.data)).toBe(true)
  })

  test("POST /api/users 创建用户成功", async () => {
    const res = await request(app)
      .post("/api/users")
      .send({
        username: "zhangsan"
      })

    expect(res.statusCode).toBe(201)
    expect(res.body.code).toBe(0)
    expect(res.body.data.username).toBe("zhangsan")
  })

  test("POST /api/users 参数错误", async () => {
    const res = await request(app)
      .post("/api/users")
      .send({})

    expect(res.statusCode).toBe(400)
    expect(res.body.code).toBe(400)
    expect(res.body.message).toBe("用户名不能为空")
  })
})

```

运行：

```
npm test
```

十五、测试登录和 Token 接口

假设 app.js 中有登录接口：

```

app.post("/api/auth/login", (req, res) => {
  const { username, password } = req.body

  if (username !== "admin" || password !== "123456") {
    return res.status(401).json({
      code: 401,
      message: "用户名或密码错误"
    })
  }

  res.json({
    code: 0,
    message: "登录成功",
    data: {
      token: "mock-token"
    }
  })
})

```



```

}))

function auth(req, res, next) {
  const authorization = req.headers.authorization

  if (authorization !== "Bearer mock-token") {
    return res.status(401).json({
      code: 401,
      message: "未授权"
    })
  }

  next()
}

app.get("/api/profile", auth, (req, res) => {
  res.json({
    code: 0,
    data: {
      username: "admin"
    }
  })
})
})

```

测试:

```

const request = require("supertest")
const app = require("../app")

describe("登录鉴权接口测试", () => {
  test("登录成功后返回 token", async () => {
    const res = await request(app)
      .post("/api/auth/login")
      .send({
        username: "admin",
        password: "123456"
      })

    expect(res.statusCode).toBe(200)
    expect(res.body.data).toHaveProperty("token")
  })

  test("登录失败返回 401", async () => {
    const res = await request(app)
      .post("/api/auth/login")
      .send({
        username: "admin",
        password: "wrong"
      })

    expect(res.statusCode).toBe(401)
    expect(res.body.message).toBe("用户名或密码错误")
  })

  test("不带 token 访问 profile 返回 401", async () => {
    const res = await request(app).get("/api/profile")

```

```

    expect(res.statusCode).toBe(401)
  })

  test("带 token 访问 profile 成功", async () => {
    const loginRes = await request(app)
      .post("/api/auth/login")
      .send({
        username: "admin",
        password: "123456"
      })

    const token = loginRes.body.data.token

    const profileRes = await request(app)
      .get("/api/profile")
      .set("Authorization", `Bearer ${token}`)

    expect(profileRes.statusCode).toBe(200)
    expect(profileRes.body.data.username).toBe("admin")
  })
})

```

十六、在 MVC 项目中添加测试

上一课我们的 MVC 项目结构大概是：

```

node-mvc-demo
├─ app.js
├─ server.js
├─ controllers
├─ models
├─ routes
├─ middlewares
├─ utils
└─ tests

```

推荐测试目录：

```

tests
├─ unit
│   ├── response.test.js
│   └─ generateToken.test.js
└─ integration
    ├── auth.test.js
    ├── product.test.js
    └─ user.test.js

```

说明：

目录	作用
unit	单元测试
integration	接口/集成测试

十七、测试工具函数

例如测试验证码生成函数。

utils/generateCode.js：

```
function generateCode(length = 6) {
  let code = ""

  for (let i = 0; i < length; i++) {
    code += Math.floor(Math.random() * 10)
  }

  return code
}

module.exports = generateCode
```

测试：

tests/unit/generateCode.test.js：

```
const generateCode = require("../utils/generateCode")

describe("generateCode", () => {
  test("默认生成 6 位验证码", () => {
    const code = generateCode()

    expect(code).toHaveLength(6)
    expect(code).toMatch(/^d{6}$/)
  })

  test("可以生成指定长度验证码", () => {
    const code = generateCode(4)

    expect(code).toHaveLength(4)
    expect(code).toMatch(/^d{4}$/)
  })
})
```

十八、测试 JWT 工具函数

utils/generateToken.js :

```
const jwt = require("jsonwebtoken")

function generateToken(user) {
  return jwt.sign(
    {
      id: user.id,
      username: user.username
    },
    process.env.JWT_SECRET,
    {
      expiresIn: "1h"
    }
  )
}

module.exports = generateToken
```

测试:

```
const jwt = require("jsonwebtoken")
const generateToken = require("../utils/generateToken")

describe("generateToken", () => {
  beforeAll(() => {
    process.env.JWT_SECRET = "test_secret"
  })

  test("应该生成有效 JWT", () => {
    const token = generateToken({
      id: "1",
      username: "admin"
    })

    const decoded = jwt.verify(token, process.env.JWT_SECRET)

    expect(decoded.id).toBe("1")
    expect(decoded.username).toBe("admin")
  })
})
```

十九、测试带数据库的接口

如果你的项目使用 MongoDB，测试时建议使用单独的测试数据库。

.env.test :

```
NODE_ENV=test
PORT=3000
MONGO_URL=mongodb://127.0.0.1:27017/node_mvc_demo_test
```

```
JWT_SECRET=test_secret
JWT_EXPIRES_IN=1h
```

不要用生产数据库做测试。

二十、配置 Jest 测试环境变量

安装：

```
npm install dotenv -D
```

创建：

```
jest.setup.js
```

```
require("dotenv").config({
  path: ".env.test"
})
```

创建：

```
jest.config.js
```

```
module.exports = {
  testEnvironment: "node",
  setupFiles: [<rootDir>/jest.setup.js],
  testMatch: ["**/tests/**/*.test.js"],
  verbose: true
}
```

修改 package.json：

```
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  }
}
```

二十一、测试 MongoDB 接口

以注册登录接口为例。

tests/integration/auth.test.js :

```
const request = require("supertest")
const mongoose = require("mongoose")
const app = require("../..//app")
const User = require("../..//models/User")

beforeAll(async () => {
  await mongoose.connect(process.env.MONGO_URL)
})

afterEach(async () => {
  await User.deleteMany({})
})

afterAll(async () => {
  await mongoose.connection.close()
})

describe("Auth 接口测试", () => {
  test("注册成功", async () => {
    const res = await request(app)
      .post("/api/auth/register")
      .send({
        username: "admin",
        password: "123456",
        nickname: "管理员"
      })

    expect(res.statusCode).toBe(200)
    expect(res.body.code).toBe(0)
    expect(res.body.data.username).toBe("admin")
  })

  test("重复注册应该失败", async () => {
    await User.create({
      username: "admin",
      password: "hashed-password"
    })

    const res = await request(app)
      .post("/api/auth/register")
      .send({
        username: "admin",
        password: "123456"
      })

    expect(res.statusCode).toBe(409)
    expect(res.body.message).toBe("用户名已存在")
  })

  test("登录成功", async () => {
    await request(app)
      .post("/api/auth/register")
      .send({
        username: "admin",
```

```

        password: "123456"
    })

    const res = await request(app)
    .post("/api/auth/login")
    .send({
        username: "admin",
        password: "123456"
    })

    expect(res.statusCode).toBe(200)
    expect(res.body.data).toHaveProperty("token")
})

test("登录密码错误", async () => {
    await request(app)
    .post("/api/auth/register")
    .send({
        username: "admin",
        password: "123456"
    })

    const res = await request(app)
    .post("/api/auth/login")
    .send({
        username: "admin",
        password: "wrong"
    })

    expect(res.statusCode).toBe(401)
    expect(res.body.message).toBe("用户名或密码错误")
})
})

```

二十二、注意 app.js 不要自动连接数据库

为了测试方便，建议：

app.js

只负责 Express 配置：

```

const express = require("express")

const app = express()

app.use(express.json())

// routes...

module.exports = app

```

server.js

负责连接数据库和启动服务：

```
require("dotenv").config()

const app = require("./app")
const connectDB = require("./config/db")

async function start() {
  await connectDB()

  const PORT = process.env.PORT || 3000

  app.listen(PORT, () => {
    console.log(`服务器启动: http://localhost:${PORT}`)
  })
}

start()
```

测试文件中自己连接测试数据库：

```
beforeAll(async () => {
  await mongoose.connect(process.env.MONGO_URL)
})
```

这样测试更可控。

二十三、测试商品接口

假设商品接口：

```
GET /api/products
POST /api/products
GET /api/products/:id
PUT /api/products/:id
DELETE /api/products/:id
```

测试文件：

tests/integration/product.test.js：

```
const request = require("supertest")
const mongoose = require("mongoose")
const app = require("../..app")
const Product = require("../..models/Product")

beforeAll(async () => {
  await mongoose.connect(process.env.MONGO_URL)
```



```
})

afterEach(async () => {
  await Product.deleteMany({})
})

afterAll(async () => {
  await mongoose.connection.close()
})

describe("Product 接口测试", () => {
  test("创建商品成功", async () => {
    const res = await request(app)
      .post("/api/products")
      .send({
        name: "苹果",
        price: 5,
        stock: 100
      })

    expect(res.statusCode).toBe(201)
    expect(res.body.code).toBe(0)
    expect(res.body.data.name).toBe("苹果")
  })

  test("创建商品参数错误", async () => {
    const res = await request(app)
      .post("/api/products")
      .send({
        name: "",
        price: -1
      })

    expect(res.statusCode).toBe(400)
    expect(res.body.message).toBe("参数校验失败")
    expect(Array.isArray(res.body.errors)).toBe(true)
  })

  test("获取商品列表", async () => {
    await Product.create({
      name: "苹果",
      price: 5,
      stock: 100
    })

    const res = await request(app)
      .get("/api/products?page=1&pageSize=10")

    expect(res.statusCode).toBe(200)
    expect(res.body.code).toBe(0)
    expect(Array.isArray(res.body.data.list)).toBe(true)
    expect(res.body.data.pagination.total).toBe(1)
  })

  test("获取商品详情", async () => {
    const product = await Product.create({
      name: "苹果",
```

```

    price: 5,
    stock: 100
  })

  const res = await request(app)
    .get(`/api/products/${product._id}`)

  expect(res.statusCode).toBe(200)
  expect(res.body.data.name).toBe("苹果")
})

test("删除商品", async () => {
  const product = await Product.create({
    name: "苹果",
    price: 5,
    stock: 100
  })

  const res = await request(app)
    .delete(`/api/products/${product._id}`)

  expect(res.statusCode).toBe(200)
  expect(res.body.message).toBe("删除商品成功")
})
})

```

二十四、测试需要登录的接口

如果商品创建需要管理员登录，测试流程一般是：

```

注册用户
生成 token
带 token 请求接口

```

示例工具函数：

tests/helpers/auth.js：

```

const jwt = require("jsonwebtoken")

function createToken(user = {}) {
  return jwt.sign(
    {
      id: user.id || "test-user-id",
      username: user.username || "admin",
      role: user.role || "admin"
    },
    process.env.JWT_SECRET,
    {
      expiresIn: "1h"
    }
  )
}

```

```
}

module.exports = {
  createToken
}
```

使用：

```
const { createToken } = require("../helpers/auth")

test("带 token 创建商品成功", async () => {
  const token = createToken({
    id: "1",
    username: "admin",
    role: "admin"
  })

  const res = await request(app)
    .post("/api/products")
    .set("Authorization", `Bearer ${token}`)
    .send({
      name: "苹果",
      price: 5,
      stock: 100
    })

  expect(res.statusCode).toBe(201)
})
```

如果你的鉴权中间件会根据 `req.user.id` 去数据库查用户，那就需要先创建测试用户。

二十五、Mock 是什么？

Mock 就是模拟某些依赖。

例如：

- 不想真的发短信
- 不想真的发邮件
- 不想真的调用支付接口
- 不想真的访问第三方 API

就可以用 Mock。

二十六、Jest Mock 基础

假设有发送邮件函数：

`utils/email.js`：

```

async function sendEmail(to, subject, content) {
  // 真实项目这里会调用邮件服务
  console.log("发送邮件", to, subject, content)

  return true
}

module.exports = {
  sendEmail
}

```

验证码服务：

services/codeService.js：

```

const { sendEmail } = require("../utils/email")

async function sendEmailCode(email, code) {
  await sendEmail(email, "验证码", `你的验证码是: ${code}`)

  return true
}

module.exports = {
  sendEmailCode
}

```

测试时不想真的发送邮件：

```

jest.mock("../utils/email", () => ({
  sendEmail: jest.fn().mockResolvedValue(true)
}))

const { sendEmail } = require("../utils/email")
const { sendEmailCode } = require("../services/codeService")

test("发送邮箱验证码", async () => {
  await sendEmailCode("test@example.com", "123456")

  expect(sendEmail).toHaveBeenCalled()
  expect(sendEmail).toHaveBeenCalledWith(
    "test@example.com",
    "验证码",
    "你的验证码是: 123456"
  )
})

```

二十七、测试覆盖率 coverage

运行：

```
npm run test:coverage
```

如果配置了：

```
{
  "scripts": {
    "test:coverage": "jest --coverage"
  }
}
```

会生成覆盖率报告：

```
coverage
```

常见指标：

指标	说明
Statements	语句覆盖率
Branches	分支覆盖率
Functions	函数覆盖率
Lines	行覆盖率

覆盖率不是越高越好，但它能帮助你发现哪些代码没有被测试。

二十八、常见测试命令

```
npm test
```

运行全部测试。

```
npm run test:watch
```

监听文件变化，自动重新测试。

```
npm run test:coverage
```

生成测试覆盖率。

只运行某个测试文件：

```
npx jest tests/integration/auth.test.js
```

只运行名字匹配的测试：

```
npx jest -t "登录成功"
```

二十九、测试命名规范

推荐：

```
xxx.test.js
```

或者：

```
xxx.spec.js
```

例如：

```
auth.test.js  
product.test.js  
generateToken.test.js
```

Jest 默认会识别这些文件。

三十、测试时常见问题

1. Jest 运行后不退出

常见原因：

- 数据库连接没关闭
- Redis 连接没关闭
- 定时器没清理
- HTTP server 没关闭

解决：

```
afterAll(async () => {  
  await mongoose.connection.close()  
})
```

如果用了 Redis：

```
afterAll(async () => {  
  await redisClient.quit()  
})
```

```
})
```

2. 测试之间数据互相影响

每个测试后清理数据：

```
afterEach(async () => {  
  await User.deleteMany({})  
  await Product.deleteMany({})  
})
```

3. 不要用生产数据库测试

一定要用测试数据库：

```
MONGO_URL=mongodb://127.0.0.1:27017/node_mvc_demo_test
```

4. 测试依赖顺序

不要让测试依赖执行顺序。

不推荐：

```
第一个测试注册  
第二个测试依赖第一个测试登录  
第三个测试依赖第二个测试的 token
```

推荐每个测试自己准备需要的数据。

5. 密码加密导致登录测试失败

注册时密码会被 bcrypt 加密。

如果你手动创建用户，不要这样：

```
await User.create({  
  username: "admin",  
  password: "123456"  
})
```

登录时 `bcrypt.compare()` 会失败。

应该：

```
const bcrypt = require("bcryptjs")

const hashedPassword = await bcrypt.hash("123456", 10)

await User.create({
  username: "admin",
  password: hashedPassword
})
```

三十一、推荐测试策略

对于初学者，建议按这个优先级写测试：

1. 工具函数优先测试

例如：

- 生成验证码
- 生成 Token
- 参数处理函数
- 时间格式化函数

这些最容易测试。

2. 核心接口测试

例如：

- 注册
 - 登录
 - 获取当前用户
 - 创建商品
 - 删除商品
 - 下单
-

3. 错误场景测试

不要只测成功，也要测失败：

- 参数为空
- 密码错误
- Token 缺失

- Token 无效
- 资源不存在
- 权限不足

三十二、一个完整 Jest 配置示例

jest.config.js :

```
module.exports = {
  testEnvironment: "node",
  setupFiles: ["<rootDir>/jest.setup.js"],
  testMatch: ["**/tests/**/*.test.js"],
  verbose: true,
  collectCoverageFrom: [
    "controllers/**/*.js",
    "middlewares/**/*.js",
    "utils/**/*.js",
    "routes/**/*.js",
    "!server.js",
    "!app.js"
  ]
}
```

jest.setup.js :

```
require("dotenv").config({
  path: ".env.test"
})

jest.setTimeout(30000)
```

package.json :

```
{
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  }
}
```

三十三、本课重点

这一课你需要掌握：

1. Jest 是常用 JavaScript 测试框架
2. 安装：

```
npm install jest -D
```

3. 测试文件常用命名：

```
xxx.test.js
```

4. 常用写法：

```
test("描述", () => {  
  expect(result).toBe(expected)  
})
```

5. 对象和数组用：

```
toEqual()
```

6. 异步测试用：

```
async/await
```

7. Express 接口测试使用：

```
npm install supertest -D
```

8. app.js 应该导出 app， server.js 负责监听端口
9. 测试数据库要和生产数据库分开
10. 测试结束后要关闭数据库连接
11. 每个测试后要清理测试数据
12. Mock 可以模拟第三方服务
13. 覆盖率可以用：

```
jest --coverage
```

课后练习

基于你的 MVC 项目完成以下任务：

1. 安装测试依赖

```
npm install jest supertest -D
```

2. 配置测试脚本

package.json :

```
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  }
}
```

3. 创建测试配置

创建：

```
jest.config.js
jest.setup.js
.env.test
```

.env.test 使用测试数据库。

4. 编写单元测试

测试：

```
utils/generateToken.js
utils/generateCode.js
utils/response.js
```

5. 编写接口测试

测试以下接口：

接口	测试点
POST /api/auth/register	注册成功、重复注册、参数错误
POST /api/auth/login	登录成功、密码错误

接口	测试点
GET /api/users/profile	未登录失败、登录成功
GET /api/products	获取商品列表
POST /api/products	创建成功、参数错误
DELETE /api/products/:id	删除成功、ID 错误

6. 运行覆盖率

```
npm run test:coverage
```

查看哪些代码还没有被测试。

下一课建议学习：

Node.js 第十七课：TypeScript + Express 项目改造入门。

Node.js 第十七课：TypeScript + Express 项目改造入门

这一课我们学习如何把 Node.js / Express 项目从 JavaScript 改造成 TypeScript。

TypeScript 是现在后端 Node.js 项目中非常常见的技术，尤其适合中大型项目。

本课内容：

1. TypeScript 是什么
2. 为什么 Node.js 项目要用 TypeScript
3. 创建 TypeScript + Express 项目
4. 配置 tsconfig.json
5. 使用 ts-node-dev 开发运行
6. 改造 Express 入口文件
7. 给 req、res、next 添加类型
8. 路由、控制器、中间件 TypeScript 写法
9. 扩展 req.user
10. TypeScript 项目目录结构

一、TypeScript 是什么？

TypeScript 简称 TS。

它可以理解为：

JavaScript 写法：

```
function add(a, b) {  
  return a + b  
}
```

这个函数的问题是：你不知道 `a` 和 `b` 应该传什么类型。

可能有人这样调用：

```
add(1, 2)  
add("1", "2")  
add({}, [])
```

TypeScript 写法：

```
function add(a: number, b: number): number {  
  return a + b  
}
```

这样就明确了：

- `a` 必须是数字
- `b` 必须是数字
- 返回值也是数字

如果你写：

```
add("1", "2")
```

编辑器会直接报错。

二、为什么 Express 项目要用 TypeScript?

TypeScript 的优点：

1. 减少运行时错误
2. 编辑器提示更好
3. 参数类型更清晰
4. 适合多人协作
5. 代码更容易维护
6. 重构更安全
7. 接口数据结构更明确

例如 JavaScript 中：

```
const id = req.params.id
```

你不确定：

```
req.params 有没有 id?  
id 是 string 还是 number?
```

TypeScript 中可以更明确：

```
const id: string = req.params.id
```

三、创建 TypeScript + Express 项目

创建项目：

```
mkdir express-ts-demo  
cd express-ts-demo  
npm init -y
```

安装运行依赖：

```
npm install express dotenv cors
```

安装开发依赖：

```
npm install typescript ts-node-dev @types/node @types/express @types/cors -D
```

说明：

包名	作用
typescript	TypeScript 编译器
ts-node-dev	开发环境直接运行 TS，并自动重启
@types/node	Node.js 类型声明
@types/express	Express 类型声明
@types/cors	cors 类型声明

四、初始化 tsconfig.json

执行：

```
npx tsc --init
```

会生成：

```
tsconfig.json
```

推荐配置如下：

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "rootDir": "./src",
    "outDir": "./dist",

    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,

    "moduleResolution": "node",
    "resolveJsonModule": true,

    "types": ["node"]
  },
  "include": ["src/**/*.ts"],
  "exclude": ["node_modules", "dist"]
}
```

字段说明：

配置	说明
target	编译后的 JS 版本
module	模块规范
rootDir	TS 源码目录
outDir	编译输出目录
strict	开启严格类型检查
esModuleInterop	允许更方便地导入 CommonJS 模块
include	指定要编译的文件
exclude	排除目录

五、配置 package.json 脚本

修改 package.json：

```
{
  "scripts": {
    "dev": "ts-node-dev --respawn --transpile-only src/server.ts",
    "build": "tsc",
    "start": "node dist/server.js"
  }
}
```

说明：

命令	作用
npm run dev	开发环境运行 TS
npm run build	编译 TS 到 dist
npm start	运行编译后的 JS

开发时：

```
npm run dev
```

生产部署时：

```
npm run build
npm start
```

六、推荐项目结构

```
express-ts-demo
├── src
│   ├── app.ts
│   ├── server.ts
│   ├── routes
│   │   └── userRoutes.ts
│   ├── controllers
│   │   └── userController.ts
│   ├── middlewares
│   │   ├── errorMiddleware.ts
│   │   └── authMiddleware.ts
│   ├── utils
│   │   └── response.ts
│   └── types
│       └── express.d.ts
├── package.json
├── tsconfig.json
└── .env
```


和之前 MVC 项目类似，只是文件后缀从 `.js` 变成 `.ts`。

七、创建 Express 入口文件

创建目录：

```
mkdir src
```

创建：

```
src/app.ts
```

代码：

```
import express from "express"
import cors from "cors"

const app = express()

app.use(cors())
app.use(express.json())
app.use(express.urlencoded({ extended: false }))

app.get("/", (req, res) => {
  res.send("TypeScript + Express 项目启动成功")
})

app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok",
    time: new Date().toISOString()
  })
})

export default app
```

注意 TypeScript 中一般使用 ES Module 导入：

```
import express from "express"
```

导出：

```
export default app
```

八、创建 server.ts

创建：

```
src/server.ts
```

代码：

```
import dotenv from "dotenv"
import app from "./app"

dotenv.config()

const PORT = process.env.PORT || 3000

app.listen(PORT, () => {
  console.log(`服务器启动: http://localhost:${PORT}`)
})
```

创建 .env：

```
PORT=3000
NODE_ENV=development
```

启动：

```
npm run dev
```

访问：

```
http://localhost:3000
http://localhost:3000/api/health
```

九、Express 中 req、res、next 类型

TypeScript 中可以给 Express 参数添加类型。

```
import { Request, Response, NextFunction } from "express"

function handler(req: Request, res: Response, next: NextFunction) {
  res.json({
    message: "ok"
  })
}
```

常用类型：

类型	来源	作用
Request	express	请求对象
Response	express	响应对象
NextFunction	express	next 函数
Router	express	路由类型

十、创建统一响应工具

创建：

```
src/utils/response.ts
```

```
import { Response } from "express"

export function success<T>(  
  res: Response,  
  data: T,  
  message = "操作成功",  
  code = 0  
) {  
  return res.json({  
    code,  
    message,  
    data  
  })  
}  
  
export function fail(  
  res: Response,  
  message = "操作失败",  
  statusCode = 400,  
  code = statusCode  
) {  
  return res.status(statusCode).json({  
    code,  
    message  
  })  
}
```

这里的：

```
<T>
```

是泛型，表示 data 可以是任意类型，但 TypeScript 会尽量推断出来。

例如：

```
success(res, { id: 1, name: "张三" })
```

TypeScript 能知道 `data` 是一个对象。

十一、创建用户控制器

创建目录：

```
mkdir src/controllers
```

创建：

```
src/controllers/userController.ts
```

```
import { Request, Response } from "express"
import { success, fail } from "../utils/response"

interface User {
  id: number
  username: string
  age: number
}

const users: User[] = [
  { id: 1, username: "张三", age: 20 },
  { id: 2, username: "李四", age: 22 }
]

export function getUserList(req: Request, res: Response) {
  success<User[]>(res, users, "获取用户列表成功")
}

export function getUserDetail(req: Request, res: Response) {
  const id = Number(req.params.id)

  const user = users.find(item => item.id === id)

  if (!user) {
    return fail(res, "用户不存在", 404)
  }

  success<User>(res, user, "获取用户详情成功")
}

export function createUser(req: Request, res: Response) {
  const { username, age } = req.body

  if (!username || age === undefined) {
    return fail(res, "username 和 age 不能为空", 400)
  }
}
```

```

const newUser: User = {
  id: users.length ? users[users.length - 1].id + 1 : 1,
  username,
  age: Number(age)
}

users.push(newUser)

return res.status(201).json({
  code: 0,
  message: "创建用户成功",
  data: newUser
})
}

```

这里我们定义了一个接口：

```

interface User {
  id: number
  username: string
  age: number
}

```

表示用户对象必须长这样：

```

{
  id: number
  username: string
  age: number
}

```

十二、创建用户路由

创建目录：

```
mkdir src/routes
```

创建：

```
src/routes/userRoutes.ts
```

```

import { Router } from "express"
import {
  getUserList,
  getUserDetail,
  createUser
} from "../controllers/userController"

```

```
const router = Router()

router.get("/", getUserList)
router.get("/:id", getUserDetail)
router.post("/", createUser)

export default router
```

在 `src/app.ts` 中注册：

```
import userRoutes from "../routes/userRoutes"
```

完整 `app.ts`：

```
import express from "express"
import cors from "cors"
import userRoutes from "../routes/userRoutes"

const app = express()

app.use(cors())
app.use(express.json())
app.use(express.urlencoded({ extended: false }))

app.get("/", (req, res) => {
  res.send("TypeScript + Express 项目启动成功")
})

app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok",
    time: new Date().toISOString()
  })
})

app.use("/api/users", userRoutes)

export default app
```

测试：

```
curl http://localhost:3000/api/users
```

```
curl http://localhost:3000/api/users/1
```

```
curl -X POST http://localhost:3000/api/users \
-H "Content-Type: application/json" \
-d '{"username":"王五","age":25}'
```

十三、给 req.body 添加类型

默认情况下：

```
req.body
```

类型通常是 `any`，类型检查不严格。

可以这样手动声明请求体类型：

```
import { Request, Response } from "express"

interface CreateUserBody {
  username: string
  age: number
}

export function createUser(
  req: Request<{}, {}, CreateUserBody>,
  res: Response
) {
  const { username, age } = req.body

  // username 是 string
  // age 是 number
}
```

`Request` 泛型参数含义：

```
Request<Params, ResBody, ReqBody, ReqQuery>
```

参数	说明
Params	路由参数
ResBody	响应体
ReqBody	请求体
ReqQuery	query 参数

示例：给 params 添加类型

```
interface UserParams {
  id: string
}

export function getUserDetail(
```

```
req: Request<UserParams>,
res: Response
) {
  const id = Number(req.params.id)

  res.json({
    id
  })
}
```

示例：给 query 添加类型

```
interface UserQuery {
  page?: string
  pageSize?: string
}

export function getUserList(
  req: Request<{}, {}, {}, UserQuery>,
  res: Response
) {
  const page = Number(req.query.page || 1)
  const pageSize = Number(req.query.pageSize || 10)

  res.json({
    page,
    pageSize
  })
}
```

注意：HTTP query 参数默认都是字符串。

十四、封装 asyncHandler

创建：

```
src/utils/asyncHandler.ts
```

```
import { Request, Response, NextFunction } from "express"

type AsyncFunction = (
  req: Request,
  res: Response,
  next: NextFunction
) => Promise<any>

function asyncHandler(fn: AsyncFunction) {
  return function (req: Request, res: Response, next: NextFunction) {
```



```
    Promise.resolve(fn(req, res, next)).catch(next)
  }
}

export default asyncHandler
```

使用：

```
import asyncHandler from "../utils/asyncHandler"

export const getUserList = asyncHandler(async (req, res) => {
  res.json({
    code: 0,
    data: []
  })
})
```

十五、自定义错误类 AppError

创建：

src/utils/AppError.ts

```
class AppError extends Error {
  statusCode: number
  code: number
  errors?: string[]

  constructor(
    message: string,
    statusCode = 500,
    code = statusCode,
    errors?: string[]
  ) {
    super(message)

    this.statusCode = statusCode
    this.code = code
    this.errors = errors

    Error.captureStackTrace(this, this.constructor)
  }
}

export default AppError
```

使用：

```
throw new AppError("用户不存在", 404)
```

十六、错误处理中间件 TypeScript 写法

创建：

```
src/middlewares/errorMiddleware.ts
```

```
import { Request, Response, NextFunction } from "express"
import AppError from "../utils/AppError"

function errorMiddleware(
  err: AppError,
  req: Request,
  res: Response,
  next: NextFunction
) {
  const statusCode = err.statusCode || 500
  const code = err.code || statusCode

  const response: {
    code: number
    message: string
    errors?: string[]
    stack?: string
  } = {
    code,
    message: err.message || "服务器内部错误"
  }

  if (err.errors) {
    response.errors = err.errors
  }

  if (process.env.NODE_ENV !== "production") {
    response.stack = err.stack
  }

  res.status(statusCode).json(response)
}

export default errorMiddleware
```

在 app.ts 中使用：

```
import errorMiddleware from "../middlewares/errorMiddleware"

app.use(errorMiddleware)
```

注意：错误处理中间件要放在路由后面。

十七、404 中间件 TypeScript 写法

创建：

```
src/middlewares/notFoundMiddleware.ts
```

```
import { Request, Response } from "express"

function notFoundMiddleware(req: Request, res: Response) {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
}

export default notFoundMiddleware
```

在 app.ts 中：

```
import notFoundMiddleware from "../middlewares/notFoundMiddleware"
import errorMiddleware from "../middlewares/errorMiddleware"

app.use(notFoundMiddleware)
app.use(errorMiddleware)
```

十八、JWT 鉴权中间件 TypeScript 写法

安装：

```
npm install jsonwebtoken
npm install @types/jsonwebtoken -D
```

创建：

```
src/middlewares/authMiddleware.ts
```

```
import { Request, Response, NextFunction } from "express"
import jwt from "jsonwebtoken"

interface JwtPayload {
  id: string
  username: string
  role: "user" | "admin"
}

export function authMiddleware(
  req: Request,
  res: Response,
```

```

    next: NextFunction
  ) {
    const authorization = req.headers.authorization

    if (!authorization) {
      return res.status(401).json({
        code: 401,
        message: "缺少 Authorization 请求头"
      })
    }

    const parts = authorization.split(" ")

    if (parts.length !== 2 || parts[0] !== "Bearer") {
      return res.status(401).json({
        code: 401,
        message: "Token 格式错误"
      })
    }

    const token = parts[1]

    try {
      const decoded = jwt.verify(
        token,
        process.env.JWT_SECRET as string
      ) as JwtPayload

      req.user = decoded

      next()
    } catch (err) {
      return res.status(401).json({
        code: 401,
        message: "Token 无效或已过期"
      })
    }
  }
}

```

但是这里会报错：

```
Property 'user' does not exist on type 'Request'
```

因为 Express 默认的 `Request` 类型上没有 `user` 属性。

下面我们解决这个问题。

十九、扩展 Express Request 类型

创建：

```
src/types/express.d.ts
```

```
declare namespace Express {  
  export interface Request {  
    user?: {  
      id: string  
      username: string  
      role: "user" | "admin"  
    }  
  }  
}
```

然后确保 `tsconfig.json` 包含：

```
{  
  "include": ["src/**/*.ts", "src/**/*.d.ts"]  
}
```

这样 TypeScript 就知道：

```
req.user
```

是合法属性。

二十、生成 JWT 工具函数

创建：

```
src/utils/generateToken.ts
```

```
import jwt from "jsonwebtoken"  
  
interface TokenUser {  
  id: string  
  username: string  
  role: "user" | "admin"  
}  
  
function generateToken(user: TokenUser): string {  
  return jwt.sign(  
    {  
      id: user.id,  
      username: user.username,  
      role: user.role  
    },  
    process.env.JWT_SECRET as string,  
    {  
      expiresIn: process.env.JWT_EXPIRES_IN || "2h"  
    }  
  )  
}
```

```
    }  
  )  
}  
  
export default generateToken
```

.env :

```
JWT_SECRET=your_secret_key  
JWT_EXPIRES_IN=2h
```

二十一、使用 Mongoose + TypeScript

如果项目使用 MongoDB，需要安装：

```
npm install mongoose
```

Mongoose 自带 TypeScript 类型，不需要额外安装 `@types/mongoose`。

用户模型 TypeScript 写法

创建：

```
src/models/User.ts
```

```
import mongoose, { Document, Schema } from "mongoose"  
  
export interface IUser extends Document {  
  username: string  
  password: string  
  nickname?: string  
  avatar?: string  
  role: "user" | "admin"  
  createdAt: Date  
  updatedAt: Date  
}  
  
const userSchema = new Schema<IUser>(  
  {  
    username: {  
      type: String,  
      required: true,  
      unique: true,  
      trim: true  
    },  
    password: {  
      type: String,
```

```

    required: true
  },
  nickname: {
    type: String,
    default: ""
  },
  avatar: {
    type: String,
    default: ""
  },
  role: {
    type: String,
    enum: ["user", "admin"],
    default: "user"
  }
},
{
  timestamps: true
}
)

const User = mongoose.model<IUser>("User", userSchema)

export default User

```

二十二、数据库连接 TypeScript 写法

创建：

src/config/db.ts

```

import mongoose from "mongoose"

async function connectDB(): Promise<void> {
  try {
    await mongoose.connect(process.env.MONGO_URL as string)
    console.log("MongoDB 连接成功")
  } catch (err) {
    if (err instanceof Error) {
      console.error("MongoDB 连接失败: ", err.message)
    } else {
      console.error("MongoDB 连接失败: ", err)
    }
  }

  process.exit(1)
}

export default connectDB

```

修改 server.ts：

```
import dotenv from "dotenv"
import app from "../app"
import connectDB from "../config/db"

dotenv.config()

const PORT = process.env.PORT || 3000

async function start() {
  await connectDB()

  app.listen(PORT, () => {
    console.log(`服务器启动: http://localhost:${PORT}`)
  })
}

start()
```

.env :

```
PORT=3000
MONGO_URL=mongodb://127.0.0.1:27017/express_ts_demo
JWT_SECRET=your_secret_key
JWT_EXPIRES_IN=2h
```

二十三、TypeScript 版注册登录控制器

安装：

```
npm install bcryptjs
npm install @types/bcryptjs -D
```

创建：

src/controllers/authController.ts

```
import { Request, Response } from "express"
import bcrypt from "bcryptjs"
import User from "../models/User"
import asyncHandler from "../utils/asyncHandler"
import generateToken from "../utils/generateToken"
import AppError from "../utils/AppError"
import { success } from "../utils/response"

interface RegisterBody {
  username: string
  password: string
  nickname?: string
}
```



```
interface LoginBody {
  username: string
  password: string
}

export const register = asyncHandler(async (
  req: Request<{}, {}>, RegisterBody>,
  res: Response
) => {
  const { username, password, nickname } = req.body

  const existUser = await User.findOne({ username })

  if (existUser) {
    throw new AppError("用户名已存在", 409)
  }

  const hashedPassword = await bcrypt.hash(password, 10)

  const user = await User.create({
    username,
    password: hashedPassword,
    nickname
  })

  success(res, {
    id: user._id,
    username: user.username,
    nickname: user.nickname,
    role: user.role
  }, "注册成功")
})

export const login = asyncHandler(async (
  req: Request<{}, {}>, LoginBody>,
  res: Response
) => {
  const { username, password } = req.body

  const user = await User.findOne({ username })

  if (!user) {
    throw new AppError("用户名或密码错误", 401)
  }

  const isMatch = await bcrypt.compare(password, user.password)

  if (!isMatch) {
    throw new AppError("用户名或密码错误", 401)
  }

  const token = generateToken({
    id: user._id.toString(),
    username: user.username,
    role: user.role
  })
})
```

```
    success(res, {
      token,
      user: {
        id: user._id,
        username: user.username,
        nickname: user.nickname,
        avatar: user.avatar,
        role: user.role
      }
    }, "登录成功")
  })
})
```

二十四、认证路由 TypeScript 写法

创建：

```
src/routes/authRoutes.ts
```

```
import { Router } from "express"
import { register, login } from "../controllers/authController"

const router = Router()

router.post("/register", register)
router.post("/login", login)

export default router
```

在 app.ts 中注册：

```
import authRoutes from "../routes/authRoutes"

app.use("/api/auth", authRoutes)
```

二十五、TypeScript 版用户信息接口

src/controllers/userController.ts：

```
import { Request, Response } from "express"
import User from "../models/User"
import asyncHandler from "../utils/asyncHandler"
import AppError from "../utils/AppError"
import { success } from "../utils/response"

export const getProfile = asyncHandler(async (
  req: Request,
```

```

    res: Response
  ) => {
    if (!req.user) {
      throw new AppError("未登录", 401)
    }

    const user = await User.findById(req.user.id).select("-password")

    if (!user) {
      throw new AppError("用户不存在", 404)
    }

    success(res, user, "获取用户信息成功")
  })

```

src/routes/userRoutes.ts :

```

import { Router } from "express"
import { getProfile } from "../controllers/userController"
import { authMiddleware } from "../middlewares/authMiddleware"

const router = Router()

router.get("/profile", authMiddleware, getProfile)

export default router

```

app.ts :

```

import userRoutes from "../routes/userRoutes"

app.use("/api/users", userRoutes)

```

二十六、参数校验：Joi + TypeScript

安装：

```
npm install joi
```

创建：

src/validators/authValidator.ts

```

import Joi from "joi"

export const registerSchema = Joi.object({
  username: Joi.string()
    .trim()

```

```

    .min(3)
    .max(20)
    .required()
    .messages({
      "string.empty": "用户名不能为空",
      "string.min": "用户名至少 3 个字符",
      "string.max": "用户名最多 20 个字符",
      "any.required": "用户名不能为空"
    }),

password: Joi.string()
  .min(6)
  .max(30)
  .required()
  .messages({
    "string.empty": "密码不能为空",
    "string.min": "密码至少 6 位",
    "string.max": "密码最多 30 位",
    "any.required": "密码不能为空"
  }),

nickname: Joi.string()
  .trim()
  .max(30)
  .allow("")
})

```

校验中间件:

```
src/middlewares/validateMiddleware.ts
```

```

import { Request, Response, NextFunction } from "express"
import { ObjectSchema } from "joi"

type ValidateProperty = "body" | "query" | "params"

function validate(schema: ObjectSchema, property: ValidateProperty = "body") {
  return function (req: Request, res: Response, next: NextFunction) {
    const { error, value } = schema.validate(req[property], {
      abortEarly: false,
      stripUnknown: true,
      convert: true
    })

    if (error) {
      const errors = error.details.map(item => item.message)

      return res.status(400).json({
        code: 400,
        message: "参数校验失败",
        errors
      })
    }
  }
}

```

```
    req[property] = value

    next()
  }
}

export default validate
```

路由中使用：

```
import { Router } from "express"
import { register, login } from "../controllers/authController"
import validate from "../middlewares/validateMiddleware"
import { registerSchema } from "../validators/authValidator"

const router = Router()

router.post("/register", validate(registerSchema), register)
router.post("/login", login)

export default router
```

二十七、完整 app.ts 示例

```
import express from "express"
import cors from "cors"

import authRoutes from "../routes/authRoutes"
import userRoutes from "../routes/userRoutes"

import notFoundMiddleware from "../middlewares/notFoundMiddleware"
import errorMiddleware from "../middlewares/errorMiddleware"

const app = express()

app.use(cors())
app.use(express.json({ limit: "1mb" }))
app.use(express.urlencoded({ extended: false, limit: "1mb" }))

app.get("/", (req, res) => {
  res.send("TypeScript + Express 项目启动成功")
})

app.get("/api/health", (req, res) => {
  res.json({
    code: 0,
    message: "ok",
    time: new Date().toISOString()
  })
})

app.use("/api/auth", authRoutes)
```

```
app.use("/api/users", userRoutes)

app.use(notFoundMiddleware)
app.use(errorMiddleware)

export default app
```

二十八、开发与生产运行流程

开发环境：

```
npm run dev
```

编译：

```
npm run build
```

运行编译后的 JS：

```
npm start
```

编译后会生成：

```
dist
├─ app.js
├─ server.js
├─ routes
├─ controllers
└─ ...
```

生产环境一般运行：

```
node dist/server.js
```

PM2 部署：

```
pm2 start dist/server.js --name express-ts-demo
```

二十九、常见 TypeScript 类型

1. string

```
const username: string = "admin"
```

2. number

```
const age: number = 20
```

3. boolean

```
const isAdmin: boolean = true
```

4. array

```
const ids: number[] = [1, 2, 3]
const names: string[] = ["张三", "李四"]
```

5. object interface

```
interface Product {
  id: number
  name: string
  price: number
}

const product: Product = {
  id: 1,
  name: "苹果",
  price: 5
}
```

6. 可选属性

```
interface User {
  id: number
  username: string
  avatar?: string
}
```

7. 联合类型

```
type Role = "user" | "admin"

const role: Role = "admin"
```

三十、TypeScript 常见错误

1. Cannot find module 'xxx' or its corresponding type declarations

原因：缺少类型声明。

解决：

```
npm install @types/xxx -D
```

例如：

```
npm install @types/express -D
npm install @types/jsonwebtoken -D
```

2. Property 'user' does not exist on type Request

原因：Express Request 默认没有 `user`。

解决：创建类型扩展文件：

```
declare namespace Express {
  export interface Request {
    user?: {
      id: string
      username: string
      role: "user" | "admin"
    }
  }
}
```

3. process.env.JWT_SECRET 类型错误

`process.env.JWT_SECRET` 的类型是：

```
string | undefined
```

但 `jwt.sign` 需要 `string`。

可以写：

```
process.env.JWT_SECRET as string
```

更严谨的做法是启动时检查环境变量是否存在。

4. req.query.page 类型不好用

Express 中 `query` 的类型比较复杂，实际开发中经常需要手动转换：


```
const page = Number(req.query.page || 1)
```

或者配合 Joi 校验后再使用。

三十一、JavaScript 项目如何逐步迁移到 TypeScript?

不建议一口气把所有文件改完。

推荐顺序：

1. 安装 TypeScript
2. 新增 tsconfig.json
3. 先改入口文件：
 - app.js -> app.ts
 - server.js -> server.ts
4. 再改工具函数：
 - utils
5. 再改中间件：
 - middlewares
6. 再改路由：
 - routes
7. 再改控制器：
 - controllers
8. 最后改模型：
 - models

也可以先允许 JS 和 TS 混用：

```
{
  "compilerOptions": {
    "allowJs": true
  }
}
```

但长期建议统一成 .ts。

三十二、本课重点

这一课你需要掌握：

1. TypeScript 是带类型系统的 JavaScript
2. Express 项目使用 TS 可以减少错误、提升维护性
3. 安装核心依赖：

```
npm install typescript ts-node-dev @types/node @types/express -D
```

4. `src` 放 TS 源码
5. `dist` 放编译后的 JS
6. `app.ts` 导出 Express 应用
7. `server.ts` 启动服务
8. Express 常用类型：

```
Request  
Response  
NextFunction  
Router
```

9. 可以用 `interface` 定义请求体、用户、商品等结构
10. 可以扩展 `Express.Request` 添加 `req.user`
11. 生产环境需要先：

```
npm run build
```

再运行：

```
npm start
```

课后练习

把你之前的 MVC 项目改造成 TypeScript 版本。

要求：

1. 安装依赖

```
npm install typescript ts-node-dev @types/node @types/express -D
```

如果用到这些库，也安装类型：

```
npm install @types/cors @types/jsonwebtoken @types/bcryptjs -D
```

2. 创建目录结构

```
src  
├─ app.ts  
├─ server.ts
```

```
|— routes
|— controllers
|— middlewares
|— models
|— utils
|— validators
└— types
```

3. 完成基础接口

实现：

方法	路径	功能
GET	/api/health	健康检查
POST	/api/auth/register	注册
POST	/api/auth/login	登录
GET	/api/users/profile	获取当前用户

4. 类型要求

至少定义：

```
interface RegisterBody {
  username: string
  password: string
  nickname?: string
}
```

```
interface LoginBody {
  username: string
  password: string
}
```

```
interface JwtPayload {
  id: string
  username: string
  role: "user" | "admin"
}
```

5. 扩展 req.user

创建：

```
src/types/express.d.ts
```

让下面代码不报错：

```
req.user.id
```

下一课建议学习：

Node.js 第十八课：Docker 部署 Node.js + MongoDB + Redis 项目。

Node.js 第十八课：Docker 部署 Node.js + MongoDB + Redis 项目

这一课学习如何使用 **Docker** 部署一个完整后端项目：

```
Node.js / Express  
MongoDB  
Redis
```

前面我们部署 Node.js 项目时，需要手动在服务器上安装：

- Node.js
- npm
- MongoDB
- Redis
- PM2
- Nginx

这种方式能用，但环境配置比较麻烦。

Docker 可以把运行环境打包起来，让项目在不同服务器上都能以相同方式运行。

一、Docker 是什么？

Docker 可以理解为：

一个轻量级的应用运行环境打包工具。

你可以把项目和运行环境打包成一个镜像，然后在任何安装了 Docker 的机器上运行。

例如你的项目需要：

```
Node.js 20  
MongoDB
```

Redis
环境变量
端口配置

以前你要手动安装这些环境。

用了 Docker 后，可以通过配置文件一次性启动：

```
docker compose up -d
```

二、Docker 的核心概念

1. 镜像 image

镜像可以理解为“应用模板”。

比如：

```
node:20-alpine  
mongo:7  
redis:7-alpine  
nginx:alpine
```

这些都是镜像。

2. 容器 container

容器是镜像运行后的实例。

比如用 `mongo:7` 镜像启动后，会得到一个 MongoDB 容器。

3. Dockerfile

Dockerfile 用来描述如何构建你自己的项目镜像。

例如：

```
FROM node:20-alpine  
  
WORKDIR /app  
  
COPY package*.json ./  
  
RUN npm install --omit=dev
```

```
COPY . .

EXPOSE 3000

CMD ["node", "server.js"]
```

4. docker-compose.yml

docker-compose.yml 用来同时管理多个容器。

比如一个后端项目可能需要：

```
app 容器
mongo 容器
redis 容器
```

使用 Docker Compose 可以一次启动所有服务。

三、安装 Docker

Ubuntu / Debian 简单安装

```
curl -fsSL https://get.docker.com | bash
```

启动 Docker：

```
systemctl start docker
systemctl enable docker
```

查看版本：

```
docker -v
```

查看 Docker Compose：

```
docker compose version
```

如果能显示版本，说明安装成功。

四、准备一个 Express 项目

假设项目结构如下：

```
node-docker-demo
├─ app.js
├─ server.js
├─ package.json
├─ package-lock.json
├─ .env
├─ Dockerfile
├─ .dockerignore
├─ docker-compose.yml
├─ config
│   └─ db.js
│   └─ redis.js
├─ models
│   └─ User.js
└─ routes
    └─ userRoutes.js
```

五、示例项目代码

1. package.json

```
{
  "name": "node-docker-demo",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.18.3",
    "mongoose": "^8.0.0",
    "redis": "^4.6.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

安装依赖：

```
npm install express mongoose redis dotenv
npm install nodemon -D
```

2. app.js

```

const express = require("express")
const userRoutes = require("./routes/userRoutes")
const { redisClient } = require("./config/redis")

const app = express()

app.use(express.json())
app.use(express.urlencoded({ extended: false }))

app.get("/", (req, res) => {
  res.send("Node.js Docker Demo")
})

app.get("/api/health", async (req, res) => {
  let redisStatus = "unknown"

  try {
    await redisClient.ping()
    redisStatus = "connected"
  } catch (err) {
    redisStatus = "disconnected"
  }

  res.json({
    code: 0,
    message: "ok",
    env: process.env.NODE_ENV,
    mongo: "connected",
    redis: redisStatus,
    time: new Date().toISOString()
  })
})

app.use("/api/users", userRoutes)

app.use((req, res) => {
  res.status(404).json({
    code: 404,
    message: "接口不存在"
  })
})

module.exports = app

```

3. server.js

```

require("dotenv").config()

const app = require("./app")
const connectDB = require("./config/db")
const { connectRedis } = require("./config/redis")

```



```
const PORT = process.env.PORT || 3000

async function start() {
  await connectDB()
  await connectRedis()

  app.listen(PORT, () => {
    console.log(`服务器启动: http://localhost:${PORT}`)
  })
}

start()
```

4. config/db.js

```
const mongoose = require("mongoose")

async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGO_URL)
    console.log("MongoDB 连接成功")
  } catch (err) {
    console.error("MongoDB 连接失败: ", err.message)
    process.exit(1)
  }
}

module.exports = connectDB
```

5. config/redis.js

```
const { createClient } = require("redis")

const redisClient = createClient({
  url: process.env.REDIS_URL
})

redisClient.on("error", err => {
  console.error("Redis 错误: ", err)
})

async function connectRedis() {
  try {
    if (!redisClient.isOpen) {
      await redisClient.connect()
      console.log("Redis 连接成功")
    }
  } catch (err) {
    console.error("Redis 连接失败: ", err.message)
  }
}
```

```
    process.exit(1)
  }
}

module.exports = {
  redisClient,
  connectRedis
}
```

6. models/User.js

```
const mongoose = require("mongoose")

const userSchema = new mongoose.Schema(
  {
    username: {
      type: String,
      required: true,
      unique: true
    },
    age: {
      type: Number,
      default: 0
    }
  },
  {
    timestamps: true
  }
)

module.exports = mongoose.model("User", userSchema)
```

7. routes/userRoutes.js

```
const express = require("express")
const User = require("../models/User")
const { redisClient } = require("../config/redis")

const router = express.Router()

router.get("/", async (req, res) => {
  const cacheKey = "users:list"

  const cacheData = await redisClient.get(cacheKey)

  if (cacheData) {
    return res.json({
      code: 0,
      message: "获取成功",
    })
  }
})
```

```

        cache: true,
        data: JSON.parse(cacheData)
    })
}

const users = await User.find().sort({ createdAt: -1 }).lean()

await redisClient.set(cacheKey, JSON.stringify(users), {
    EX: 60
})

res.json({
    code: 0,
    message: "获取成功",
    cache: false,
    data: users
})
})

router.post("/", async (req, res) => {
    const { username, age } = req.body

    if (!username) {
        return res.status(400).json({
            code: 400,
            message: "username 不能为空"
        })
    }

    const user = await User.create({
        username,
        age
    })

    await redisClient.del("users:list")

    res.status(201).json({
        code: 0,
        message: "创建成功",
        data: user
    })
})

module.exports = router

```

六、重点：Docker 中不能用 localhost 连接数据库

本地开发时，你可能这样写：

```

MONGO_URL=mongodb://127.0.0.1:27017/node_demo
REDIS_URL=redis://127.0.0.1:6379

```

但是在 Docker Compose 中，127.0.0.1 指的是 **当前容器自己**，不是 MongoDB 容器，也不是 Redis 容器。

在 Docker Compose 网络中，服务之间可以用服务名互相访问。

如果 docker-compose.yml 中服务名是：

```
mongo:
redis:
```

那么 Node.js 应该这样连接：

```
MONGO_URL=mongodb://mongo:27017/node_docker_demo
REDIS_URL=redis://redis:6379
```

这是 Docker 部署中非常重要的点。

七、创建 Dockerfile

项目根目录创建：

```
Dockerfile
```

内容：

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install --omit=dev

COPY . .

EXPOSE 3000

CMD ["node", "server.js"]
```

解释：

```
FROM node:20-alpine
```

使用 Node.js 20 的轻量镜像。

```
WORKDIR /app
```

容器内工作目录是 `/app`。

```
COPY package*.json ./
```

先复制依赖文件。

```
RUN npm install --omit=dev
```

安装生产依赖。

```
COPY . .
```

复制项目代码。

```
EXPOSE 3000
```

声明容器使用 3000 端口。

```
CMD ["node", "server.js"]
```

容器启动时执行命令。

八、创建 `.dockerignore`

项目根目录创建：

```
.dockerignore
```

内容：

```
node_modules
npm-debug.log
.env
.git
.gitignore
Dockerfile
docker-compose.yml
logs
coverage
```

作用类似 `.gitignore`，防止无用文件被复制进镜像。

注意：

```
.env
```

一般不要打进镜像，生产环境用 Docker Compose 的环境变量传入。

九、创建 docker-compose.yml

项目根目录创建：

```
docker-compose.yml
```

内容：

```
services:
  app:
    build: .
    container_name: node_app
    restart: always
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
      PORT: 3000
      MONGO_URL: mongodb://mongo:27017/node_docker_demo
      REDIS_URL: redis://redis:6379
    depends_on:
      - mongo
      - redis

  mongo:
    image: mongo:7
    container_name: node_mongo
    restart: always
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

  redis:
    image: redis:7-alpine
    container_name: node_redis
    restart: always
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

volumes:
  mongo_data:
  redis_data:
```

十、docker-compose.yml 说明

1. app 服务

```
app:
  build: .
```

表示使用当前目录的 Dockerfile 构建 Node.js 应用镜像。

```
ports:
  - "3000:3000"
```

表示：

宿主机 3000 端口 -> 容器 3000 端口

浏览器访问：

http://服务器IP:3000

就会进入 app 容器。

2. depends_on

```
depends_on:
  - mongo
  - redis
```

表示 app 会在 mongo 和 redis 容器启动后再启动。

注意：

`depends_on` 只保证容器启动顺序，不保证 MongoDB / Redis 已经完全准备好。

实际生产中可以加健康检查或在应用里做重连机制。

3. volumes

```
volumes:
  - mongo_data:/data/db
```

表示 MongoDB 数据会保存到 Docker volume。

如果不挂载 volume，容器删除后数据可能丢失。

十一、启动项目

在项目根目录执行：

```
docker compose up -d
```

第一次会：

1. 拉取 mongo 镜像
2. 拉取 redis 镜像
3. 构建 app 镜像
4. 启动三个容器

查看容器：

```
docker compose ps
```

或者：

```
docker ps
```

查看日志：

```
docker compose logs -f
```

查看 app 日志：

```
docker compose logs -f app
```

十二、测试接口

健康检查：

```
curl http://localhost:3000/api/health
```

返回类似：

```
{
  "code": 0,
  "message": "ok",
  "env": "production",
  "mongo": "connected",
  "redis": "connected",
  "time": "2026-05-28T10:00:00.000Z"
}
```


创建用户：

```
curl -X POST http://localhost:3000/api/users \
-H "Content-Type: application/json" \
-d "{\"username\":\"张三\",\"age\":20}"
```

获取用户：

```
curl http://localhost:3000/api/users
```

第一次：

```
{
  "cache": false
}
```

第二次：

```
{
  "cache": true
}
```

说明 Redis 缓存生效。

十三、常用 Docker Compose 命令

1. 启动

```
docker compose up -d
```

2. 查看状态

```
docker compose ps
```

3. 查看日志

```
docker compose logs -f
```

查看某个服务：

```
docker compose logs -f app
```

4. 停止服务

```
docker compose stop
```

5. 启动已停止服务

```
docker compose start
```

6. 停止并删除容器

```
docker compose down
```

注意：默认不会删除 volume。

7. 停止并删除容器和数据卷

```
docker compose down -v
```

危险：这会删除 MongoDB / Redis 数据。

8. 重新构建

修改 Dockerfile 或 package.json 后：

```
docker compose up -d --build
```

9. 进入 app 容器

```
docker exec -it node_app sh
```

如果是 Debian/Ubuntu 镜像，可能用：

```
docker exec -it node_app bash
```

10. 进入 MongoDB 容器

```
docker exec -it node_mongo mongosh
```

11. 进入 Redis 容器

```
docker exec -it node_redis redis-cli
```

测试：

```
PING
```

返回：

```
PONG
```

十四、开发环境 docker-compose 配置

上面的配置更偏生产环境。

开发环境通常希望：

- 代码修改后自动生效
- 使用 nodemon
- 挂载本地代码到容器

可以创建：

```
docker-compose.dev.yml
```

内容：

```
services:
  app:
    image: node:20-alpine
    container_name: node_app_dev
    working_dir: /app
    command: sh -c "npm install && npm run dev"
    ports:
```

```
- "3000:3000"
environment:
  NODE_ENV: development
  PORT: 3000
  MONGO_URL: mongodb://mongo:27017/node_docker_demo_dev
  REDIS_URL: redis://redis:6379
volumes:
  - ./app
  - /app/node_modules
depends_on:
  - mongo
  - redis

mongo:
  image: mongo:7
  container_name: node_mongo_dev
  restart: always
  ports:
    - "27017:27017"
  volumes:
    - mongo_dev_data:/data/db

redis:
  image: redis:7-alpine
  container_name: node_redis_dev
  restart: always
  ports:
    - "6379:6379"
  volumes:
    - redis_dev_data:/data

volumes:
  mongo_dev_data:
  redis_dev_data:
```

启动开发环境：

```
docker compose -f docker-compose.dev.yml up -d
```

查看日志：

```
docker compose -f docker-compose.dev.yml logs -f app
```

停止：

```
docker compose -f docker-compose.dev.yml down
```

十五、生产环境优化 Dockerfile

生产环境建议使用更规范的 Dockerfile：

```
FROM node:20-alpine

ENV NODE_ENV=production

WORKDIR /app

COPY package*.json ./

RUN npm ci --omit=dev

COPY . .

EXPOSE 3000

USER node

CMD ["node", "server.js"]
```

说明：

```
RUN npm ci --omit=dev
```

`npm ci` 会根据 `package-lock.json` 精确安装依赖，适合 CI/CD 和生产构建。

```
USER node
```

使用非 root 用户运行 Node.js，更安全。

十六、TypeScript 项目的 Dockerfile

如果你上一课已经把项目改成 TypeScript，生产环境需要先编译：

项目结构：

```
src
dist
package.json
tsconfig.json
```

推荐 Dockerfile：

```
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./
COPY tsconfig.json ./

RUN npm ci
```

```
COPY src ./src

RUN npm run build

FROM node:20-alpine

ENV NODE_ENV=production

WORKDIR /app

COPY package*.json ./

RUN npm ci --omit=dev

COPY --from=builder /app/dist ./dist

EXPOSE 3000

USER node

CMD ["node", "dist/server.js"]
```

这叫 **多阶段构建**。

好处：

- 构建阶段安装 TypeScript 等开发依赖
- 运行阶段只保留生产依赖和编译后的 JS
- 镜像更小
- 更安全

十七、TypeScript docker-compose.yml 示例

```
services:
  app:
    build: .
    container_name: node_ts_app
    restart: always
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
      PORT: 3000
      MONGO_URL: mongodb://mongo:27017/node_ts_demo
      REDIS_URL: redis://redis:6379
      JWT_SECRET: your_secret_key
      JWT_EXPIRES_IN: 2h
    depends_on:
      - mongo
      - redis
```

```
mongo:
  image: mongo:7
  container_name: node_ts_mongo
  restart: always
  volumes:
    - mongo_data:/data/db

redis:
  image: redis:7-alpine
  container_name: node_ts_redis
  restart: always
  volumes:
    - redis_data:/data

volumes:
  mongo_data:
  redis_data:
```

十八、环境变量单独放到 .env

Docker Compose 支持读取 `.env`。

项目根目录：

```
.env
```

```
NODE_ENV=production
PORT=3000
MONGO_DATABASE=node_docker_demo
JWT_SECRET=your_secret_key
JWT_EXPIRES_IN=2h
```

`docker-compose.yml`：

```
services:
  app:
    build: .
    ports:
      - "${PORT}:3000"
    environment:
      NODE_ENV: ${NODE_ENV}
      PORT: 3000
      MONGO_URL: mongodb://mongo:27017/${MONGO_DATABASE}
      REDIS_URL: redis://redis:6379
      JWT_SECRET: ${JWT_SECRET}
      JWT_EXPIRES_IN: ${JWT_EXPIRES_IN}
```

注意：

- Compose 的 `.env` 是给 docker-compose 变量替换用的
- Node.js 里的 `process.env` 来自 `environment`
- 不要把生产 `.env` 提交到 Git

`.gitignore` :

```
node_modules
.env
dist
logs
coverage
```

十九、MongoDB 设置账号密码

生产环境不建议 MongoDB 无密码运行。

`docker-compose.yml` :

```
services:
  mongo:
    image: mongo:7
    container_name: node_mongo
    restart: always
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: example_password
    volumes:
      - mongo_data:/data/db
```

Node.js 连接地址:

```
MONGO_URL: mongodb://root:example_password@mongo:27017/node_docker_demo?authSource=admin
```

完整 app 环境变量:

```
environment:
  MONGO_URL: mongodb://root:example_password@mongo:27017/node_docker_demo?authSource=admin
```

二十、Redis 设置密码

生产环境 Redis 也建议设置密码。

```
redis:
  image: redis:7-alpine
  container_name: node_redis
```



```
restart: always
command: redis-server --requirepass example_redis_password
volumes:
  - redis_data:/data
```

Node.js 连接地址:

```
REDIS_URL: redis://:example_redis_password@redis:6379
```

注意格式:

```
redis://:密码@主机:端口
```

二十一、更安全的生产 compose 示例

```
services:
  app:
    build: .
    container_name: node_app
    restart: always
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
      PORT: 3000
      MONGO_URL: mongodb://root:example_mongo_password@mongo:27017/node_docker_demo?
authSource=admin
      REDIS_URL: redis://:example_redis_password@redis:6379
      JWT_SECRET: your_long_random_secret
      JWT_EXPIRES_IN: 2h
    depends_on:
      - mongo
      - redis

  mongo:
    image: mongo:7
    container_name: node_mongo
    restart: always
    environment:
      MONGO_INITDB_ROOT_USERNAME: root
      MONGO_INITDB_ROOT_PASSWORD: example_mongo_password
    volumes:
      - mongo_data:/data/db

  redis:
    image: redis:7-alpine
    container_name: node_redis
    restart: always
    command: redis-server --requirepass example_redis_password
    volumes:
      - redis_data:/data
```

```
volumes:
  mongo_data:
  redis_data:
```

生产中建议不要暴露 MongoDB 和 Redis 到公网，所以这里没有写：

```
ports:
  - "27017:27017"
  - "6379:6379"
```

这样它们只能被 Docker 内部网络中的 app 访问。

二十二、配合 Nginx 反向代理

生产环境通常不直接让用户访问：

```
http://服务器IP:3000
```

而是：

```
http://你的域名.com
https://你的域名.com
```

Nginx 可以装在宿主机，也可以用 Docker 容器。

方式一：宿主机 Nginx 代理 Docker app

Node app 映射了：

```
ports:
  - "3000:3000"
```

Nginx 配置：

```
server {
  listen 80;
  server_name example.com;

  location / {
    proxy_pass http://127.0.0.1:3000;

    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
    proxy_set_header X-Forwarded-Proto $scheme;
  }
}
```

方式二：Nginx 也放进 Docker Compose

创建：

nginx.conf

```
events {}

http {
    server {
        listen 80;

        server_name localhost;

        location / {
            proxy_pass http://app:3000;

            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}
```

docker-compose.yml 增加 nginx：

```
services:
  nginx:
    image: nginx:alpine
    container_name: node_nginx
    restart: always
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - app

  app:
    build: .
    container_name: node_app
    restart: always
    environment:
      NODE_ENV: production
      PORT: 3000
```

```
MONGO_URL: mongodb://mongo:27017/node_docker_demo
REDIS_URL: redis://redis:6379
depends_on:
  - mongo
  - redis

mongo:
  image: mongo:7
  volumes:
    - mongo_data:/data/db

redis:
  image: redis:7-alpine
  volumes:
    - redis_data:/data

volumes:
  mongo_data:
  redis_data:
```

注意：这时 app 可以不暴露 3000 给宿主机：

```
# 不写 ports 也可以
```

因为 Nginx 和 app 在同一个 Docker 网络中，可以通过：

```
http://app:3000
```

访问。

二十三、上传文件目录 volumes

如果项目有文件上传，例如：

```
uploads
```

Docker 容器删除后，容器内文件会丢失。

所以要挂载 volume。

```
services:
  app:
    build: .
    volumes:
      - uploads_data:/app/uploads

volumes:
  uploads_data:
```

或者挂载到宿主机目录：

```
services:
  app:
    build: .
    volumes:
      - ./uploads:/app/uploads
```

两种区别：

方式	特点
Docker volume	Docker 管理，适合生产
宿主机目录	文件直观看得到，适合开发或简单部署

二十四、查看 Docker volume

查看所有 volume：

```
docker volume ls
```

查看某个 volume：

```
docker volume inspect node-docker-demo_mongo_data
```

删除无用 volume：

```
docker volume prune
```

注意：删除 volume 可能导致数据库数据丢失。

二十五、项目更新部署流程

如果代码更新了：

```
git pull
docker compose up -d --build
```

查看日志：

```
docker compose logs -f app
```

如果只改了环境变量：

```
docker compose up -d
```

如果要重启：

```
docker compose restart app
```

二十六、Docker 日志查看

查看所有服务日志：

```
docker compose logs -f
```

查看 app：

```
docker compose logs -f app
```

查看最近 100 行：

```
docker compose logs --tail=100 app
```

查看 mongo：

```
docker compose logs -f mongo
```

查看 redis：

```
docker compose logs -f redis
```

二十七、Docker 容器内排查问题

进入 app 容器：

```
docker exec -it node_app sh
```

查看环境变量：

```
env
```

测试能否访问 MongoDB：

```
ping mongo
```

注意 alpine 镜像可能没有 ping。

可以直接看应用日志，或者临时安装工具不太推荐。

测试 Redis：

```
docker exec -it node_redis redis-cli
```

如果 Redis 有密码：

```
docker exec -it node_redis redis-cli -a example_redis_password
```

二十八、健康检查 healthcheck

可以给服务添加健康检查。

例如 Redis：

```
redis:
  image: redis:7-alpine
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5
```

MongoDB：

```
mongo:
  image: mongo:7
  healthcheck:
    test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
    interval: 10s
    timeout: 5s
    retries: 5
```

app：

```
app:
  build: .
  healthcheck:
    test: ["CMD", "wget", "-q0-", "http://localhost:3000/api/health"]
    interval: 30s
    timeout: 5s
    retries: 3
```

注意：node:alpine 默认不一定有 wget，如果没有，需要安装或换其他检查方式。

二十九、Docker Compose 网络

Docker Compose 默认会创建一个网络。

同一个 docker-compose.yml 里的服务可以通过服务名访问：

```
app -> mongo:27017
app -> redis:6379
nginx -> app:3000
```

这就是为什么连接地址写：

```
mongodb://mongo:27017/db
redis://redis:6379
```

而不是：

```
localhost
```

三十、Docker 部署常见错误

1. app 连接 MongoDB 失败

错误：

```
ECONNREFUSED 127.0.0.1:27017
```

原因：在容器里用了 localhost。

错误：

```
MONGO_URL=mongodb://127.0.0.1:27017/node_demo
```

正确：

```
MONGO_URL=mongodb://mongo:27017/node_demo
```

2. app 连接 Redis 失败

错误：


```
ECONNREFUSED 127.0.0.1:6379
```

正确：

```
REDIS_URL=redis://redis:6379
```

3. 修改代码后没生效

如果是生产 Dockerfile 构建镜像，需要重新构建：

```
docker compose up -d --build
```

4. MongoDB 数据丢失

原因：没有挂载 volume，容器删除后数据丢失。

正确：

```
volumes:  
  - mongo_data:/data/db
```

5. 端口冲突

错误：

```
port is already allocated
```

说明宿主机端口被占用。

查看：

```
lsof -i:3000
```

解决：

- 停掉占用端口的进程
- 或修改映射端口

例如：

```
ports:
  - "3001:3000"
```

访问：

```
http://localhost:3001
```

6. 容器一直重启

查看日志：

```
docker compose logs -f app
```

常见原因：

- 环境变量缺失
- 数据库连接失败
- 代码启动报错
- package 依赖没安装

三十一、Docker 和 PM2 还需要一起用吗？

Docker 部署时，通常不一定需要 PM2。

原因：

Docker 本身可以通过：

```
restart: always
```

实现容器异常退出后自动重启。

例如：

```
app:
  restart: always
```

但是 PM2 也有价值：

- 多进程 cluster
- 应用日志管理
- 进程监控

常见选择：

简单项目

```
Docker restart: always + Node.js 单进程
```

即可。

高并发生产项目

可以考虑：

```
Docker + PM2 cluster
```

但更常见的云原生方式是：

```
多个容器副本 + 负载均衡
```

初学阶段建议：

```
Docker Compose + restart: always
```

三十二、TypeScript + Docker + MongoDB + Redis 完整结构

```
node-ts-docker-demo
├── src
│   ├── app.ts
│   ├── server.ts
│   ├── config
│   │   ├── db.ts
│   │   └── redis.ts
│   ├── routes
│   ├── controllers
│   ├── models
│   └── middlewares
├── dist
├── package.json
├── package-lock.json
├── tsconfig.json
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
└── .env
```

生产启动：

```
docker compose up -d --build
```

查看：

```
docker compose ps
docker compose logs -f app
```

三十三、本课重点

这一课你需要掌握：

1. Docker 可以统一项目运行环境
2. Dockerfile 用来构建 Node.js 镜像
3. docker-compose.yml 用来编排多个服务
4. Node.js、MongoDB、Redis 可以用 Compose 一键启动
5. Docker 容器中不要用 localhost 连接其他容器
6. 服务之间通过服务名访问：

```
mongo
redis
app
```

7. MongoDB 数据要挂载 volume：

```
mongo_data:/data/db
```

8. Redis 数据也可以挂载 volume：

```
redis_data:/data
```

9. 生产环境不建议暴露 MongoDB 和 Redis 端口到公网
10. 修改代码后需要：

```
docker compose up -d --build
```

11. 查看日志：

```
docker compose logs -f app
```

12. TypeScript 项目建议使用多阶段构建
-

课后练习

基于你之前的 MVC 或 TypeScript 项目，完成 Docker 化部署。

1. 创建 Dockerfile

要求：

- 使用 `node:20-alpine`
 - 设置工作目录 `/app`
 - 安装生产依赖
 - 暴露 3000 端口
 - 启动项目
-

2. 创建 docker-compose.yml

包含三个服务：

```
app
mongo
redis
```

要求：

- app 连接 MongoDB
 - app 连接 Redis
 - MongoDB 使用 volume 保存数据
 - Redis 使用 volume 保存数据
 - app 对外暴露 3000 端口
-

3. 修改环境变量

Docker 中使用：

```
MONGO_URL=mongodb://mongo:27017/你的数据库名
REDIS_URL=redis://redis:6379
```

4. 启动测试

```
docker compose up -d --build
```

测试：

```
curl http://localhost:3000/api/health
```

5. 验证数据库和缓存

实现或测试：

- 新增用户写入 MongoDB
 - 用户列表缓存到 Redis
 - 第二次请求从 Redis 读取
-

下一课建议学习：

Node.js 第十九课：CI/CD 自动化部署 GitHub Actions 入门。

还要继续学吗

可以继续学，但不一定要按课一直往后堆。你现在已经学到第 18 课，覆盖了 Node.js 后端开发的大部分主线：

- Node.js 基础模块
- Express
- 路由和中间件
- 数据库 MySQL / MongoDB
- 登录注册 JWT
- 文件上传
- 部署
- MVC 项目结构
- 参数校验
- 日志和性能
- Redis
- Jest 测试
- TypeScript
- Docker

这些已经足够你开始做真实项目了。

我建议你接下来有两个方向

方向一：继续学进阶内容

如果你想继续系统学习，可以继续学：

1. **CI/CD 自动化部署 GitHub Actions**
2. **Swagger / OpenAPI 接口文档**
3. **WebSocket 实时通信**
4. **消息队列 BullMQ / RabbitMQ**
5. **权限系统 RBAC**

6. 支付接口
7. Redis 深入
8. NestJS 框架
9. 微服务基础
10. 高并发和性能优化

适合目标：

想做中大型后端项目，或者准备后端工程师工作。

方向二：先暂停新知识，开始做项目

我更推荐你现在开始做一个完整项目。

因为学到这里，如果不做项目，很容易：

- 知识点记不牢
- 不知道怎么组合使用
- 遇到真实问题不会排查
- 学了很多但没有作品

你可以做这些项目：

项目 1：用户后台管理系统

功能：

- 注册登录
 - JWT 鉴权
 - 用户列表
 - 角色权限
 - 文件上传头像
 - MongoDB / MySQL
 - 参数校验
 - 日志
 - Docker 部署
-

项目 2：商品订单系统

功能：

- 商品 CRUD
- 商品图片上传
- 商品分类
- 用户登录
- 下单

- 订单列表
 - Redis 缓存商品列表
 - Jest 测试
 - Docker 部署
-

项目 3：博客 API 系统

功能：

- 登录注册
 - 文章发布
 - 文章分类
 - 标签
 - 评论
 - 点赞
 - 图片上传
 - 搜索
 - 分页
 - 接口文档
-

我的建议

如果你是初学者，我建议：

先别继续学第 19 课，先做一个完整 Express + MongoDB + Redis + JWT 项目。

做项目时，你会真正理解：

- 路由怎么拆
 - Controller 怎么写
 - Model 怎么设计
 - Token 怎么鉴权
 - Redis 什么时候用
 - 错误怎么统一处理
 - Docker 怎么部署
-

推荐下一步路线

你可以这样安排：

第一步：做项目

做一个：

Node.js 商品管理 API 项目

包含：

- 用户注册登录
- JWT 鉴权
- 商品 CRUD
- 商品图片上传
- 商品列表分页
- Redis 缓存
- Joi 参数校验
- Winston 日志
- Jest 接口测试
- Docker 部署

第二步：补进阶内容

项目做完后再继续学：

1. GitHub Actions 自动部署
2. Swagger 接口文档
3. WebSocket
4. RBAC 权限系统
5. NestJS

如果你想继续学，我可以继续讲

下一课是：

Node.js 第十九课：CI/CD 自动化部署 GitHub Actions 入门

但如果你想做项目，我也可以带你从 0 开始搭建：

Node.js 实战项目：商品管理系统 API

我建议你选这个。

你可以回复我：

1. 继续第十九课
2. 开始实战项目
3. 帮我规划一个 Node.js 项目
4. 先复习前面内容